CA 9P325 - Full Stack Development Lab
List of Practicals with Solutions

HTML5, CSS3 and JavaScript

1. Develop a web application using HTML, CSS, and JavaScript to create a list dynamically, manipulate it, and view the number of elements in it dynamically. Create a button dynamically, and make it handle an event dynamically.
2. Develop a web application using HTML, CSS, and JavaScript to create a text, paragraph, and button dynamically, and make the button handle an event dynamically to find the factorial value of a number as user input using recursion. Use the input function in JavaScript to get the user's input.
3. Develop a web application using HTML, CSS, and JavaScript to create a text, paragraph, and button dynamically, and make the button handle an event dynamically to generate the Fibonacci series of a number of terms as user input using recursion. Use the input function in JavaScript to get the user's input.
4. Develop a web application using HTML, CSS, and JavaScript to display the current age of a person in years, months, and days for his/her date of birth. Use asynchronous and arrow functions in ES6 JavaScript.
5. Develop a web application using HTML, CSS, and JavaScript to convert the decimal number (including fractional part) into binary, octal, and hexadecimal, and vice versa. Use asynchronous and arrow functions in ES6 JavaScript.
6. Develop a Simple Web Application to Perform Arithmetic Operations (addition, subtraction, multiplication and Division) using Buttons, Text Elements, addEventListener() and Arrow functions in JavaScript and use only Flexbox Layout in CSS.
7. Develop a Simple Web Application to Perform Arithmetic Operations (addition, subtraction, multiplication and Division) using Buttons, Text Elements, addEventListener(), classes, and arrow member functions in JavaScript and use only Grid Layout in CSS.

RactJS

8. Develop a ReactJS application to perform simple arithmetic operations such as addition, subtraction, multiplication, and division, on the two given numbers through HTML input controls and display the output in the HTML paragraph element.
9. Develop a ReactJS application to calculate factorials and the fibonacci series using recursion based on user input through an HTML input control and display the output in an HTML paragraph element. (NOTE: Only one paragraph element to display either the factorial value or the fibonacci series based on the user navigation).
10. Develop a simple CRUD application (Create, Read, Update, and Delete) using ReactJS for managing users.
11. Develop a simple CRUD application (Create, Read, Update, and Delete) using ReactJS for student mark list generation with Register Number, Name, Marks of Subject 1, Subject 2, Subject 3, Subject 4, Subject 5, Result, CGPA, and Grade. Result, CGPA, and Grade are calculated whenever a new student record is inserted or updated in the MySQL Database.

12. Develop a simple CRUD application (Create, Read, Update, and Delete) using ReactJS for employee payroll generation with Employee ID, Name, Basic Pay, DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay. DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay are calculated whenever a new employee record is inserted or updated in the MySQL Database.

NodeJS

13. Develop a NodeJS web application with an HTML/CSS UI that performs arithmetic calculations using a custom module. The application should capture user input, process the selected operation (Addition, Subtraction, Multiplication, Division) upon button submission, and display the result in a <p> tag on the same screen.
14. Develop a NodeJS Application using HTML and CSS to display all or specific records in a web page from a MySQL Database table based on user inputs through HTML input and submit button controls.
15. Develop a simple CRUD application (Create, Read, Update, and Delete) using only NodeJS with HTML5 and CSS3 for employee payroll generation with Employee ID, Name, Basic Pay, DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay. DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay are calculated whenever a new employee record is inserted or updated in the MySQL Database.
16. Develop a simple CRUD application (Create, Read, Update, and Delete) using only NodeJS with HTML5 and CSS3 for employee payroll generation with Employee ID, Name, Basic Pay, DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay. DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay are calculated whenever a new employee record is inserted or updated in the MongoDB.

INDEX

S.No.	Title	Page No.
	HTML5, CSS3 and JavaScript	
1	Dynamic List Manipulation with JavaScript	
2	Factorial Calculator Using Recursion with Dynamic Controls	
3	Fibonacci Series Using Recursion with Dynamic Controls	
4	Age Calculator Using ES6 Asynchronous Functions	
5	Number System Converter Using ES6	
6	Arithmetic Operations Using ES6 and Flexbox	
7	Arithmetic Operations Using JavaScript Classes and Grid Layout	
	ReactJS	
8	Arithmetic Operations Using ReactJS	
9	Factorial and Fibonacci Using Recursion and ReactJS	
10	User Management CRUD Application Using ReactJS	
11	Student Mark List CRUD Application Using ReactJS and MySQL	
12	Employee Payroll CRUD Application Using ReactJS and MySQL	
	NodeJS	
13	Arithmetic Calculator Using NodeJS Custom Module	
14	MySQL Records Retrieval Using NodeJS	
15	Employee Payroll CRUD Using NodeJS and MySQL	
16	Employee Payroll CRUD Using NodeJS and MongoDB	

Ex. No.: 1
Date:

Dynamic List Manipulation with JavaScript

Aim:

To develop a web application using HTML, CSS, and JavaScript to dynamically create and manipulate a list, view the current number of elements in the list, create a button programmatically at runtime, and attach a dynamic event handler to it.

Procedure:

Step 1: Start the process.

Step 2: Create an HTML5 structure containing a heading, an initial unordered list (myList) with default elements ("Coffee", "Tea"), instruction text paragraphs, and multiple control buttons.

Step 3: Define a JavaScript function addElement() that prompts the user for text input, dynamically creates a new <li> element containing that text, and appends it to the list.

Step 4: Define a JavaScript function removeElement() that accesses the unordered list and removes its last child element.

Step 5: Define a JavaScript function removeAllElement() that uses a while loop to continuously remove all child nodes from the list until it is empty.

Step 6: Define a JavaScript function elementCount() that counts the current number of child elements in the list, dynamically creates a paragraph element to display this count, and appends it to the document body.

Step 7: Define a JavaScript function addButton() that dynamically creates a new input button element, assigns its type, value, and an onclick event attribute, and appends it to the document body.

Step 8: Define a JavaScript function hello() that triggers an alert box displaying a welcome message when the dynamically created button is clicked.

Step 9: Open the HTML file in a web browser to interactively append, remove, count list items, and dynamically generate new buttons with event handlers.

Step 10: Stop the process.

DOM-JavaScript.html

```
<!DOCTYPE html>
<html>
   <title>Create a list dynamically, manipulate it, and view the number of
elements in it
   dynamically. Create a button dynamically, and make it handle an event
dynamically.</title>
<body>
   <p><h1>Create a list dynamically, manipulate it, and view the number of
      elements in it dynamically. Create a button dynamically, and make it
      handle an event dynamically.</h1></p>
```

```
<ul id="myList">
  <li>Coffee</li>
  <li>Tea</li>
</ul>

<p>Click "Append" to append an item to the end of the list:</p>
<p>Click "Remove" to remove an item to the end of the list:</p>
<p>Click "Remove All" to remove all items from the list:</p>
<p>Click "Element Count" to get the length of the child in the list:</p>

<button onclick="addElement()">Append</button>
<button onclick="removeElement()">Remove</button>
<button onclick="removeAllElement()">Remove All</button>
<button onclick="elementCount()">Element Count</button>
<button onclick="addButton()">Create a New Button to handle an
Event</button>

<script>
function addElement() {
      // Create an "li" node:
      const node = document.createElement("li");
      // Create a text node:
      const textnode = document.createTextNode(prompt("Enter an item to be
added into the list:"));
      // Append the text node to the "li" node:
      node.appendChild(textnode);
      // Append the "li" node to the list:
      document.getElementById("myList").appendChild(node);
}

function removeElement() {
      // Remove the last element from the list:
      const list = document.getElementById("myList");
      list.removeChild(list.lastElementChild);
}

function removeAllElement() {
    const list = document.getElementById("myList");
    while (list.hasChildNodes()) {
        list.removeChild(list.firstChild);
    }
}

function elementCount() {
    const list = document.getElementById("myList");
    //alert("Number of Elements in the List is "+list.children.length);
    // Create element:
    const para = document.createElement("p");
    para.innerText = "Number of Elements in the List is "+list.children.length;
    // Append to body:
```

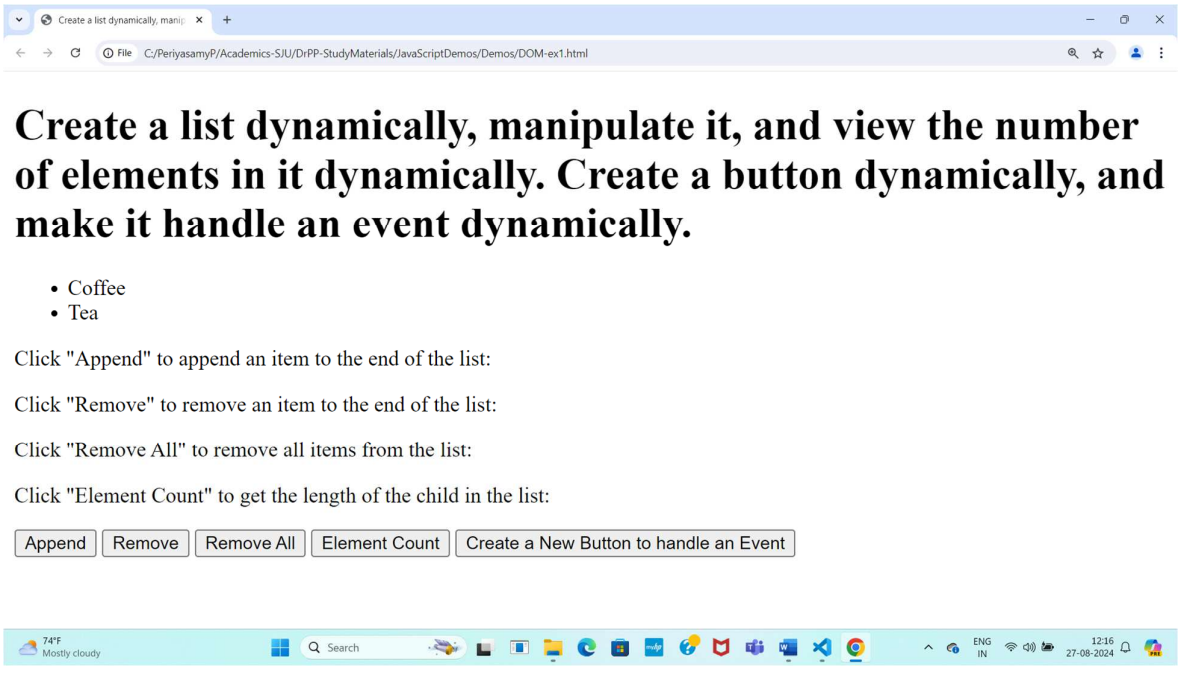
```
    document.body.appendChild(para);
}

function addButton() {
  var x = document.createElement("INPUT");
  x.setAttribute("type", "button");
  x.setAttribute("value", "Click me");
  x.setAttribute("onclick", "hello()");
  document.body.appendChild(x);
}

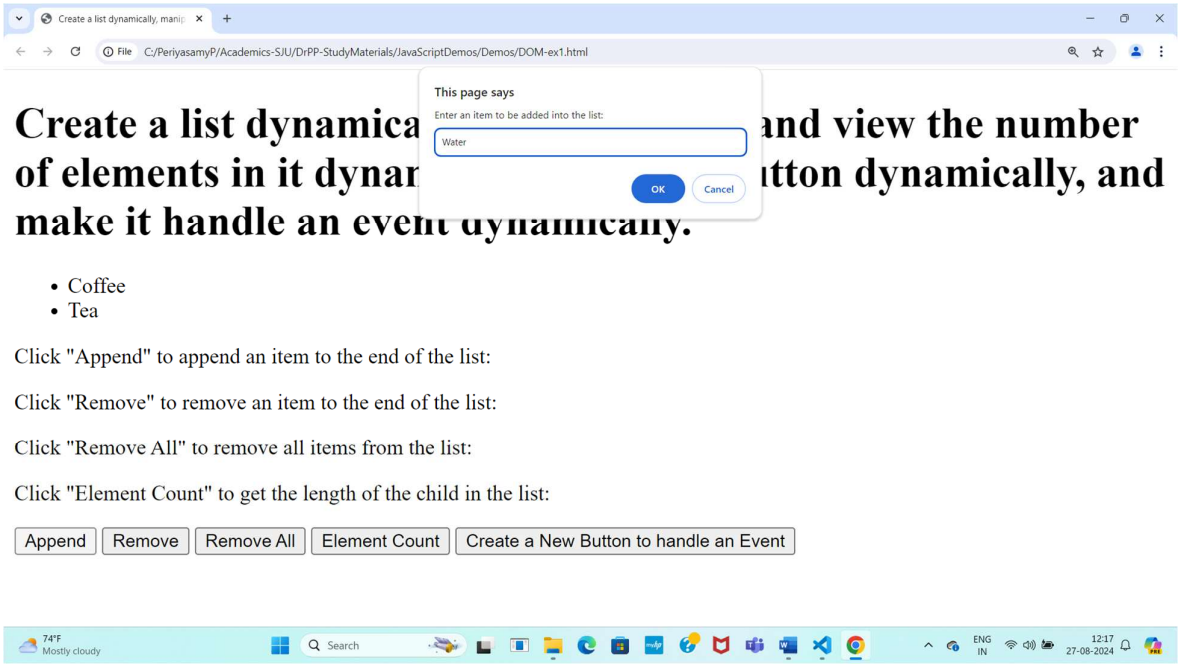
function hello()
{
    alert("Welcome to JavaScript Programming using Document Object
Model!")
}
</script>

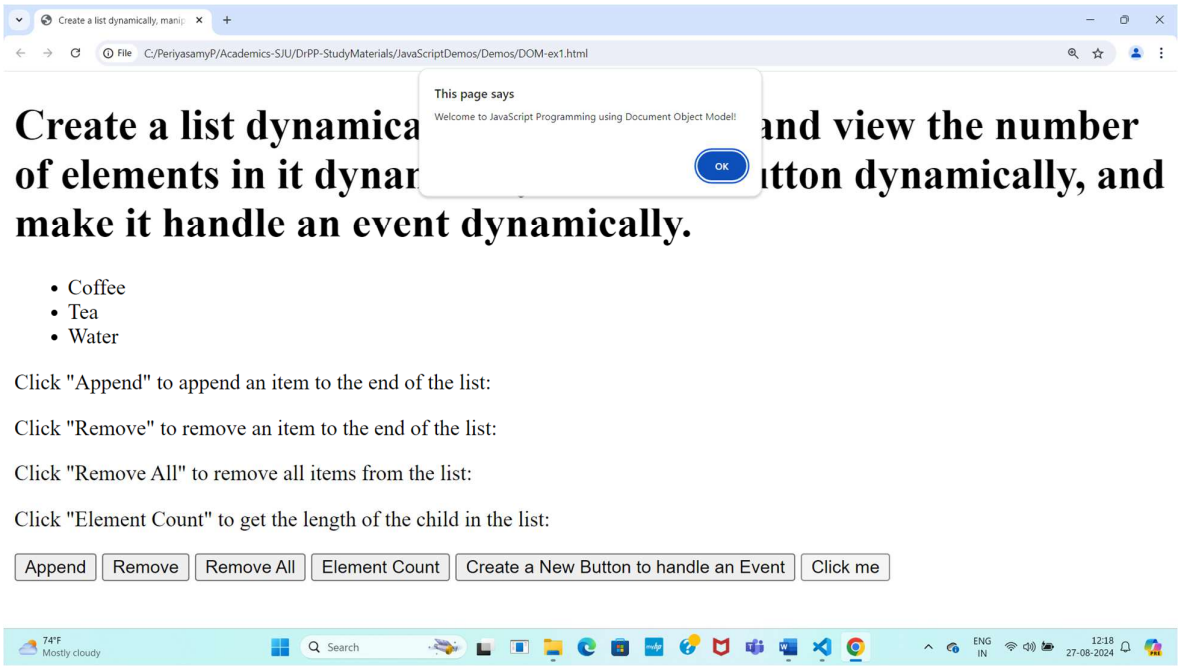
</body>
</html>
```

INPUT



OUTPUT





Result:

The web application was successfully developed using HTML, CSS, and JavaScript. The program successfully demonstrates dynamic list creation, item manipulation (adding or removing individual items and clearing all elements), real-time list counting, and the programmatic creation of a button at runtime with a dynamic event listener using the Document Object Model (DOM).

Ex. No.: 2
Date:

Factorial Calculator Using Recursion with Dynamic Controls

Aim:

To develop a web application using HTML, CSS, and JavaScript to dynamically create text input, paragraph, and button elements, use JavaScript's input function to capture user input, and programmatically attach an event handler to compute the factorial value of a number using recursion.

Procedure:

Step 1: Start the process.

Step 2: Create an HTML5 structure including CSS styles (.button, .text, .mystyle) for layout formatting and interactive trigger elements (div tags).

Step 3: Define a JavaScript function createText() that dynamically creates an <input> element of type text, assigns an ID (number) and CSS class (text), and appends it to the document body.

Step 4: Define a JavaScript function createButton() that dynamically creates an input button element, assigns a value (Factorial Value Computation), a CSS class (button), an onclick event handler calling findFactVal(), and appends it to the document body.

Step 5: Define a JavaScript function createParagraph() that dynamically creates a <p> paragraph element, assigns an ID (result) and styling class (mystyle), and appends it to the document body.

Step 6: Define a recursive helper function factorial(n) that checks if n is 0 or 1 (returning 1) or otherwise returns n * factorial(n - 1).

Step 7: Define the computation function findFactVal() to parse the numeric value entered in the text input, pass it to the recursive factorial() function, and inject the final result inner HTML into the paragraph container.

Step 8: Open the HTML file in a web browser, click the trigger actions sequentially to build the DOM elements, enter a number, click the compute button, and view the output.

Step 9: Stop the process.

DynamicButton-EventHandling-Factorial.html

```
<!DOCTYPE html>
<html>
<head>

<title>Develop a web application using HTML, CSS, and JavaScript to create
a text, paragraph, and button dynamically, and make the button handle an
event dynamically to find the factorial value of a number as user input using
recursion. Use the input function in JavaScript to get the user's input, or
use HTML input controls to collect user input.</title>
```

```
<style>
.button {
  background-color: #04AA6D;
  border: none;
  color: white;
  padding: 15px 32px;
  text-align: center;
  text-decoration: none;
  display: inline-block;
  font-size: 16px;
  margin: 4px 2px;
  cursor: pointer;
}
.text {
  width: 100%;
  padding: 12px 20px;
  margin: 8px 0;
  display: inline-block;
  border: 1px solid red;
  border-radius: 4px;
  box-sizing: border-box;
}
.mystyle {
  width: 75%;
  padding: 12px 20px;
  margin: 8px 0;
  box-sizing: border-box;
  border: 2px solid red;
  border-radius: 4px;
}
</style>

</head>

<body>

<div onclick="createText()">Click here to create a New Text to get
input</div><br>
<div onclick="createButton()">Click here to create a New Button to handle
an Event</div><br>
<div onclick="createParagraph()">Click here to create a New Paragraph to
display result</div><br>

<script>
function createText() {
  var x = document.createElement("INPUT");
  x.setAttribute("type", "text");
  x.setAttribute("id", "number");
  x.setAttribute("class", "text");
```

```
  document.body.appendChild(x);
}

function createButton() {
  var x = document.createElement("INPUT");
  x.setAttribute("type", "button");
  x.setAttribute("class", "button");
  x.setAttribute("value", "Factorial Value Computation");
  x.setAttribute("onclick", "findFactVal()");
  document.body.appendChild(x);
}

function createParagraph() {
  var x = document.createElement("P");
  x.setAttribute("id", "result");
  x.setAttribute("class", "mystyle");
  document.body.appendChild(x);
}

function findFactVal()
{
    var n,f;
    //var n,i,f=1;

    //n=parseInt(prompt("Enter a value to find the factorial value:"));
    n=parseInt(document.getElementById('number').value);

    //for(i=1;i<=n;i++) f=f*i;
    //alert("Factorial Value of "+n+" is "+f);

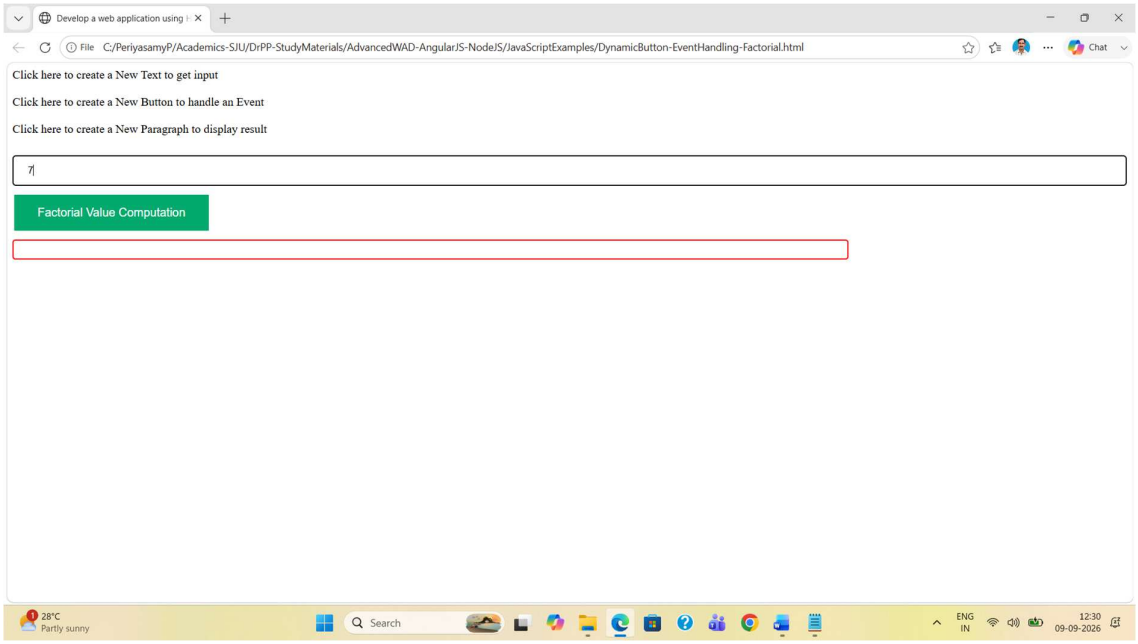
    f=factorial(n);

    document.getElementById('result').innerHTML="Factorial Value of "+n+" is
"+f;
}

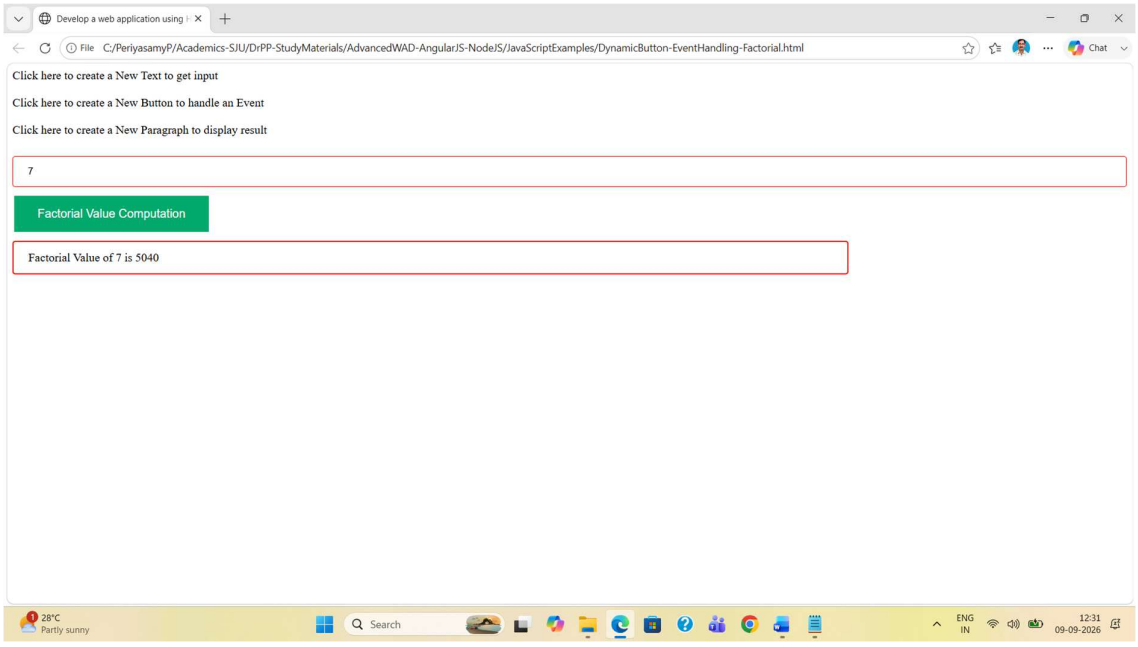
function factorial(n) {
    if (n === 0 || n === 1) {
        return 1;
    } else {
        return (n * factorial(n - 1));
    }
}
</script>

</body>
</html>
```

INPUT



OUTPUT



Result:

The web application was successfully developed using HTML, CSS, and JavaScript. The program successfully demonstrates the dynamic creation of text, paragraph, and button elements, captures user input via JavaScript's prompt() function, programmatically attaches an event listener to the button, and computes the factorial of the input number using recursion.

3. **Develop a web application using HTML, CSS, and JavaScript to create a text, paragraph, and button dynamically, and make the button handle an event dynamically to generate the Fibonacci series of a number of terms as user input using recursion. Use the input function in JavaScript to get the user's input.**

DynamicButton-EventHandling-Fibonacci.html

```
<!DOCTYPE html>
<html>
<head>

<title>Develop a web application using HTML, CSS, and JavaScript to create a text, paragraph, and button dynamically, and make the button handle an event dynamically to generate the Fibonacci series of a number of terms as user input using recursion. Use the input function in JavaScript to get the user's input, or use HTML input controls to collect user input.</title>

<style>
.button {
  background-color: #04AA6D;
  border: none;
  color: white;
  padding: 15px 32px;
  text-align: center;
  text-decoration: none;
  display: inline-block;
  font-size: 16px;
  margin: 4px 2px;
  cursor: pointer;
}
.text {
  width: 100%;
  padding: 12px 20px;
  margin: 8px 0;
  display: inline-block;
  border: 1px solid red;
  border-radius: 4px;
  box-sizing: border-box;
}
.mystyle {
  width: 75%;
  padding: 12px 20px;
  margin: 8px 0;
  box-sizing: border-box;
  border: 2px solid red;
  border-radius: 4px;
}
</style>
```

```
</head>

<body>

<div onclick="createText()">Click here to create a New Text to get
input</div><br>
<div onclick="createButton()">Click here to create a New Button to handle
an Event</div><br>
<div onclick="createParagraph()">Click here to create a New Paragraph to
display result</div><br>

<script>
function createText() {
  var x = document.createElement("INPUT");
  x.setAttribute("type", "text");
  x.setAttribute("id", "number");
  x.setAttribute("class", "text");
  document.body.appendChild(x);
}

function createButton() {
  var x = document.createElement("INPUT");
  x.setAttribute("type", "button");
  x.setAttribute("class", "button");
  x.setAttribute("value", "Fibonacci Series Generation");
  x.setAttribute("onclick", "genFiboSeries()");
  document.body.appendChild(x);
}

function createParagraph() {
  var x = document.createElement("P");
  x.setAttribute("id", "result");
  x.setAttribute("class", "mystyle");
  document.body.appendChild(x);
}

function genFiboSeries()
{
    // take nth term input from the user
    //nTerms=parseInt(prompt("Enter how many fibonacci terms do you want
to generate?"));
    const nTerms = parseInt(document.getElementById('number').value);

    const f = [];

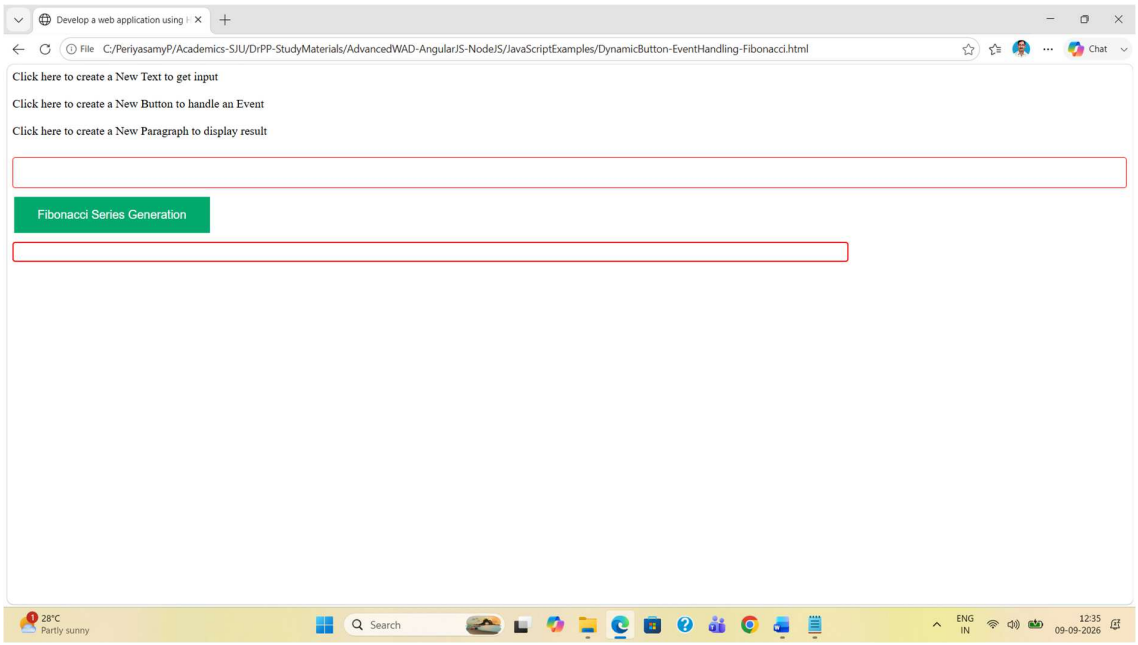
    if(nTerms <= 0)
          alert('Enter a positive integer.');
    else {
         for(let i = 0; i < nTerms; i++) {
```

```
                // Adding element to the array
                 f[i]=fibonacci(i);
            }
          document.getElementById('result').innerHTML = "Fibonacci series of <font size=6 face=verdana color=green>"+nTerms+"</font> terms are <font size=6 face=verdana color=green>"+f+"</font>";
    }
}

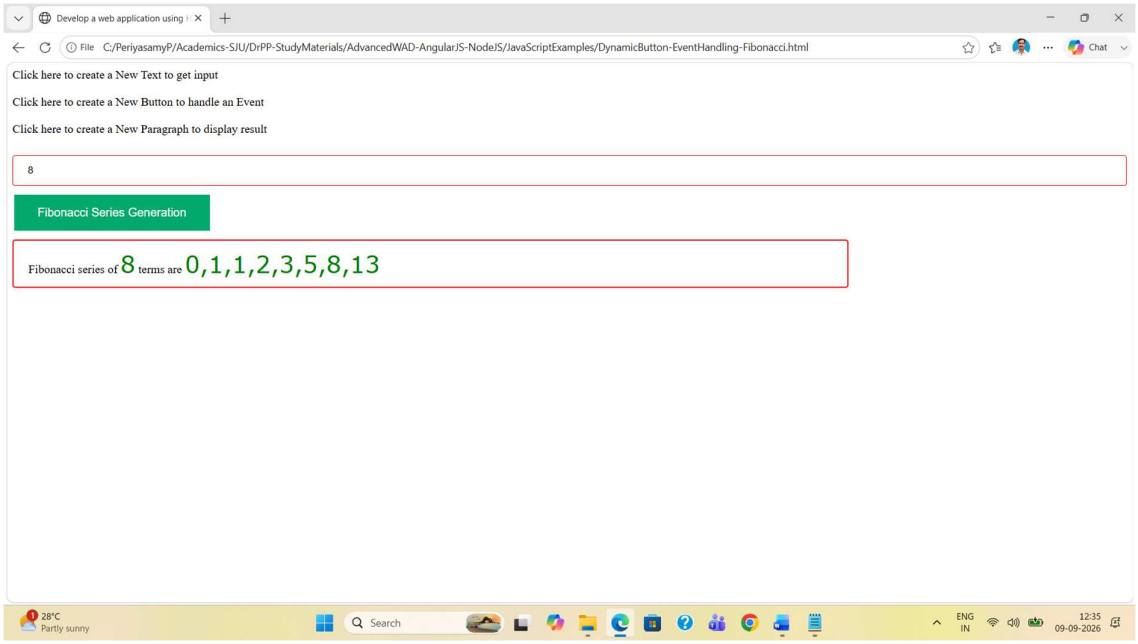
// Function to generate Fibonacci sequence using recursion
function fibonacci(n) {
    if(n < 2)
          return n;
    else
          return fibonacci(n - 1) + fibonacci(n - 2);
}
</script>

</body>
</html>
```

INPUT



OUTPUT



4. Develop a web application using HTML, CSS, and JavaScript to display the current age of a person in years, months, and days for his/her date of birth. Use asynchronous and arrow functions in ES6 JavaScript.

Age-Calculator-Arrow-Asynchronous-Await.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Age Calculator with ES6 Async/Await</title>
    <style>
        :root {
            --primary: #4f46e5;
            --primary-hover: #4338ca;
            --bg-color: #f3f4f6;
            --card-bg: #ffffff;
            --text-color: #1f2937;
            --error-color: #dc2626;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: var(--bg-color);
            color: var(--text-color);
            margin: 0;
            padding: 0;
```

```
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
}

.container {
    background-color: var(--card-bg);
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.05);
    width: 100%;
    max-width: 450px;
    box-sizing: border-box;
}

h2 {
    margin-top: 0;
    text-align: center;
    color: var(--primary);
    margin-bottom: 20px;
}

.form-group {
    margin-bottom: 20px;
}

label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
    font-size: 14px;
}

input {
    width: 100%;
    padding: 10px 12px;
    font-size: 16px;
    border: 1px solid #d1d5db;
    border-radius: 6px;
    box-sizing: border-box;
    outline: none;
    transition: border-color 0.2s;
}

input:focus {
    border-color: var(--primary);
}

button {
```

```
    width: 100%;
    background-color: var(--primary);
    color: white;
    border: none;
    padding: 12px;
    font-size: 16px;
    font-weight: 600;
    border-radius: 6px;
    cursor: pointer;
    transition: background-color 0.2s;
}

button:hover {
    background-color: var(--primary-hover);
}

.error {
    color: var(--error-color);
    font-size: 14px;
    margin-top: 10px;
    text-align: center;
    min-height: 20px;
}

.results {
    margin-top: 25px;
    border-top: 1px solid #e5e7eb;
    padding-top: 20px;
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 12px;
    text-align: center;
}

.result-box {
    background-color: #f9fafb;
    padding: 12px 8px;
    border-radius: 8px;
    border: 1px solid #e5e7eb;
}

.result-box h4 {
    margin: 0 0 5px 0;
    font-size: 12px;
    color: #6b7280;
    text-transform: uppercase;
}

.result-box p {
    margin: 0;
```

```
            font-size: 20px;
            font-weight: bold;
            color: var(--primary);
        }
    </style>
</head>
<body>

    <div class="container">
        <h2>Age Calculator</h2>

        <div class="form-group">
            <label for="dob">Select Your Birth Date:</label>
            <input type="date" id="dob">
        </div>

        <button id="calcBtn">Calculate Age</button>

        <div id="errorMsg" class="error"></div>

        <div class="results">
            <div class="result-box">
                <h4>Years</h4>
                <p id="years">-</p>
            </div>
            <div class="result-box">
                <h4>Months</h4>
                <p id="months">-</p>
            </div>
            <div class="result-box">
                <h4>Days</h4>
                <p id="days">-</p>
            </div>
        </div>
    </div>

    <script>
        // ES6 Arrow Function to calculate precise age breakdown
        const calculatePreciseAge = (birthDate, currentDate) => {
            let years = currentDate.getFullYear() - birthDate.getFullYear();
            let months = currentDate.getMonth() - birthDate.getMonth();
            let days = currentDate.getDate() - birthDate.getDate();

            if (days < 0) {
                months--;
                const prevMonth = new Date(currentDate.getFullYear(),
currentDate.getMonth(), 0);
                days += prevMonth.getDate();
            }
```

```
            if (months < 0) {
                years--;
                months += 12;
            }

            return { years, months, days };
        };

        // Asynchronous arrow function using 'await' to simulate fetching
current date from an API
        const fetchCurrentDateAsync = async () => {
            return await new Promise((resolve) => {
                setTimeout(() => {
                    resolve(new Date());
                }, 300); // Simulate network or asynchronous delay
            });
        };

        // Asynchronous arrow function coordinating validation and
calculation via 'await'
        const computeAgeAsync = async (dateString) => {
            if (!dateString) {
                throw new Error("Please select a valid birth date.");
            }

            const birthDate = new Date(dateString);

            // Using 'await' to pause execution until the asynchronous date
fetch completes
            const today = await fetchCurrentDateAsync();

            if (birthDate > today) {
                throw new Error("Birth date cannot be in the future.");
            }

            return calculatePreciseAge(birthDate, today);
        };

        // Asynchronous event listener utilizing an arrow function and 'await'
        document.getElementById('calcBtn').addEventListener('click', async ()
=> {
            const dobInput = document.getElementById('dob').value;
            const errorDiv = document.getElementById('errorMsg');
            const yearsEl = document.getElementById('years');
            const monthsEl = document.getElementById('months');
            const daysEl = document.getElementById('days');

            errorDiv.textContent = '';
            yearsEl.textContent = '...';
            monthsEl.textContent = '...';
```

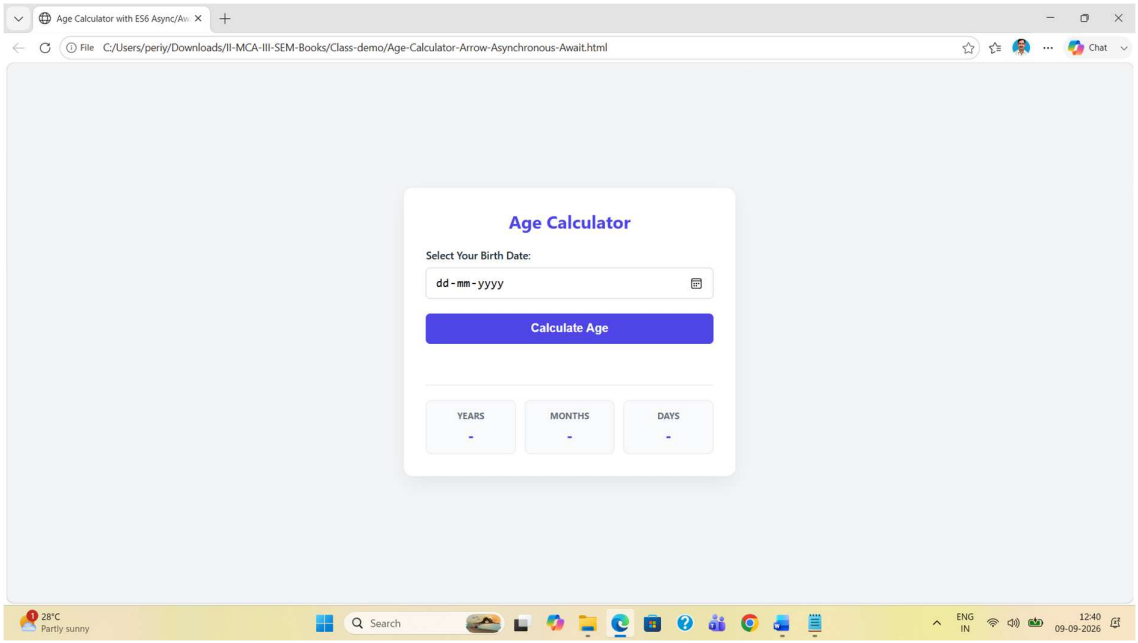
```
            daysEl.textContent = '...';

            try {
                // Using 'await' to wait for the resolution of the asynchronous
computation promise
                const age = await computeAgeAsync(dobInput);

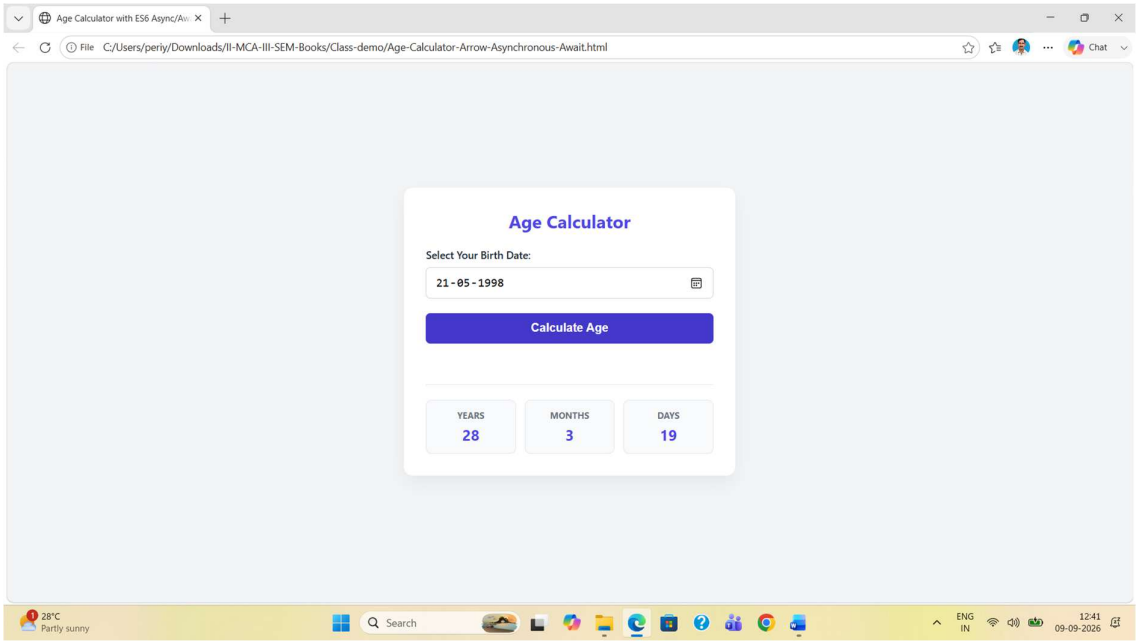
                yearsEl.textContent = age.years;
                monthsEl.textContent = age.months;
                daysEl.textContent = age.days;

            } catch (error) {
                errorDiv.textContent = error.message;
                yearsEl.textContent = '-';
                monthsEl.textContent = '-';
                daysEl.textContent = '-';
            }
        });
    </script>
</body>
</html>
```

INPUT



OUTPUT



5. Develop a web application using HTML, CSS, and JavaScript to convert the decimal number (including fractional part) into binary, octal, and hexadecimal, and vice versa. Use asynchronous and arrow functions in ES6 JavaScript.

Number-System-Conversion-wFractions-Arrow-Asynchronous.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>ES6 Async & Arrow Function Base Converter</title>
    <style>
        :root {
            --primary: #4f46e5;
            --primary-hover: #4338ca;
            --bg-color: #f3f4f6;
            --card-bg: #ffffff;
            --text-color: #1f2937;
            --error-color: #dc2626;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            background-color: var(--bg-color);
            color: var(--text-color);
            margin: 0;
            padding: 0;
```

```
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
}

.container {
    background-color: var(--card-bg);
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.05);
    width: 100%;
    max-width: 500px;
    box-sizing: border-box;
}

h2 {
    margin-top: 0;
    text-align: center;
    color: var(--primary);
    margin-bottom: 20px;
}

.form-group {
    margin-bottom: 20px;
}

label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
    font-size: 14px;
}

input, select {
    width: 100%;
    padding: 10px 12px;
    font-size: 16px;
    border: 1px solid #d1d5db;
    border-radius: 6px;
    box-sizing: border-box;
    outline: none;
    transition: border-color 0.2s;
}

input:focus, select:focus {
    border-color: var(--primary);
}

button {
```

```
    width: 100%;
    background-color: var(--primary);
    color: white;
    border: none;
    padding: 12px;
    font-size: 16px;
    font-weight: 600;
    border-radius: 6px;
    cursor: pointer;
    transition: background-color 0.2s;
}

button:hover {
    background-color: var(--primary-hover);
}

.error {
    color: var(--error-color);
    font-size: 14px;
    margin-top: 10px;
    text-align: center;
    min-height: 20px;
}

.results {
    margin-top: 25px;
    border-top: 1px solid #e5e7eb;
    padding-top: 20px;
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 15px;
}

.result-box {
    background-color: #f9fafb;
    padding: 12px;
    border-radius: 8px;
    border: 1px solid #e5e7eb;
}

.result-box h4 {
    margin: 0 0 5px 0;
    font-size: 13px;
    color: #6b7280;
    text-transform: uppercase;
}

.result-box p {
    margin: 0;
    font-size: 16px;
```

```
            font-weight: bold;
            word-break: break-all;
            color: var(--text-color);
        }
    </style>
</head>
<body>

    <div class="container">
        <h2>Base Converter (ES6)</h2>

        <div class="form-group">
            <label for="inputValue">Enter Number (e.g., 10.25, 1010.1,
FF.A):</label>
            <input type="text" id="inputValue" placeholder="e.g. 10.25, 1010.1">
        </div>

        <div class="form-group">
            <label for="inputBase">Source Base:</label>
            <select id="inputBase">
                <option value="10">Decimal (Base 10)</option>
                <option value="2">Binary (Base 2)</option>
                <option value="8">Octal (Base 8)</option>
                <option value="16">Hexadecimal (Base 16)</option>
            </select>
        </div>

        <button id="convertBtn">Convert Number</button>

        <div id="errorMsg" class="error"></div>

        <div class="results">
            <div class="result-box">
                <h4>Binary</h4>
                <p id="binaryResult">-</p>
            </div>
            <div class="result-box">
                <h4>Octal</h4>
                <p id="octalResult">-</p>
            </div>
            <div class="result-box">
                <h4>Decimal</h4>
                <p id="decimalResult">-</p>
            </div>
            <div class="result-box">
                <h4>Hexadecimal</h4>
                <p id="hexResult">-</p>
            </div>
        </div>
    </div>
```

```
    <script>
        // ES6 Arrow Function for input validation
        const isValidNumberForBase = (str, base) => {
            const parts = str.split('.');
            if (parts.length > 2) return false;
            if (parts.length === 1 && parts[0] === '') return false;
            if (parts.length === 2 && parts[0] === '' && parts[1] === '') return
false;

            const validChars = {
                2: /^[01]*$/,
                8: /^[0-7]*$/,
                10: /^[0-9]*$/,
                16: /^[0-9a-fA-F]*$/
            };

            const regex = validChars[base];
            if (!regex) return false;

            if (parts[0] !== '' && !regex.test(parts[0])) return false;
            if (parts[1] !== undefined && parts[1] !== '' && !regex.test(parts[1]))
return false;

            return true;
        };

        // ES6 Arrow Function for parsing numbers with fractional
components to decimal
        const parseToDecimal = (str, base) => {
            const parts = str.split('.');
            const intPart = parts[0] || '0';
            const fracPart = parts[1] || '';

            const decInt = parseInt(intPart, base);
            if (isNaN(decInt)) return NaN;

            let decFrac = 0;
            for (let i = 0; i < fracPart.length; i++) {
                const digit = parseInt(fracPart[i], base);
                if (isNaN(digit) || digit >= base) return NaN;
                decFrac += digit * Math.pow(base, -(i + 1));
            }

            return decInt + decFrac;
        };

        // ES6 Arrow Function for converting decimal floats to target bases
        const decimalToBase = (decimalNum, targetBase, precision = 8) => {
            if (isNaN(decimalNum)) return 'NaN';
```

```
        const intPart = Math.floor(decimalNum);
        let fracPart = decimalNum - intPart;

        const intStr = intPart.toString(targetBase);

        if (fracPart === 0) return intStr;

        let fracStr = '';
        for (let i = 0; i < precision; i++) {
            fracPart *= targetBase;
            const digit = Math.floor(fracPart);
            fracStr += digit.toString(targetBase);
            fracPart -= digit;
            if (fracPart === 0) break;
        }

        return `${intStr}.${fracStr}`.toUpperCase();
    };

    // Asynchronous Arrow Function using async/await and Promises
    const performConversion = async (value, base) => {
        return new Promise((resolve, reject) => {
            setTimeout(() => {
                const trimmedValue = value.trim();

                if (!trimmedValue) {
                    return reject(new Error("Please enter a value to convert."));
                }

                const numericBase = parseInt(base, 10);

                if (!isValidNumberForBase(trimmedValue, numericBase)) {
                    return reject(new Error(`Invalid characters or format for Base
${numericBase}.`));
                }

                const decimalValue = parseToDecimal(trimmedValue,
numericBase);

                if (isNaN(decimalValue)) {
                    return reject(new Error("Could not parse the provided
number."));
                }

                const results = {
                    binary: decimalToBase(decimalValue, 2),
                    octal: decimalToBase(decimalValue, 8),
                    decimal: decimalValue.toString(10),
                    hex: decimalToBase(decimalValue, 16)
```

```
                };

                resolve(results);
            }, 200);
        });
    };

    // Asynchronous event listener utilizing an arrow function
    document.getElementById('convertBtn').addEventListener('click', async
() => {
        const inputVal = document.getElementById('inputValue').value;
        const inputBase = document.getElementById('inputBase').value;
        const errorDiv = document.getElementById('errorMsg');

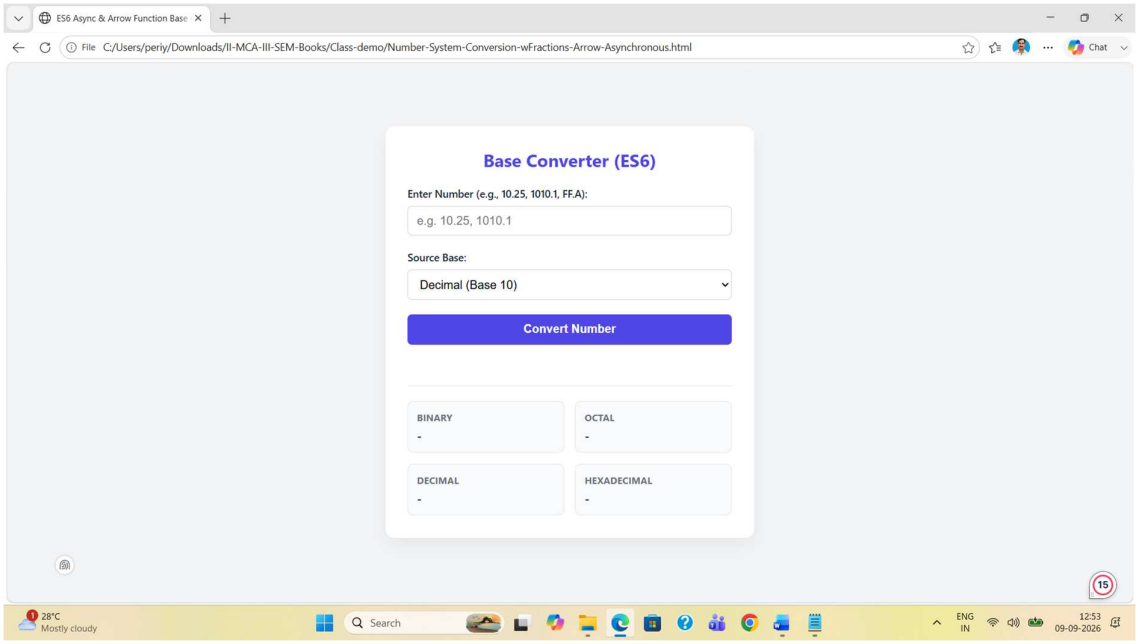
        errorDiv.textContent = '';

        try {
            const results = await performConversion(inputVal, inputBase);

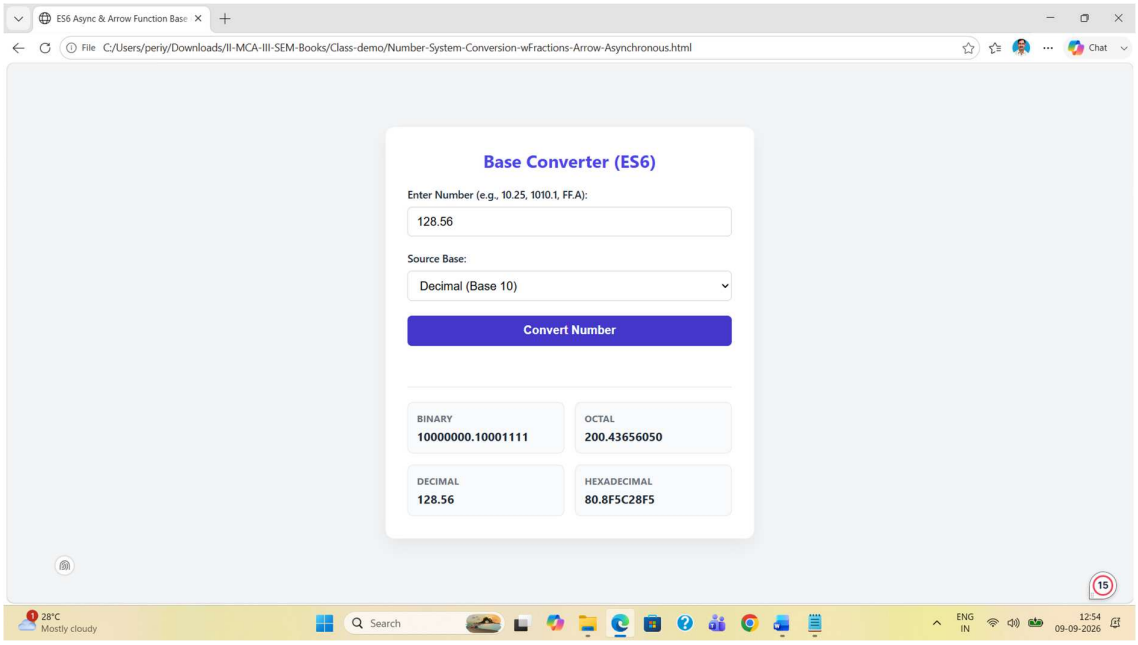
            document.getElementById('binaryResult').textContent =
results.binary;
            document.getElementById('octalResult').textContent =
results.octal;
            document.getElementById('decimalResult').textContent =
results.decimal;
            document.getElementById('hexResult').textContent = results.hex;

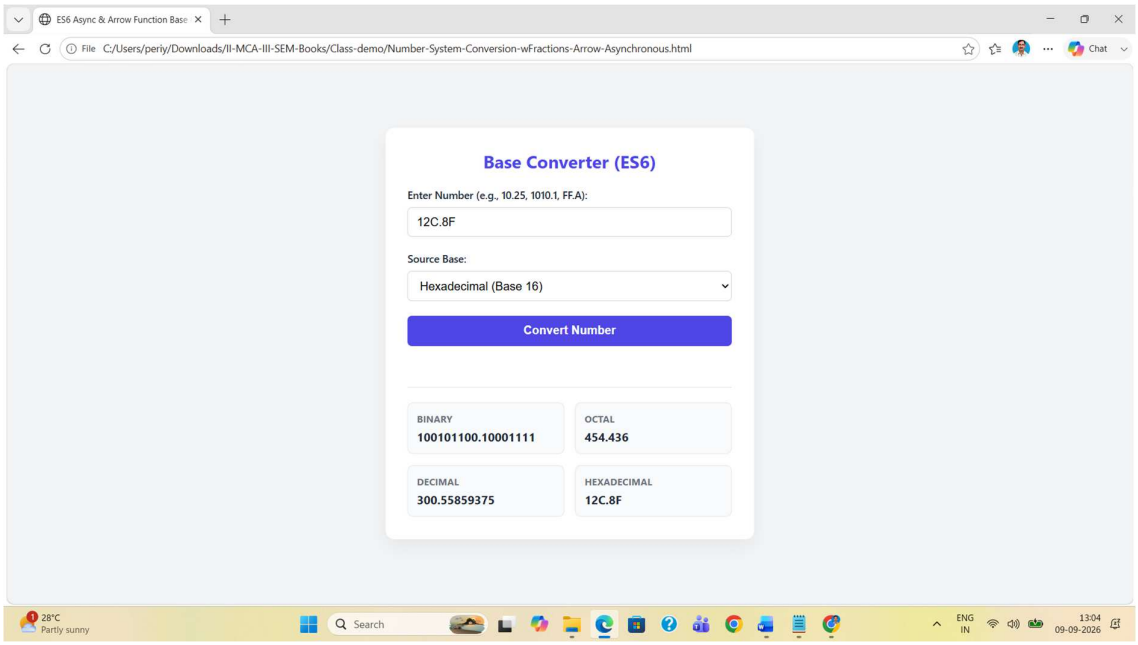
        } catch (error) {
            errorDiv.textContent = error.message;
            document.getElementById('binaryResult').textContent = '-';
            document.getElementById('octalResult').textContent = '-';
            document.getElementById('decimalResult').textContent = '-';
            document.getElementById('hexResult').textContent = '-';
        }
    });
  </script>
</body>
</html>
```

INPUT



OUTPUT





6. Develop a Simple Web Application to Perform Arithmetic Operations (addition, subtraction, multiplication and Division) using Buttons, Text Elements, addEventListener() and Arrow functions in JavaScript and use only Flexbox Layout in CSS.

Simple-Arithmetic-Classes-ArrowFunctions-FlexboxLayout.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Class-Based Simple Arithmetic Calculator (Using Flexbox Layout)</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: system-ui, -apple-system, sans-serif;
    }

    body {
      display: flex;
      justify-content: center;
      align-items: center;
      min-height: 100vh;
      background-color: #f4f6f8;
    }

    /* Outer Card Container */
```

```
.calc-card {
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
  width: 320px;
  padding: 2rem;
  background-color: #ffffff;
  border-radius: 12px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08);
}

.title {
  text-align: center;
  color: #1a1a1a;
  font-size: 1.35rem;
}

/* Inputs Flex Layout */
.input-container {
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}

.input-container input {
  padding: 0.75rem;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  font-size: 1rem;
  outline: none;
}

.input-container input:focus {
  border-color: #2563eb;
}

/* Operation Buttons Flex Layout */
.button-container {
  display: flex;
  flex-wrap: wrap;
  gap: 0.5rem;
}

/* Base Button Styling (Initial State / Before Navigation Interaction) */
.op-btn {
  display: flex;
  flex: 1 1 calc(50% - 0.5rem);
  justify-content: center;
  align-items: center;
  padding: 0.75rem;
  font-size: 1.25rem;
```

```
  font-weight: 600;
  color: #ffffff;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.2s, transform 0.1s;
}

/* Distinct Default Colors for Buttons (Before Interaction) */
.op-btn[data-op="add"] {
  background-color: #2563eb; /* Blue */
}
.op-btn[data-op="subtract"] {
  background-color: #d97706; /* Amber */
}
.op-btn[data-op="multiply"] {
  background-color: #7c3aed; /* Purple */
}
.op-btn[data-op="divide"] {
  background-color: #059669; /* Emerald */
}

/* Hover States (Before Active Click) */
.op-btn[data-op="add"]:hover { background-color: #1d4ed8; }
.op-btn[data-op="subtract"]:hover { background-color: #b45309; }
.op-btn[data-op="multiply"]:hover { background-color: #6d28d9; }
.op-btn[data-op="divide"]:hover { background-color: #047857; }

/* Active Pressing State */
.op-btn:active {
  transform: scale(0.98);
}

/* Active State (After Navigation / Selected) */
.op-btn.active {
  background-color: #0f172a !important; /* Dark Slate (After Selection) */
  color: #38bdf8;
  box-shadow: inset 0 0 0 2px #38bdf8;
}

/* Result Display Flex Layout */
.result-container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0.85rem;
  background-color: #f8fafc;
  border-radius: 6px;
  border: 1px solid #e2e8f0;
}
```

```
    .result-label {
      font-weight: 600;
      color: #64748b;
    }

    .result-value {
      font-size: 1.2rem;
      font-weight: bold;
      color: #0f172a;
    }
  </style>
</head>
<body>

  <main class="calc-card">
    <h2 class="title">Simple Arithmetic Calculator (Using Flexbox Layout)</h2>

    <!-- Input Text Elements -->
    <div class="input-container">
      <input type="number" id="num1" placeholder="Enter first number">
      <input type="number" id="num2" placeholder="Enter second number">
    </div>

    <!-- Operation Buttons (Flexbox) -->
    <div class="button-container">
      <button class="op-btn" data-op="add">+</button>
      <button class="op-btn" data-op="subtract">−</button>
      <button class="op-btn" data-op="multiply">×</button>
      <button class="op-btn" data-op="divide">÷</button>
    </div>

    <!-- Result Display Text Element -->
    <div class="result-container">
      <span class="result-label">Result:</span>
      <span id="result-display" class="result-value">0</span>
    </div>
  </main>

  <script>
    class CalculatorApp {
      constructor() {
        // DOM Element References
        this.num1Input = document.getElementById("num1");
        this.num2Input = document.getElementById("num2");
        this.resultDisplay = document.getElementById("result-display");
        this.buttons = document.querySelectorAll(".op-btn");

        // Initialize Event Listeners
        this.init();
      }
```

```
    // --- ARROW MEMBER FUNCTIONS FOR ARITHMETIC ---
    add = (a, b) => a + b;
    subtract = (a, b) => a - b;
    multiply = (a, b) => a * b;
    divide = (a, b) => (b === 0 ? "Cannot divide by 0" : a / b);

    // --- ARROW MEMBER FUNCTION FOR PROCESSING CALCULATION ---
    calculate = (operation) => {
      const val1 = parseFloat(this.num1Input.value);
      const val2 = parseFloat(this.num2Input.value);

      // Validate inputs
      if (isNaN(val1) || isNaN(val2)) {
        this.resultDisplay.textContent = "Enter numbers";
        return;
      }

      // Map operation string to member function
      const operations = {
        add: this.add,
        subtract: this.subtract,
        multiply: this.multiply,
        divide: this.divide
      };

      const outcome = operations[operation](val1, val2);

      // Format output display
      this.resultDisplay.textContent = typeof outcome === "number"
        ? Number.isInteger(outcome) ? outcome : outcome.toFixed(2)
        : outcome;
    };

    // --- ARROW MEMBER FUNCTION FOR EVENT REGISTRATION & STATE STYLING ---
    init = () => {
      this.buttons.forEach((button) => {
        button.addEventListener("click", (event) => {
          // Remove active class from all buttons (reset color state)
          this.buttons.forEach((btn) => btn.classList.remove("active"));

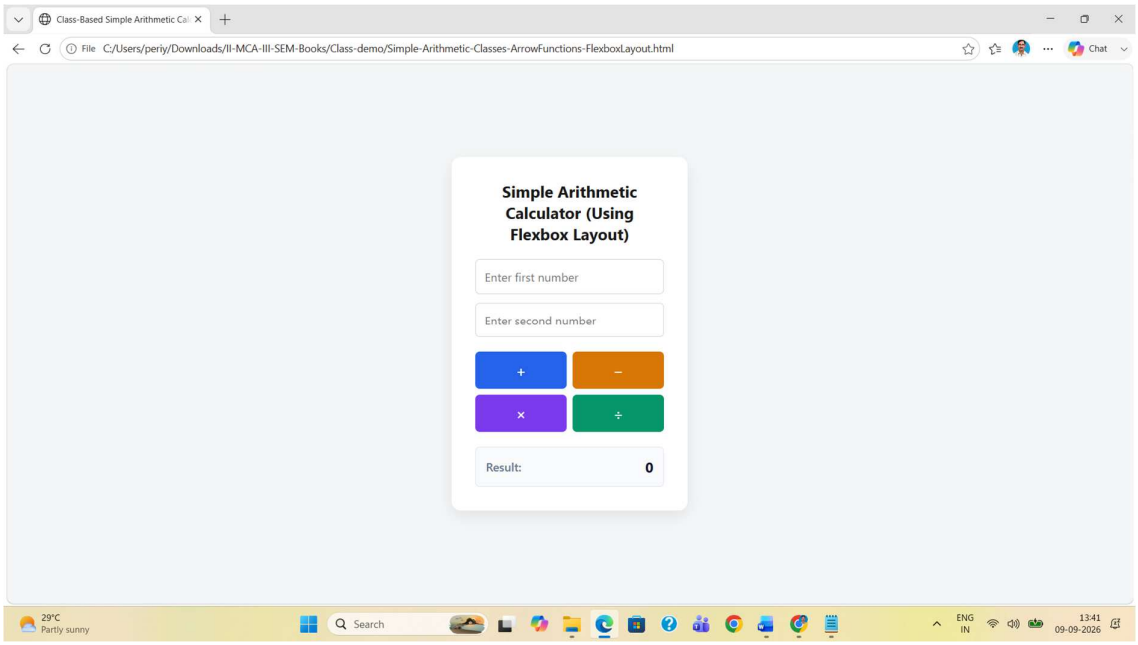
          // Add active class to current button (changes color after interaction)
          event.currentTarget.classList.add("active");

          const operation = event.currentTarget.getAttribute("data-op");
          this.calculate(operation);
        });
      });
    };
```

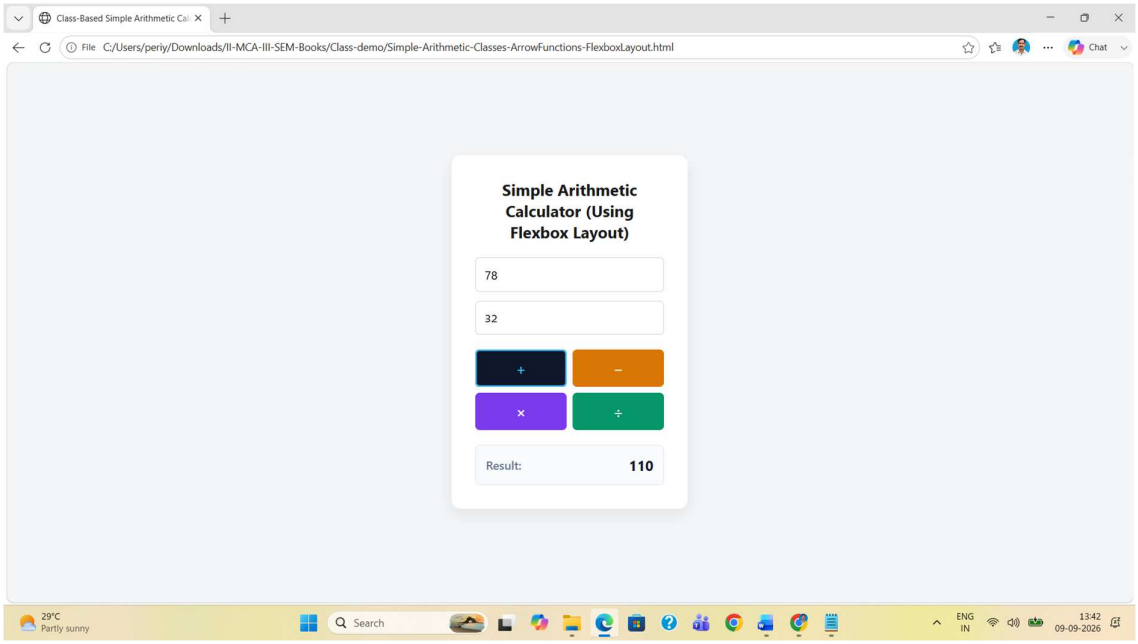
```
    }

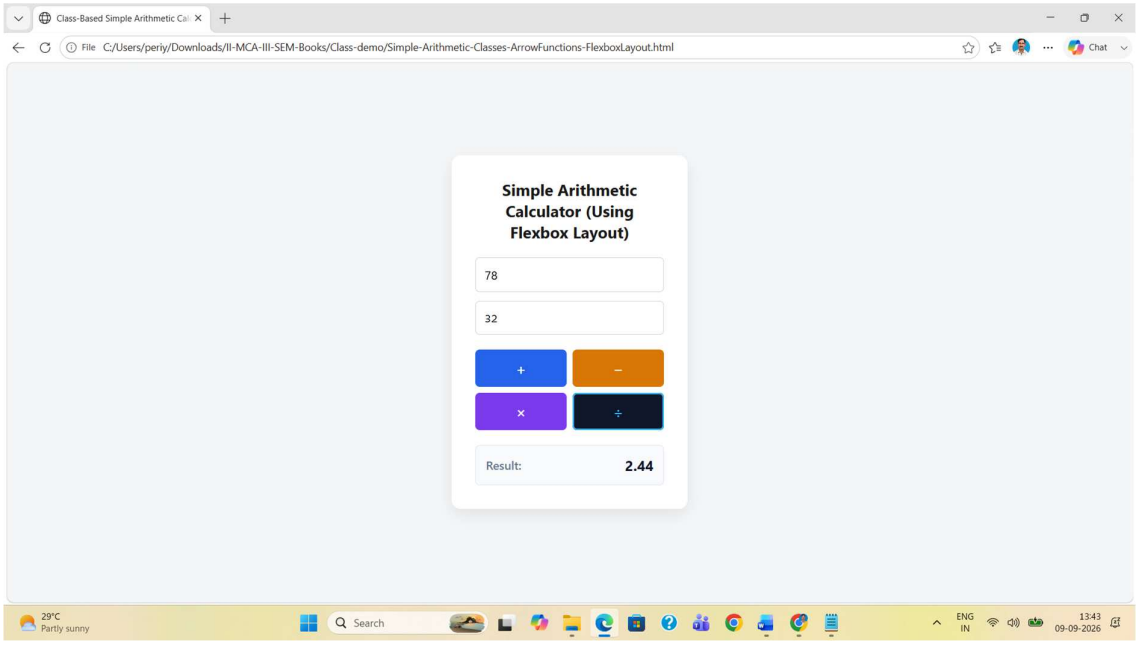
    // Instantiate the class once DOM content is ready
    document.addEventListener("DOMContentLoaded", () => {
      new CalculatorApp();
    });
  </script>
</body>
</html>
```

INPUT



OUTPUT


Class-Based Simple Arithmetic Ca
File C:/Users/periy/Downloads/II-MCA-III-SEM-Books/Class-demo/Simple-Arithmetic-Classes-ArrowFunctions-FlexboxLayout.html
Chat
Simple Arithmetic Calculator (Using Flexbox Layout)
78
32
+
−
×
÷
Result:
110
29°C
Partly sunny
Search
ENG
IN
13:42
09-09-2026


Class-Based Simple Arithmetic Ca
File C:/Users/periy/Downloads/II-MCA-III-SEM-Books/Class-demo/Simple-Arithmetic-Classes-ArrowFunctions-FlexboxLayout.html
Chat
Simple Arithmetic Calculator (Using Flexbox Layout)
78
32
+
−
×
÷
Result:
2.44
29°C
Partly sunny
Search
ENG
IN
13:43
09-09-2026

7. Develop a Simple Web Application to Perform Arithmetic Operations (addition, subtraction, multiplication and Division) using Buttons, Text Elements, addEventListener(), classes, and arrow member functions in JavaScript and use only Grid Layout in CSS.

Simple-Arithmetic-Classes-ArrowFunctions-GridLayout.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Class-Based Simple Arithmetic Calculator (Using Grid Layout)</title>
  <style>
    * {
      box-sizing: border-box;
      margin: 0;
      padding: 0;
      font-family: system-ui, -apple-system, sans-serif;
    }

    body {
      display: grid;
      place-items: center;
      min-height: 100vh;
      background-color: #f4f6f8;
    }

    /* Outer Card Container */
    .calc-card {
      display: grid;
      grid-template-columns: 1fr;
      gap: 1.25rem;
      width: 320px;
      padding: 2rem;
      background-color: #ffffff;
      border-radius: 12px;
      box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08);
    }

    .title {
      text-align: center;
      color: #1a1a1a;
      font-size: 1.35rem;
    }

    /* Inputs Grid Layout */
    .input-container {
      display: grid;
      grid-template-columns: 1fr;
      gap: 0.75rem;
```

```
}

.input-container input {
  padding: 0.75rem;
  border: 1px solid #d1d5db;
  border-radius: 6px;
  font-size: 1rem;
  outline: none;
}

.input-container input:focus {
  border-color: #2563eb;
}

/* Operation Buttons Grid Layout */
.button-container {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 0.5rem;
}

/* Base Button Styling (Initial State / Before Navigation Interaction) */
.op-btn {
  display: grid;
  place-items: center;
  padding: 0.75rem;
  font-size: 1.25rem;
  font-weight: 600;
  background-color: #2563eb; /* Primary Blue (Before Interaction) */
  color: #ffffff;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background-color 0.2s, transform 0.1s;
}

/* Hover State (Before Click Navigation) */
.op-btn:hover {
  background-color: #1d4ed8;
}

/* Active State (When Navigated / Pressed) */
.op-btn:active {
  background-color: #15803d; /* Switches to Green during interaction */
  transform: scale(0.98);
}

/* Distinct Colors for Selected / Navigated Operation States */
.op-btn[data-op="add"] {
  background-color: #2563eb; /* Blue */
}
```

```
    .op-btn[data-op="subtract"] {
      background-color: #d97706; /* Amber */
    }
    .op-btn[data-op="multiply"] {
      background-color: #7c3aed; /* Purple */
    }
    .op-btn[data-op="divide"] {
      background-color: #059669; /* Emerald */
    }

    /* Active Navigated State (After Selection) */
    .op-btn.active {
      background-color: #0f172a !important; /* Dark Slate (After Selection) */
      color: #38bdf8;
      box-shadow: inset 0 0 0 2px #38bdf8;
    }

    /* Result Display Grid Layout */
    .result-container {
      display: grid;
      grid-template-columns: auto 1fr;
      align-items: center;
      padding: 0.85rem;
      background-color: #f8fafc;
      border-radius: 6px;
      border: 1px solid #e2e8f0;
    }

    .result-label {
      font-weight: 600;
      color: #64748b;
    }

    .result-value {
      text-align: right;
      font-size: 1.2rem;
      font-weight: bold;
      color: #0f172a;
    }
  </style>
</head>
<body>

  <main class="calc-card">
    <h2 class="title">Simple Arithmetic Calculator (Using Grid Layout)</h2>

    <!-- Input Text Elements -->
    <div class="input-container">
      <input type="number" id="num1" placeholder="Enter first number">
      <input type="number" id="num2" placeholder="Enter second number">
    </div>
```

```
    <!-- Operation Buttons (CSS Grid) -->
    <div class="button-container">
      <button class="op-btn" data-op="add">+</button>
      <button class="op-btn" data-op="subtract">−</button>
      <button class="op-btn" data-op="multiply">×</button>
      <button class="op-btn" data-op="divide">÷</button>
    </div>

    <!-- Result Display Text Element -->
    <div class="result-container">
      <span class="result-label">Result:</span>
      <span id="result-display" class="result-value">0</span>
    </div>
  </main>

  <script>
    class CalculatorApp {
      constructor() {
        // DOM Element References
        this.num1Input = document.getElementById("num1");
        this.num2Input = document.getElementById("num2");
        this.resultDisplay = document.getElementById("result-display");
        this.buttons = document.querySelectorAll(".op-btn");

        // Initialize Event Listeners
        this.init();
      }

      // --- ARROW MEMBER FUNCTIONS FOR ARITHMETIC ---
      add = (a, b) => a + b;
      subtract = (a, b) => a - b;
      multiply = (a, b) => a * b;
      divide = (a, b) => (b === 0 ? "Cannot divide by 0" : a / b);

      // --- ARROW MEMBER FUNCTION FOR PROCESSING CALCULATION -
--
      calculate = (operation) => {
        const val1 = parseFloat(this.num1Input.value);
        const val2 = parseFloat(this.num2Input.value);

        // Validate inputs
        if (isNaN(val1) || isNaN(val2)) {
          this.resultDisplay.textContent = "Enter numbers";
          return;
        }

        // Map operation string to member function
        const operations = {
          add: this.add,
          subtract: this.subtract,
```

```
        multiply: this.multiply,
        divide: this.divide
      };

      const outcome = operations[operation](val1, val2);

      // Format output display
      this.resultDisplay.textContent = typeof outcome === "number"
        ? Number.isInteger(outcome) ? outcome : outcome.toFixed(2)
        : outcome;
    };

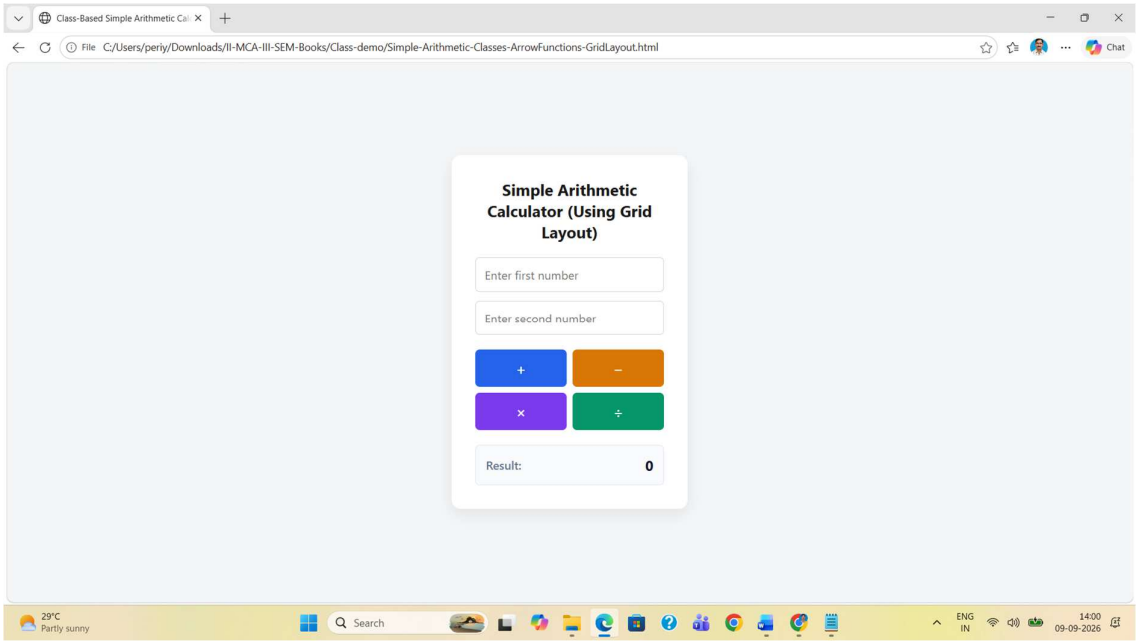
    // --- ARROW MEMBER FUNCTION FOR EVENT REGISTRATION & ACTIVE STYLING ---
    init = () => {
      this.buttons.forEach((button) => {
        button.addEventListener("click", (event) => {
          // Remove active class from all buttons (reset color state)
          this.buttons.forEach((btn) => btn.classList.remove("active"));

          // Add active class to current button (changes color after navigation selection)
          event.currentTarget.classList.add("active");

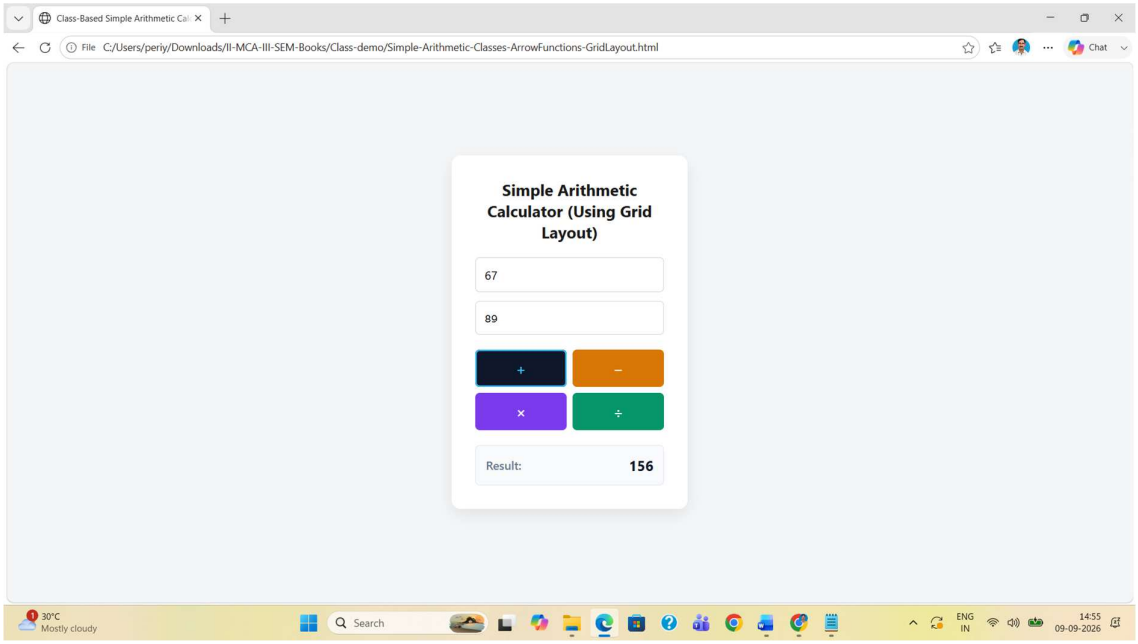
          const operation = event.currentTarget.getAttribute("data-op");
          this.calculate(operation);
        });
      });
    };
  }

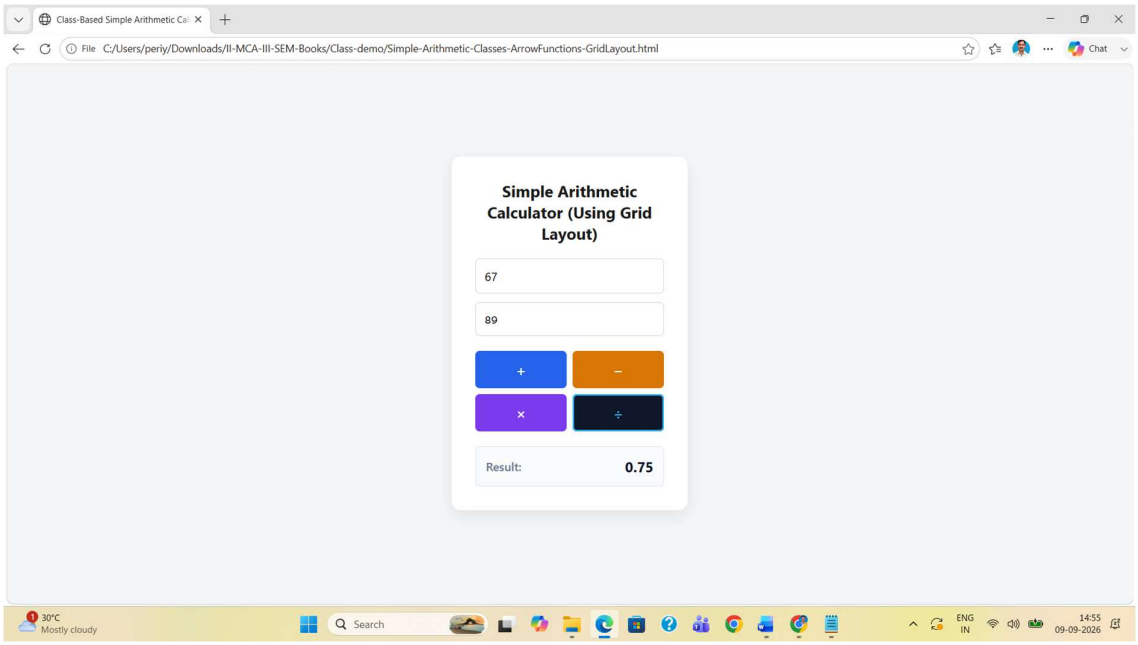
  // Instantiate the class once DOM content is ready
  document.addEventListener("DOMContentLoaded", () => {
    new CalculatorApp();
  });
 </script>
</body>
</html>
```

INPUT



OUTPUT





REACT JS

8. Develop a ReactJS application to perform simple arithmetic operations such as addition, subtraction, multiplication, and division, on the two given numbers through HTML input controls and display the output in the HTML paragraph element.

To create a ReactJS application, do the following:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**npm create vite@latest simple-arithmetic -- --template react**

```
npm notice run npx
npm notice run create-vite simple-arithmetic-app --template react
|
o  Which linter to use?
|  Oxlint
|
o  Install with npm and start now?
|  Yes
|
o  Scaffolding project in C:\ReactJSProjects\simple-arithmetic-app...
|
o  Installing dependencies with npm...

added 24 packages, and audited 25 packages in 22s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
```

```
o  Starting dev server...
npm notice run simple-arithmetic-app@0.0.0 dev
npm notice run vite

  VITE v8.2.2  ready in 1511 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

App.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

/* Page centering layout using Grid */
.page-wrapper {
  display: grid;
  place-items: center;
  min-height: 100vh;
  background-color: #f4f6f8;
}

/* Main Calculator Container Layout using Grid */
.calculator-container {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1.25rem;
  width: 100%;
  max-width: 420px;
  padding: 25px;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background-color: #ffffff;
}

h2 {
  color: #2c3e50;
  margin-top: 0;
  text-align: center;
}

/* Input Fields Grid Layout */
.input-group {
  display: grid;
  grid-template-columns: 1fr;
```

```
  gap: 6px;
}

.input-group label {
  font-weight: 600;
  color: #34495e;
}

.input-group input {
  padding: 10px;
  font-size: 16px;
  border: 1px solid #bdc3c7;
  border-radius: 4px;
  outline: none;
  transition: border-color 0.2s;
}

.input-group input:focus {
  border-color: #3498db;
}

/* Button Navigation Grid Layout */
.button-group {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px;
}

/* Base / BEFORE Navigation State Color */
.button-group button {
  padding: 12px;
  font-size: 15px;
  font-weight: bold;
  color: #ffffff;
  background-color: #3498db; /* Default Blue (Before Selection) */
  border: none;
  border-radius: 4px;
  cursor: pointer;
  transition: all 0.2s ease;
}

.button-group button:hover {
  background-color: #2980b9;
}

/* Selected / AFTER Navigation State Color */
.button-group button.selected {
  background-color: #2c3e50; /* Dark Slate (After Selection) */
  color: #2ecc71; /* Accent Highlight */
  box-shadow: inset 0 0 0 2px #2ecc71;
}
```

```
/* Output Message Area */
.output-message {
  font-size: 18px;
  font-weight: bold;
  color: #2c3e50;
  text-align: center;
  background-color: #f8f9fa;
  padding: 15px;
  border-radius: 4px;
  border-left: 4px solid #3498db;
  margin-bottom: 0;
}
```

App.jsx

```
import React, { useState } from 'react';
import './App.css';

function App() {
  const [num1, setNum1] = useState('');
  const [num2, setNum2] = useState('');
  const [result, setResult] = useState(null);
  const [activeOp, setActiveOp] = useState(null); // Track selected button state

  const handleCalculation = (operator) => {
    setActiveOp(operator); // Update state to trigger "after" navigation styling

    const val1 = parseFloat(num1);
    const val2 = parseFloat(num2);

    if (isNaN(val1) || isNaN(val2)) {
      setResult('Please enter valid numbers in both fields.');
      return;
    }

    let calcResult;
    switch (operator) {
      case '+':
        calcResult = val1 + val2;
        break;
      case '-':
        calcResult = val1 - val2;
        break;
      case '*':
        calcResult = val1 * val2;
        break;
      case '/':
        if (val2 === 0) {
          setResult('Cannot divide by zero.');
```

```
        return;
      }
      calcResult = val1 / val2;
      break;
    default:
      return;
  }

  setResult(`Result: ${calcResult}`);
};

return (
  <div className="page-wrapper">
    <div className="calculator-container">
      <h2>React Arithmetic Calculator</h2>

      {/* Inputs using CSS Grid Layout */}
      <div className="input-group">
        <label htmlFor="num1">First Number:</label>
        <input
          id="num1"
          type="number"
          value={num1}
          onChange={(e) => setNum1(e.target.value)}
          placeholder="Enter first number"
        />
      </div>

      <div className="input-group">
        <label htmlFor="num2">Second Number:</label>
        <input
          id="num2"
          type="number"
          value={num2}
          onChange={(e) => setNum2(e.target.value)}
          placeholder="Enter second number"
        />
      </div>

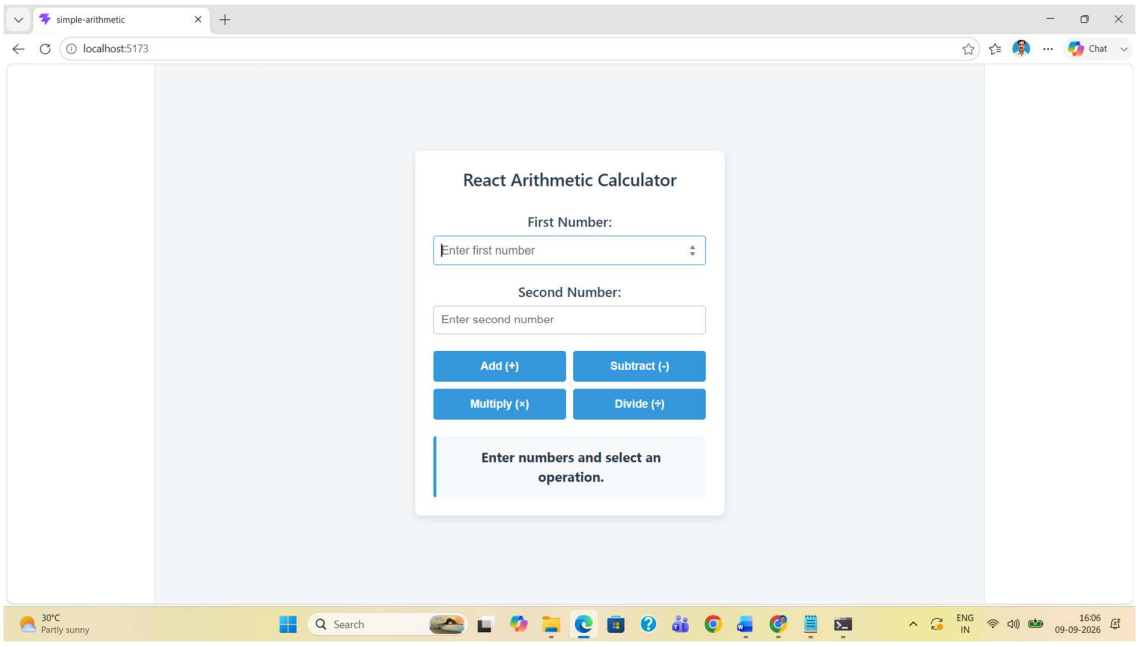
      {/* Operation Buttons using CSS Grid Layout */}
      <div className="button-group">
        <button
          className={activeOp === '+' ? 'selected' : ''}
          onClick={() => handleCalculation('+')}
        >
          Add (+)
        </button>
        <button
          className={activeOp === '-' ? 'selected' : ''}
          onClick={() => handleCalculation('-')}
        >
```

```
            Subtract (-)
          </button>
          <button
            className={activeOp === '*' ? 'selected' : ''}
            onClick={() => handleCalculation('*')}
          >
            Multiply (×)
          </button>
          <button
            className={activeOp === '/' ? 'selected' : ''}
            onClick={() => handleCalculation('/')}
          >
            Divide (÷)
          </button>
        </div>

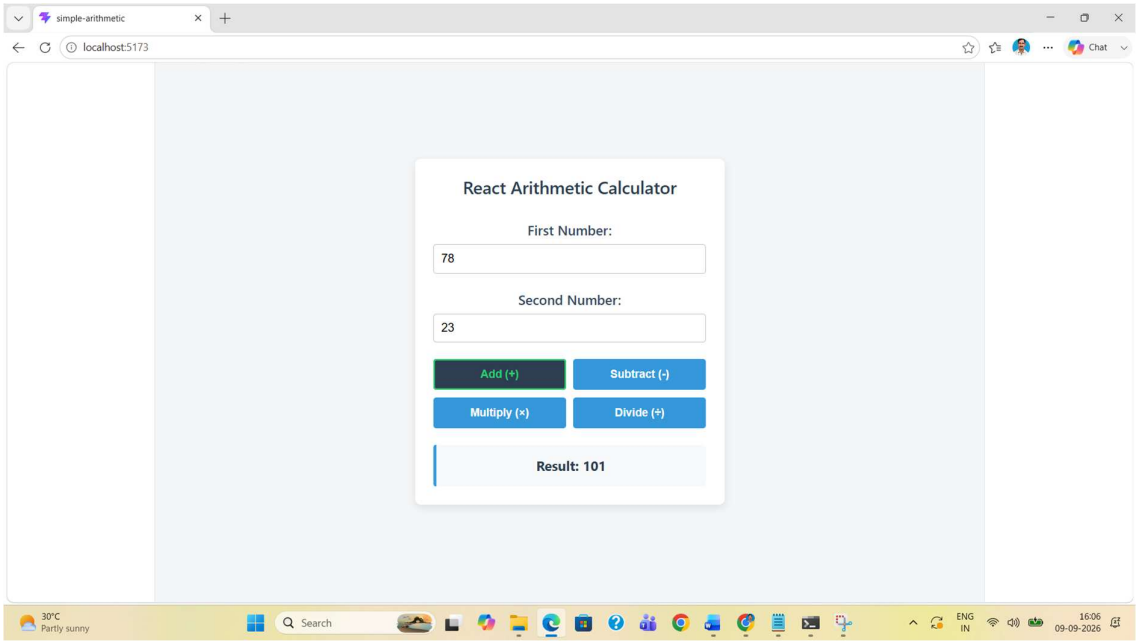
        {/* Output Paragraph Element */}
        <p className="output-message">
          {result !== null ? result : 'Enter numbers and select an operation.'}
        </p>
      </div>
    </div>
  );
}

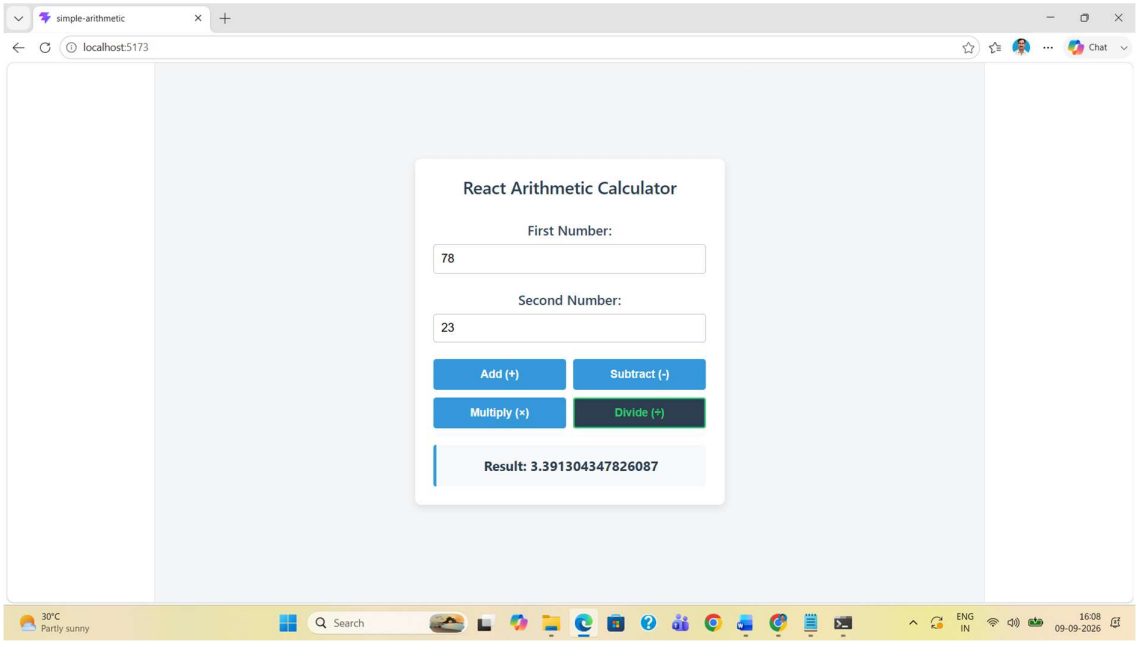
export default App;
```

INPUT



OUTPUT





9. Develop a ReactJS application to calculate factorials and the fibonacci series using recursion based on user input through an HTML input control and display the output in an HTML paragraph element. (NOTE: Only one paragraph element to display either the factorial value or the fibonacci series based on the user navigation).

To create a ReactJS application, do the following:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**npm create vite@latest Fact-Fibo-Recursion -- --template react**

```
npm notice run npx
npm notice run create-vite Fact-Fibo-Recursion --template react
|
o  Which linter to use?
|  Oxlint
|
o  Install with npm and start now?
|  Yes
|
o  Scaffolding project in C:\ReactJSProjects\Fact-Fibo-Recursion...
|
o  Installing dependencies with npm...

added 24 packages, and audited 25 packages in 22s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
o  Starting dev server...
npm notice run Fact-Fibo-Recursion@0.0.0 dev
npm notice run vite

  VITE v8.2.2  ready in 1511 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

App.css

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

/* Page Outer Centering Layout */
body, html {
  height: 100%;
}

.page-wrapper {
  display: flex;
  justify-content: center;
  align-items: center;
```

```
  min-height: 100vh;
  background-color: #f4f6f8;
}

/* Card Container */
.math-container {
  background: #ffffff;
  width: 100%;
  max-width: 380px;
  padding: 24px;
  border-radius: 10px;
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08);
  text-align: center;
  font-family: Arial, sans-serif;
}

h2 {
  color: #1a252f;
  margin-bottom: 20px;
}

.input-section {
  margin-bottom: 20px;
}

.input-section label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;
  color: #333;
}

.input-section input {
  width: 100%;
  padding: 10px 12px;
  font-size: 1rem;
  border: 1px solid #ccc;
  border-radius: 5px;
  outline: none;
  transition: border-color 0.2s;
}

.input-section input:focus {
  border-color: #0d9488;
}

/* Navigation Buttons Container */
.nav-buttons {
  display: flex;
  justify-content: space-between;
  gap: 10px;
```

```
  margin-bottom: 20px;
}

/* BEFORE NAVIGATION / Default State */
.nav-buttons button {
  flex: 1;
  padding: 10px;
  font-size: 0.95rem;
  font-weight: bold;
  cursor: pointer;
  border: 2px solid #0d9488;  /* Teal Outline (Before Selection) */
  background-color: #ffffff;
  color: #0d9488;
  border-radius: 6px;
  transition: all 0.2s ease-in-out;
}

.nav-buttons button:hover {
  background-color: #f0fdfa;
}

/* AFTER NAVIGATION / Active Selected State */
.nav-buttons button.active {
  background-color: #0f172a;  /* Dark Navy Fill (After Selection) */
  border-color: #0f172a;
  color: #38bdf8;             /* Cyan Text */
  box-shadow: inset 0 0 0 2px #38bdf8;
}

/* Output Display Paragraph */
.single-output-paragraph {
  min-height: 40px;
  padding: 12px;
  background-color: #eef6ff;
  border-left: 4px solid #0d9488;
  border-radius: 4px;
  font-size: 1.05rem;
  color: #1a252f;
  margin: 0;
  word-break: break-word;
}
```

App.jsx

```
import React, { useState } from 'react';
import './App.css';

// --- Recursive Helper Functions ---

// 1. Recursive function to calculate Factorial: n! = n * (n - 1)!
const getFactorial = (n) => {
```

```
  if (n < 0) return 'Invalid input (N must be ≥ 0)';
  if (n === 0 || n === 1) return 1;
  return n * getFactorial(n - 1);
};

// 2. Recursive function to find the nth Fibonacci term
const getFibonacciTerm = (n) => {
  if (n <= 0) return 0;
  if (n === 1) return 1;
  return getFibonacciTerm(n - 1) + getFibonacciTerm(n - 2);
};

// Helper function to build the complete Fibonacci series sequence recursively
        up to N
const generateFibonacciSeries = (n) => {
  if (n < 0) return 'Invalid input (N must be ≥ 0)';
  const sequence = [];
  for (let i = 0; i <= n; i++) {
    sequence.push(getFibonacciTerm(i));
  }
  return sequence.join(', ');
};

function App() {
  const [num, setNum] = useState('');
  const [mode, setMode] = useState('factorial'); // 'factorial' or 'fibonacci'

  const n = parseInt(num, 10);

  // Compute output string dynamically based on active mode
  let resultText = '';
  if (num === '' || isNaN(n)) {
    resultText = 'Please enter a valid non-negative integer.';
  } else if (n < 0) {
    resultText = 'Please enter a non-negative number (≥ 0).';
  } else if (mode === 'factorial') {
    resultText = `Factorial of ${n} (${n}!): ${getFactorial(n)}`;
  } else if (mode === 'fibonacci') {
          resultText = `Fibonacci Series up to terms [0..${n}]:
        ${generateFibonacciSeries(n)}`;
  }

  return (
    <div className="page-wrapper">
      <div className="math-container">
        <h2>Factorial and Fibonacci Generator (Using ReactJS)</h2>

        {/* Input Control */}
        <div className="input-section">
          <label htmlFor="numInput">Enter Number (N):</label>
          <input
```

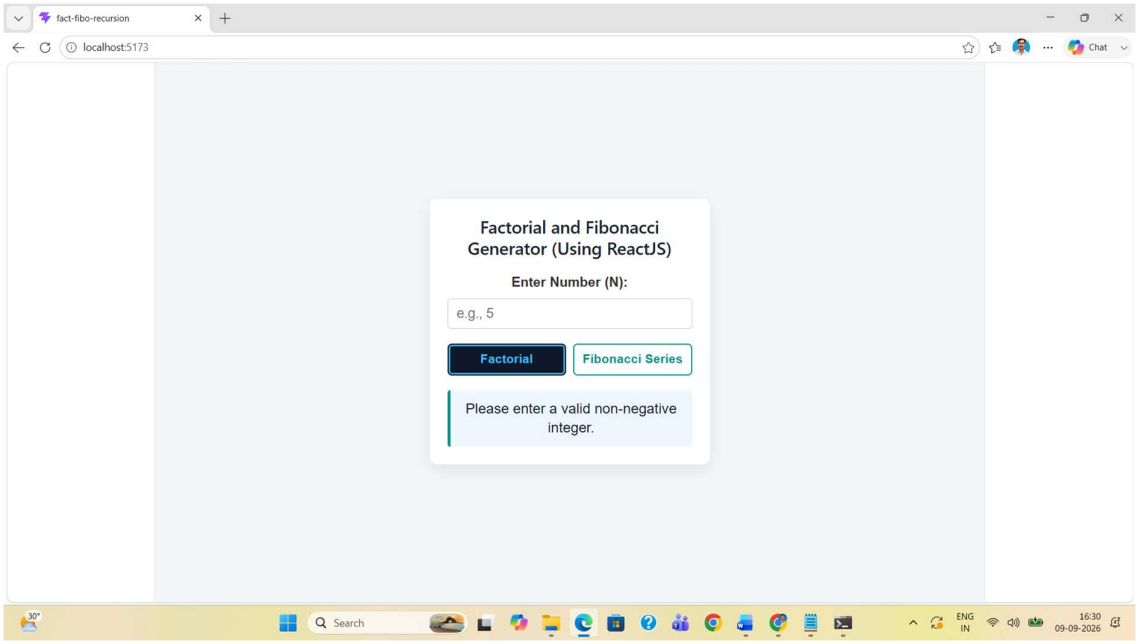
```
            id="numInput"
            type="number"
            min="0"
            value={num}
            onChange={(e) => setNum(e.target.value)}
            placeholder="e.g., 5"
          />
        </div>

        {/* Navigation Controls */}
        <div className="nav-buttons">
          <button
            className={mode === 'factorial' ? 'active' : ''}
            onClick={() => setMode('factorial')}
          >
            Factorial
          </button>
          <button
            className={mode === 'fibonacci' ? 'active' : ''}
            onClick={() => setMode('fibonacci')}
          >
            Fibonacci Series
          </button>
        </div>

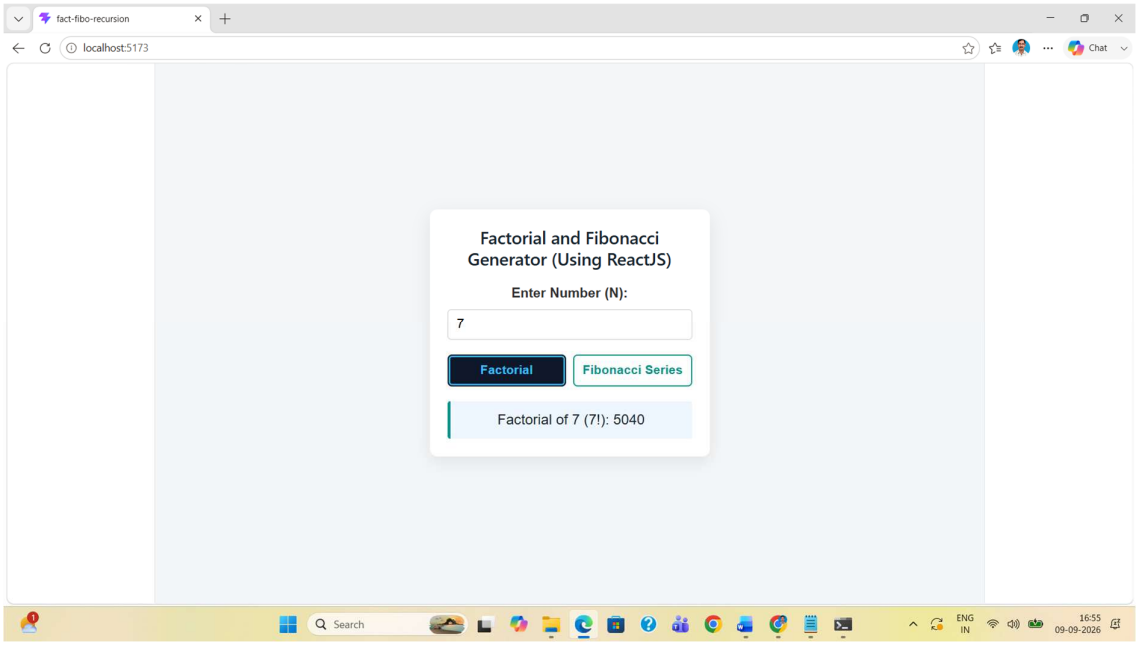
        {/* EXACTLY ONE SINGLE PARAGRAPH ELEMENT FOR ALL OUTPUTS
         */}
        <p className="single-output-paragraph">
          {resultText}
        </p>
      </div>
    </div>
  );
}

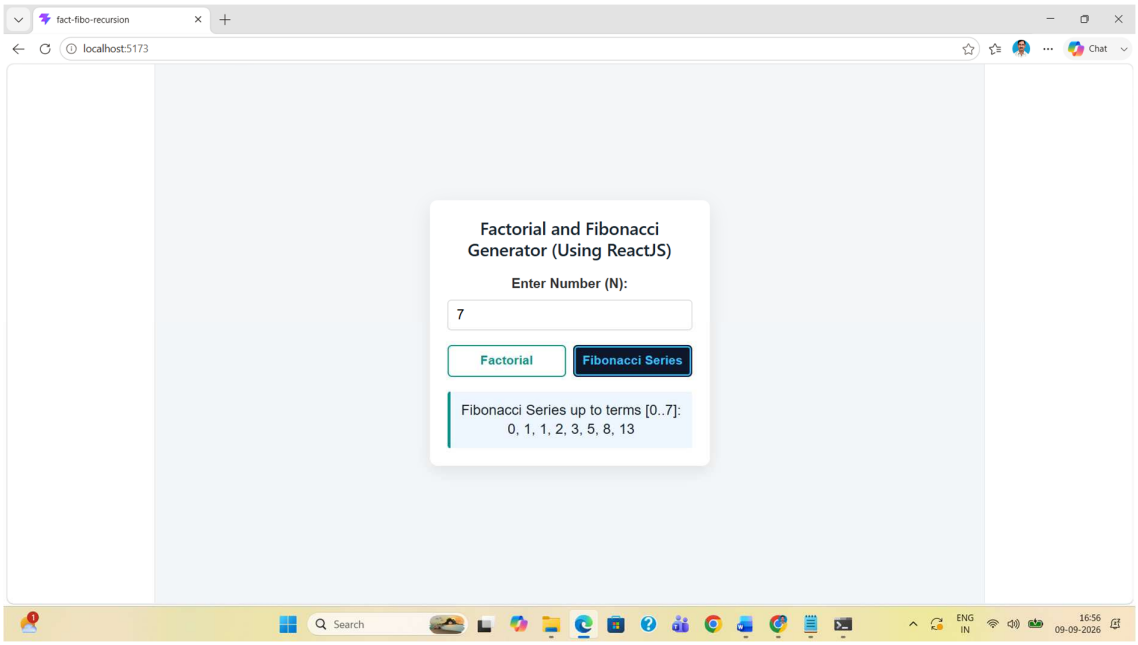
export default App;
```

INPUT



OUTPUT





10. Develop a simple CRUD application (Create, Read, Update, and Delete) using ReactJS for managing users.

To create a ReactJS application, do the following:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**npm create vite@latest user-management-CRUD-app -- --template react**
npm notice run npx
npm notice run create-vite user-management-CRUD-app --template react
|
o Package name:
| user-management-crud-app
|
o Which linter to use?
| Oxlint
|
o Install with npm and start now?
| Yes
|
o Scaffolding project in C:\ReactJSProjects\user-management-CRUD-app...
|
o Installing dependencies with npm...

added 24 packages, and audited 25 packages in 12s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

```
|
o  Starting dev server...
npm notice run user-management-crud-app@0.0.0 dev
npm notice run vite

  VITE v8.2.2  ready in 1730 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
h

  Shortcuts
  press r + enter to restart the server
  press u + enter to show server url
  press o + enter to open in browser
  press c + enter to clear console
  press q + enter to quit
```

App.css

```
.user-management-container {
  padding: 20px;
  font-family: sans-serif;
  max-width: 900px;
  margin: auto;
}

.user-management-container h2 {
  margin-bottom: 16px;
}

/* Form Styles */
.user-form {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
  flex-wrap: wrap;
}

.user-form input,
.user-form select {
  padding: 8px 12px;
  font-size: 0.95rem;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.user-form input:disabled {
  background-color: #f3f4f6;
```

```
  cursor: not-allowed;
}

.btn-submit {
  cursor: pointer;
  color: #fff;
  border: none;
  padding: 8px 16px;
  border-radius: 4px;
  font-weight: bold;
}

.btn-add {
  background-color: #16a34a;
}

.btn-update {
  background-color: #d97706;
}

.btn-cancel {
  cursor: pointer;
  background-color: #6b7280;
  color: #ffffff;
  border: none;
  padding: 8px 16px;
  border-radius: 4px;
}

/* Table Styles */
.user-table {
  width: 100%;
  text-align: left;
  border-collapse: collapse;
  border: 1px solid #ddd;
}

.user-table th,
.user-table td {
  padding: 8px 12px;
  border: 1px solid #ddd;
}

.user-table thead tr {
  background-color: #f2f2f2;
}

.btn-edit {
  margin-right: 6px;
  padding: 4px 10px;
  cursor: pointer;
```

```
  border: 1px solid #ccc;
  background-color: #ffffff;
  border-radius: 4px;
}

.btn-delete {
  padding: 4px 10px;
  cursor: pointer;
  color: red;
  border: 1px solid red;
  background-color: #ffffff;
  border-radius: 4px;
}

.empty-row {
  text-align: center;
  color: #6b7280;
}
```

App.jsx

```
import React, { useState } from 'react';
import './App.css';

export default function UserManagement() {
  const [users, setUsers] = useState([
    { id: 'USR101', name: 'Alex Johnson', email: 'alex@example.com', role:
      'Admin' },
    { id: 'USR102', name: 'Sarah Connor', email: 'sarah@example.com', role:
      'Editor' },
  ]);

  const initialForm = { id: '', name: '', email: '', role: 'User' };
  const [formData, setFormData] = useState(initialForm);
  const [isEditing, setIsEditing] = useState(false);

  // Handle Input Changes
  const handleChange = (e) => {
    setFormData({ ...formData, [e.target.name]: e.target.value });
  };

  // CREATE or UPDATE User
  const handleSubmit = (e) => {
    e.preventDefault();
    if (!formData.id || !formData.name || !formData.email) return;

    if (isEditing) {
      // Update existing record by matching ID
        setUsers(users.map((user) => (user.id === formData.id ? formData :
        user)));
      setIsEditing(false);
```

```
  } else {
    // Prevent duplicate ID insertion
    if (users.some((user) => user.id === formData.id)) {
      alert('User ID already exists! Please use a unique ID.');
      return;
    }
    setUsers([...users, formData]);
  }
  setFormData(initialForm);
};

// Pre-fill form for EDIT
const handleEdit = (user) => {
  setFormData(user);
  setIsEditing(true);
};

// DELETE User
const handleDelete = (id) => {
  setUsers(users.filter((user) => user.id !== id));
};

return (
  <div className="user-management-container">
    <h2>User Management System</h2>

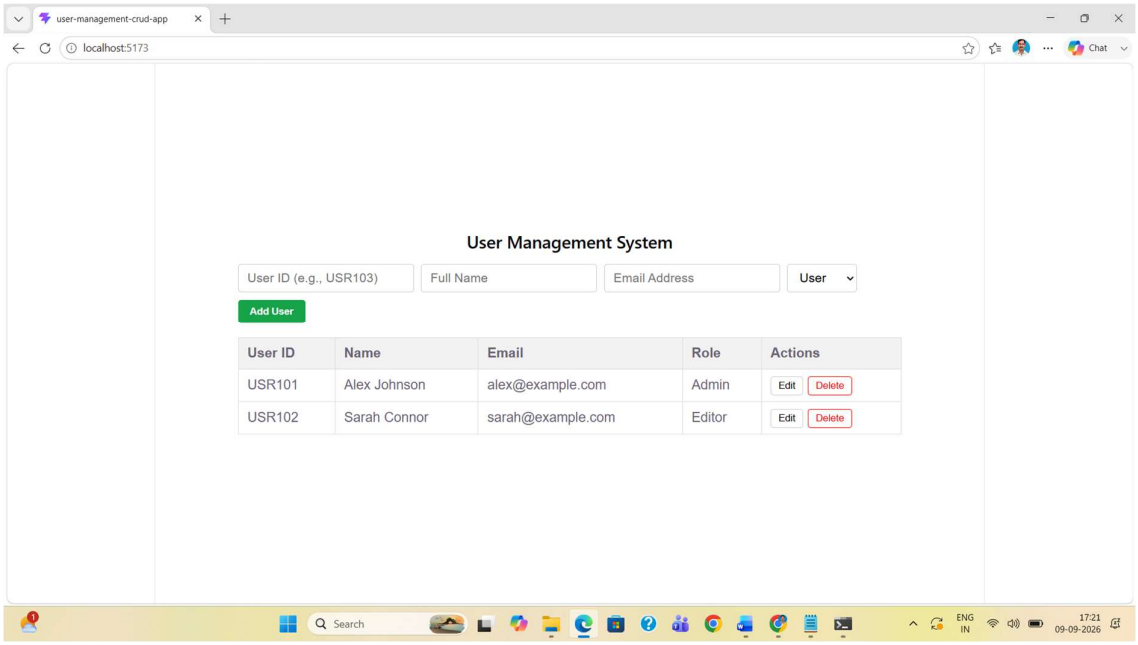
    {/* Input Form with ID Input */}
    <form onSubmit={handleSubmit} className="user-form">
      <input
        type="text"
        name="id"
        placeholder="User ID (e.g., USR103)"
        value={formData.id}
        onChange={handleChange}
        disabled={isEditing} /* Prevent changing ID while editing */
        required
      />
      <input
        type="text"
        name="name"
        placeholder="Full Name"
        value={formData.name}
        onChange={handleChange}
        required
      />
      <input
        type="email"
        name="email"
        placeholder="Email Address"
        value={formData.email}
        onChange={handleChange}
```

```
    required
   />
   <select name="role" value={formData.role} onChange={handleChange}>
    <option value="User">User</option>
    <option value="Editor">Editor</option>
    <option value="Admin">Admin</option>
   </select>
   <button
    type="submit"
    className={`btn-submit ${isEditing ? 'btn-update' : 'btn-add'}`}
   >
    {isEditing ? 'Update User' : 'Add User'}
   </button>
   {isEditing && (
    <button
     type="button"
     className="btn-cancel"
     onClick={() => { setIsEditing(false); setFormData(initialForm); }}
    >
     Cancel
    </button>
   )}
  </form>

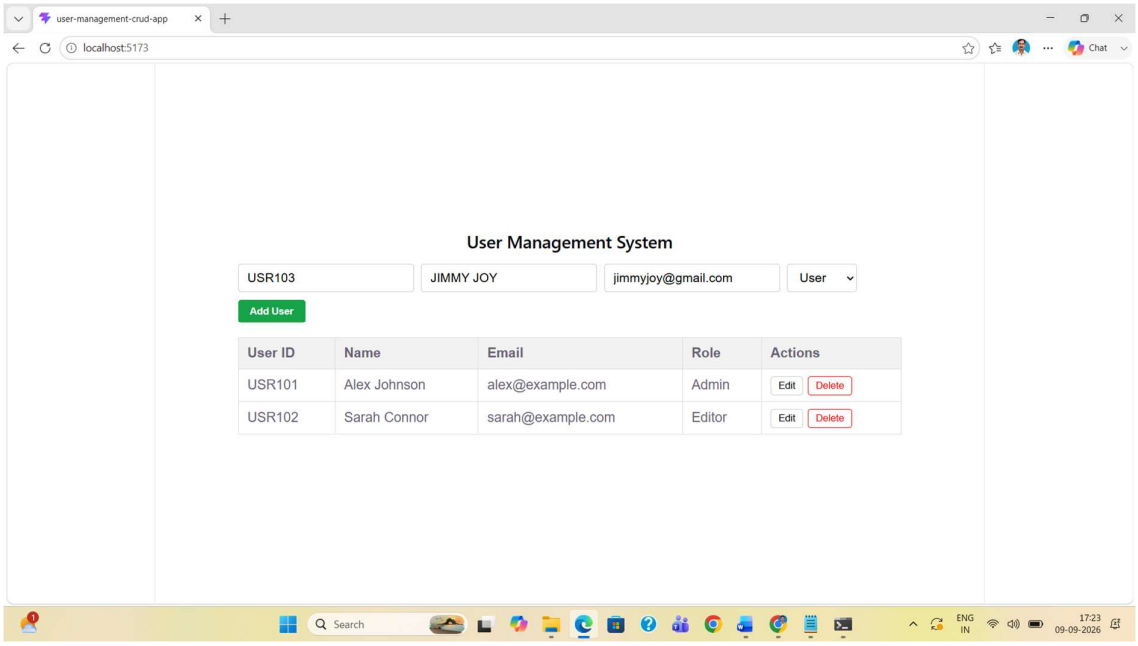
  {/* User Table */}
  <table className="user-table">
   <thead>
    <tr>
     <th>User ID</th>
     <th>Name</th>
     <th>Email</th>
     <th>Role</th>
     <th>Actions</th>
    </tr>
   </thead>
   <tbody>
    {users.map((user) => (
     <tr key={user.id}>
      <td>{user.id}</td>
      <td>{user.name}</td>
      <td>{user.email}</td>
      <td>{user.role}</td>
      <td>
                    <button  className="btn-edit"  onClick={()  =>
    handleEdit(user)}>Edit</button>
                  <button  className="btn-delete"  onClick={()  =>
    handleDelete(user.id)}>Delete</button>
      </td>
     </tr>
    ))}
    {users.length === 0 && (
```

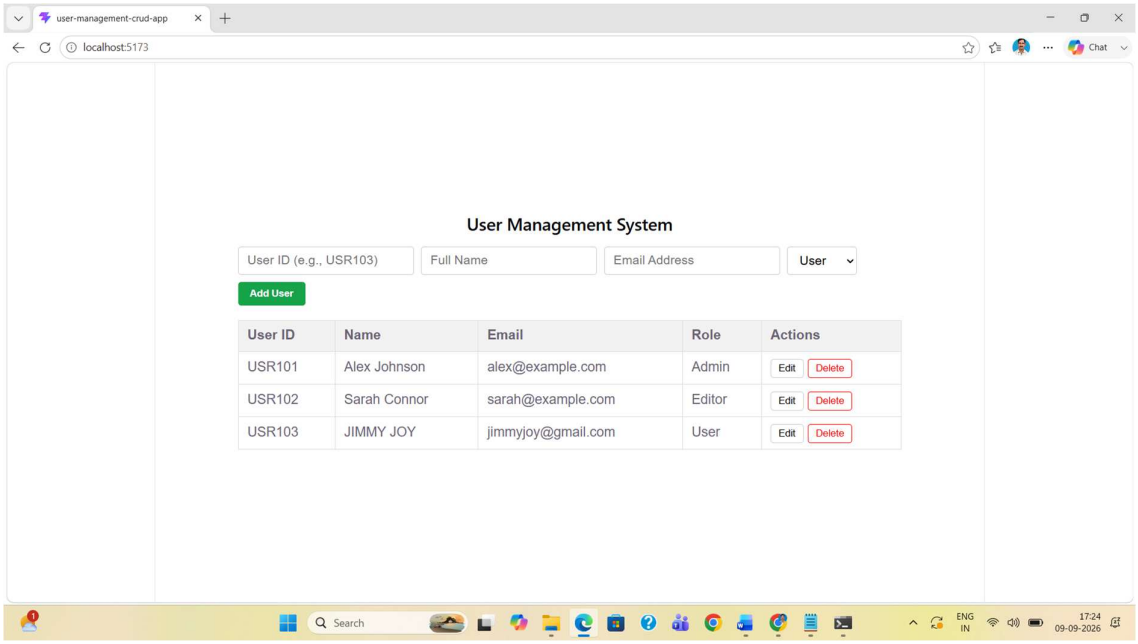
```
            <tr>
              <td colSpan="5" className="empty-row">No users found.</td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  );
}
```

INPUT



OUTPUT





11. Develop a simple CRUD application (Create, Read, Update, and Delete) using ReactJS for student mark list generation with Register Number, Name, Marks of Subject 1, Subject 2, Subject 3, Subject 4, Subject 5, Result, CGPA, and Grade. Result, CGPA, and Grade are calculated whenever a new student record is inserted or updated in the MySQL Database.

To create a ReactJS application, do the following:

Launch the WAMP/XAMPP Application Server and then execute the following script in MySQL to create the students database and table:

SQL Commands

```
CREATE DATABASE IF NOT EXISTS student_db;
USE student_db;

CREATE TABLE IF NOT EXISTS students (
  registerNumber VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  mark1 INT NOT NULL,
  mark2 INT NOT NULL,
  mark3 INT NOT NULL,
  mark4 INT NOT NULL,
  mark5 INT NOT NULL,
  result VARCHAR(10),
  cgpa DECIMAL(4, 2),
  grade VARCHAR(5)
);
```

Open a command prompt and do the following:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**mkdir Student-Marklist-MySQL**

C:\ReactJSProjects>**cd Student-Marklist-MySQL**

C:\ReactJSProjects\Student-Marklist-MySQL>**mkdir backend**

C:\ReactJSProjects\Student-Marklist-MySQL>**cd backend**

C:\ReactJSProjects\Student-Marklist-MySQL\backend>

Backend (server.js)
Initialize a Node project (npm init -y) and install dependencies (npm install express mysql2 cors).

server.js

```
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

// MySQL Connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',      // Replace with your MySQL username
  password: '',      // Replace with your MySQL password
  database: 'student_db'
});

db.connect(err => {
  if (err) throw err;
  console.log('MySQL Connected...');
});

// Helper function to calculate Result, CGPA, and Grade
const calculatePerformance = (s1, s2, s3, s4, s5) => {
   const marks = [Number(s1), Number(s2), Number(s3), Number(s4), Number(s5)];

  // Result: Pass if all subjects >= 40, else Fail
  const isPassed = marks.every(m => m >= 40);
```

```
  const result = isPassed ? 'Pass' : 'Fail';

  // CGPA: Average percentage scaled to a 10-point scale
  const total = marks.reduce((acc, curr) => acc + curr, 0);
  const avg = total / 5;
  const cgpa = Number((avg / 10).toFixed(2));

  // Grade calculation
  let grade = 'F';
  if (isPassed) {
    if (avg >= 90) grade = 'O';
    else if (avg >= 80) grade = 'A+';
    else if (avg >= 70) grade = 'A';
    else if (avg >= 60) grade = 'B+';
    else if (avg >= 50) grade = 'B';
    else grade = 'C';
  }

  return { result, cgpa, grade };
};

// READ: Fetch all students
app.get('/api/students', (req, res) => {
  db.query('SELECT * FROM students', (err, results) => {
    if (err) return res.status(500).send(err);
    res.json(results);
  });
});

// CREATE: Add new student record
app.post('/api/students', (req, res) => {
  const { reg_no, name, sub1, sub2, sub3, sub4, sub5 } = req.body;
  const { result, cgpa, grade } = calculatePerformance(sub1, sub2, sub3, sub4,
sub5);

  const sql = `INSERT INTO students
    (reg_no, name, sub1, sub2, sub3, sub4, sub5, result, cgpa, grade)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`;

  db.query(sql, [reg_no, name, sub1, sub2, sub3, sub4, sub5, result, cgpa,
grade], (err, result) => {
    if (err) return res.status(500).send(err);
    res.json({ message: 'Student record created successfully!' });
  });
});

// UPDATE: Modify existing student record
app.put('/api/students/:id', (req, res) => {
  const { id } = req.params;
  const { reg_no, name, sub1, sub2, sub3, sub4, sub5 } = req.body;
```

```
  const { result, cgpa, grade } = calculatePerformance(sub1, sub2, sub3, sub4,
sub5);

  const sql = `UPDATE students SET
     reg_no=?, name=?, sub1=?, sub2=?, sub3=?, sub4=?, sub5=?, result=?,
cgpa=?, grade=?
    WHERE id=?`;

  db.query(sql, [reg_no, name, sub1, sub2, sub3, sub4, sub5, result, cgpa,
grade, id], (err, result) => {
    if (err) return res.status(500).send(err);
    res.json({ message: 'Student record updated successfully!' });
  });
});

// DELETE: Remove student record
app.delete('/api/students/:id', (req, res) => {
  const { id } = req.params;
  db.query('DELETE FROM students WHERE id=?', [id], (err, result) => {
    if (err) return res.status(500).send(err);
    res.json({ message: 'Student record deleted!' });
  });
});

app.listen(5000, () => console.log('Server running on port 5000'));
```

App.css

```
:root {
  --primary: #4f46e5;
  --primary-hover: #4338ca;
  --bg-color: #f8fafc;
  --card-bg: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --success: #16a34a;
  --success-bg: #dcfce7;
  --danger: #dc2626;
  --danger-hover: #b91c1c;
  --danger-bg: #fee2e2;
  --warning: #d97706;
  --warning-hover: #b45309;
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
```

```
body {
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
        Helvetica, Arial, sans-serif;
  background-color: var(--bg-color);
  color: var(--text-main);
  line-height: 1.5;
}

.container {
  max-width: 1100px;
  margin: 0 auto;
  padding: 2rem 1rem;
}

.app-header {
  margin-bottom: 2rem;
  text-align: center;
}

.app-header h1 {
  font-size: 2rem;
  color: var(--text-main);
}

.app-header p {
  color: var(--text-muted);
  font-size: 0.95rem;
}

.card {
  background: var(--card-bg);
  border: 1px solid var(--border-color);
  border-radius: 8px;
  padding: 1.5rem;
  margin-bottom: 2rem;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
}

.card h2 {
  font-size: 1.25rem;
  margin-bottom: 1.25rem;
  color: var(--text-main);
}

/* Form Styles */
.student-form {
  display: flex;
  flex-direction: column;
  gap: 1.25rem;
}
```

```
.form-group label {
  display: block;
  font-size: 0.85rem;
  font-weight: 600;
  margin-bottom: 0.35rem;
  color: var(--text-main);
}

.form-group input {
  width: 100%;
  padding: 0.6rem 0.8rem;
  border: 1px solid var(--border-color);
  border-radius: 6px;
  font-size: 0.95rem;
  outline: none;
  transition: border-color 0.2s;
}

.form-group input:focus {
  border-color: var(--primary);
}

.row-2 {
  display: grid;
  grid-template-columns: 1fr 2fr;
  gap: 1rem;
}

.marks-grid {
  display: grid;
  grid-template-columns: repeat(5, 1fr);
  gap: 0.75rem;
}

.form-actions {
  display: flex;
  gap: 0.75rem;
}

/* General Buttons */
.btn {
  padding: 0.6rem 1.2rem;
  border-radius: 6px;
  font-size: 0.9rem;
  font-weight: 600;
  cursor: pointer;
  border: none;
  transition: background-color 0.2s, transform 0.1s;
}

.btn:active {
```

```
  transform: scale(0.98);
}

.btn-primary {
  background-color: var(--primary);
  color: white;
}

.btn-primary:hover {
  background-color: var(--primary-hover);
}

.btn-secondary {
  background-color: #94a3b8;
  color: white;
}

.btn-secondary:hover {
  background-color: #78869b;
}

/* Action Buttons (Table Edit & Delete) */
.actions-cell {
  display: flex;
  gap: 0.5rem;
  align-items: center;
}

.btn-action {
  padding: 0.35rem 0.75rem;
  font-size: 0.8rem;
  border-radius: 4px;
  font-weight: 600;
  color: white;
  border: none;
}

.btn-edit {
  background-color: var(--warning);
}

.btn-edit:hover {
  background-color: var(--warning-hover);
}

.btn-delete {
  background-color: var(--danger);
}

.btn-delete:hover {
  background-color: var(--danger-hover);
```

```
}

/* Table Styles */
.table-responsive {
  overflow-x: auto;
}

.student-table {
  width: 100%;
  border-collapse: collapse;
  text-align: left;
  font-size: 0.9rem;
}

.student-table th,
.student-table td {
  padding: 0.75rem 1rem;
  border-bottom: 1px solid var(--border-color);
  vertical-align: middle;
}

.student-table th {
  background-color: #f1f5f9;
  font-weight: 600;
  color: var(--text-muted);
}

.student-table tbody tr:hover {
  background-color: #f8fafc;
}

/* Status & Indicators */
.badge {
  display: inline-block;
  padding: 0.2rem 0.6rem;
  border-radius: 999px;
  font-size: 0.75rem;
  font-weight: 700;
}

.badge.pass {
  background-color: var(--success-bg);
  color: var(--success);
}

.badge.fail {
  background-color: var(--danger-bg);
  color: var(--danger);
}

.grade-pill {
```

```
  font-weight: 700;
  color: var(--primary);
}

.status-text {
  text-align: center;
  color: var(--text-muted);
  padding: 1.5rem 0;
}

/* Responsive Layout */
@media (max-width: 768px) {
  .row-2,
  .marks-grid {
    grid-template-columns: 1fr;
  }
}
```

App.jsx

```
import React, { useState, useEffect } from 'react';
import axios from 'axios';
import './App.css';

const initialFormState = {
  reg_no: '',
  name: '',
  sub1: '',
  sub2: '',
  sub3: '',
  sub4: '',
  sub5: ''
};

export default function App() {
  const [students, setStudents] = useState([]);
  const [formData, setFormData] = useState(initialFormState);
  const [editingId, setEditingId] = useState(null);
  const [loading, setLoading] = useState(false);

  const API_URL = 'http://localhost:5000/api/students';

  const fetchStudents = async () => {
    try {
      setLoading(true);
      const res = await axios.get(API_URL);
      setStudents(res.data);
    } catch (error) {
      console.error('Error fetching students:', error);
    } finally {
```

```
    setLoading(false);
  }
};

useEffect(() => {
  fetchStudents();
}, []);

const handleChange = (e) => {
  const { name, value } = e.target;
  setFormData((prev) => ({ ...prev, [name]: value }));
};

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    if (editingId) {
      await axios.put(`${API_URL}/${editingId}`, formData);
    } else {
      await axios.post(API_URL, formData);
    }
    handleCancel();
    fetchStudents();
  } catch (error) {
    console.error('Error saving record:', error);
  }
};

const handleEdit = (student) => {
  setEditingId(student.id);
  setFormData({
    reg_no: student.reg_no,
    name: student.name,
    sub1: student.sub1,
    sub2: student.sub2,
    sub3: student.sub3,
    sub4: student.sub4,
    sub5: student.sub5
  });
};

const handleDelete = async (id) => {
  if (window.confirm('Are you sure you want to delete this record?')) {
    try {
      await axios.delete(`${API_URL}/${id}`);
      fetchStudents();
    } catch (error) {
      console.error('Error deleting record:', error);
    }
  }
};
```

```
const handleCancel = () => {
  setEditingId(null);
  setFormData(initialFormState);
};

return (
  <div className="container">
    <header className="app-header">
      <h1>Student Mark List Manager</h1>
      <p>Calculate grades, CGPA, and results automatically</p>
    </header>

    <section className="card form-card">
      <h2>{editingId ? 'Edit Student Record' : 'Add New Student'}</h2>
      <form onSubmit={handleSubmit} className="student-form">
        <div className="form-group row-2">
          <div>
            <label>Register Number</label>
            <input
              type="text"
              name="reg_no"
              placeholder="e.g., REG101"
              value={formData.reg_no}
              onChange={handleChange}
              required
            />
          </div>
          <div>
            <label>Student Name</label>
            <input
              type="text"
              name="name"
              placeholder="Full Name"
              value={formData.name}
              onChange={handleChange}
              required
            />
          </div>
        </div>

        <div className="form-group marks-grid">
          {[1, 2, 3, 4, 5].map((num) => (
            <div key={num}>
              <label>Subject {num}</label>
              <input
                type="number"
                name={`sub${num}`}
                placeholder="0-100"
                value={formData[`sub${num}`]}
                onChange={handleChange}
```

```
            min="0"
            max="100"
            required
          />
        </div>
      ))}
    </div>

    <div className="form-actions">
      <button type="submit" className="btn btn-primary">
        {editingId ? 'Update Record' : 'Save Record'}
      </button>
      {editingId && (
                <button type="button" className="btn btn-secondary"
  onClick={handleCancel}>
          Cancel
        </button>
      )}
    </div>
  </form>
</section>

<section className="card table-card">
  <h2>Student Mark List</h2>
  {loading ? (
    <p className="status-text">Loading records...</p>
  ) : students.length === 0 ? (
    <p className="status-text">No records found. Add a student to get
   started.</p>
  ) : (
    <div className="table-responsive">
      <table className="student-table">
        <thead>
          <tr>
            <th>Reg No</th>
            <th>Name</th>
            <th>Sub 1</th>
            <th>Sub 2</th>
            <th>Sub 3</th>
            <th>Sub 4</th>
            <th>Sub 5</th>
            <th>Result</th>
            <th>CGPA</th>
            <th>Grade</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {students.map((s) => (
            <tr key={s.id}>
              <td><strong>{s.reg_no}</strong></td>
```

```
                <td>{s.name}</td>
                <td>{s.sub1}</td>
                <td>{s.sub2}</td>
                <td>{s.sub3}</td>
                <td>{s.sub4}</td>
                <td>{s.sub5}</td>
                <td>
                 <span className={`badge ${s.result === 'Pass' ? 'pass' : 'fail'}`}>
                   {s.result}
                 </span>
                </td>
                <td>{s.cgpa}</td>
                <td>
                 <span className="grade-pill">{s.grade}</span>
                </td>
                <td className="actions-cell">
                 <button
                   type="button"
                   className="btn btn-action btn-edit"
                   onClick={() => handleEdit(s)}
                 >
                   Edit
                 </button>
                 <button
                   type="button"
                   className="btn btn-action btn-delete"
                   onClick={() => handleDelete(s.id)}
                 >
                   Delete
                 </button>
                </td>
               </tr>
              ))}
             </tbody>
            </table>
           </div>
         )}
        </section>
      </div>
    );
}
```

Steps to create and run the application

To run NodeJS, do the following steps in that command prompt:

C:\ReactJSProjects>**cd Student-Marklist-MySQL**

C:\ReactJSProjects\Student-Marklist-MySQL>**cd backend**

C:\ReactJSProjects\Student-Marklist-MySQL\backend>**dir**

```
 Volume in drive C is Windows
 Volume Serial Number is EA37-06AE

 Directory of C:\ReactJSProjects\Student-Marklist-MySQL\backend

15-08-2026  20:59    <DIR>          .
15-08-2026  20:53    <DIR>          ..
15-08-2026  20:59    <DIR>          node_modules
15-08-2026  20:59            34,764 package-lock.json
15-08-2026  20:59               128 package.json
15-08-2026  20:49             3,219 server.js
               3 File(s)         38,111 bytes
               3 Dir(s)  56,210,452,480 bytes free

C:\ReactJSProjects\Student-Marklist-MySQL\backend>node server.js
Server running on port 5000
MySQL Connected...
```

Open another command prompt and do the following:

```
C:\Users\periy>cd C:\ReactJSProjects

C:\ReactJSProjects>cd Student-Marklist-MySQL

C:\ReactJSProjects\Student-Marklist-MySQL>npm create vite@latest frontend -- --template react
npm notice run npx
npm notice run create-vite frontend --template react
|
o  Package name:
|  user-management-crud-app
|
o  Which linter to use?
|  Oxlint
|
o  Install with npm and start now?
|  Yes
|
o  Scaffolding project in C:\ReactJSProjects\Student-Marklist-MySQL\frontend...
|
o  Installing dependencies with npm...

added 24 packages, and audited 25 packages in 12s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
o  Starting dev server...
```

```
npm notice run frontend@0.0.0 dev
npm notice run vite

  VITE v8.2.2  ready in 1730 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
h

  Shortcuts
  press r + enter to restart the server
  press u + enter to show server url
  press o + enter to open in browser
  press c + enter to clear console
  press q + enter to quit
```

If the frontend has already been created, then do the following steps:

```
C:\Users\periy>cd C:\ReactJSProjects\Student-Marklist-MySQL\frontend

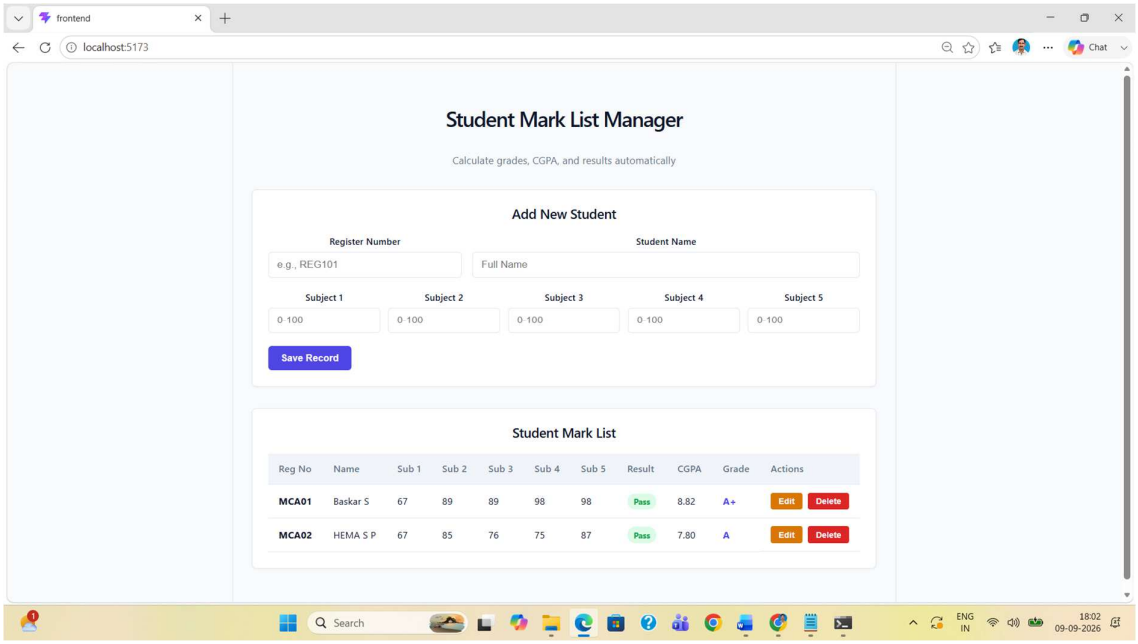
C:\ReactJSProjects\Student-Marklist-MySQL\frontend>npm run dev
npm notice run frontend@0.0.0 dev
npm notice run vite

  VITE v8.2.1  ready in 369 ms

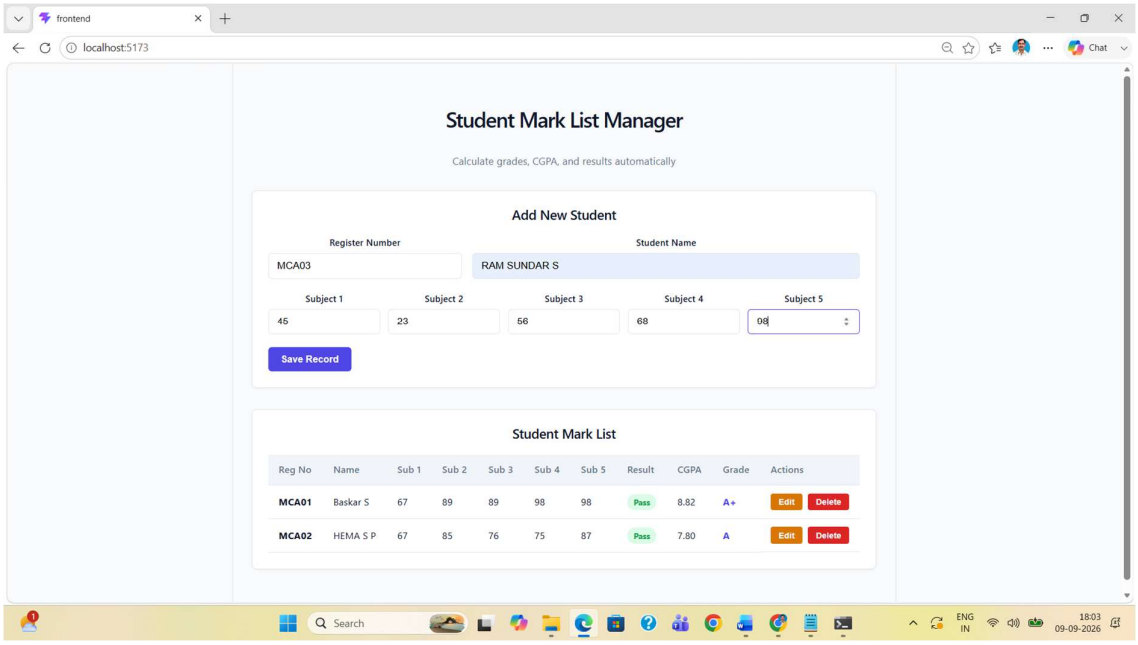
  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
h

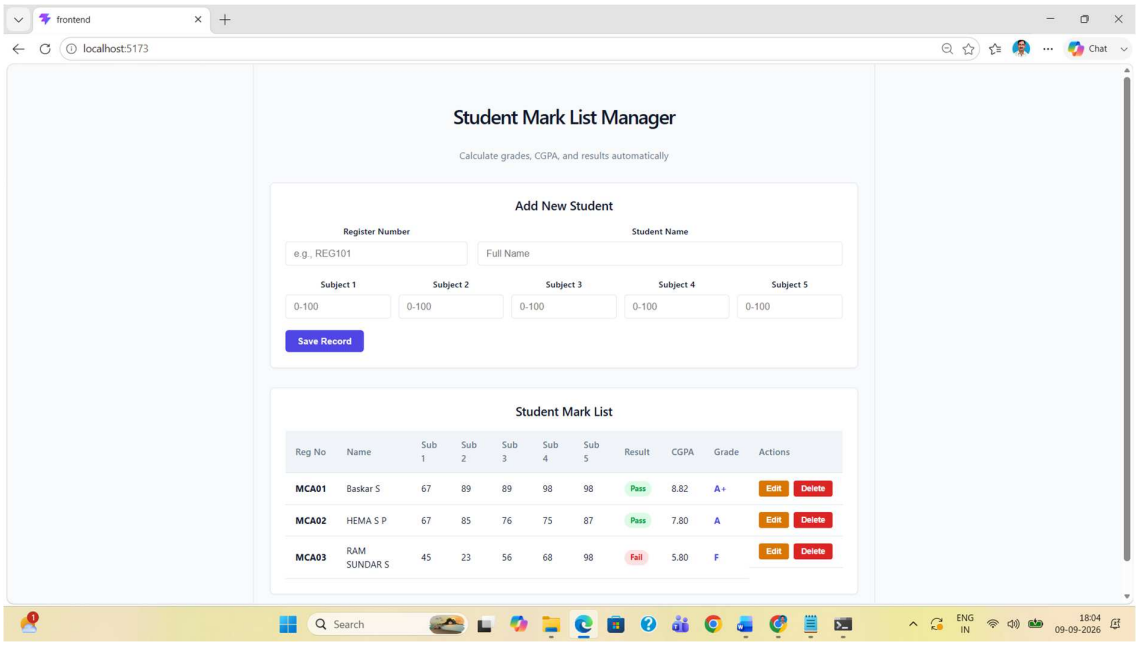
  Shortcuts
  press r + enter to restart the server
  press u + enter to show server url
  press o + enter to open in browser
  press c + enter to clear console
  press q + enter to quit
o
```

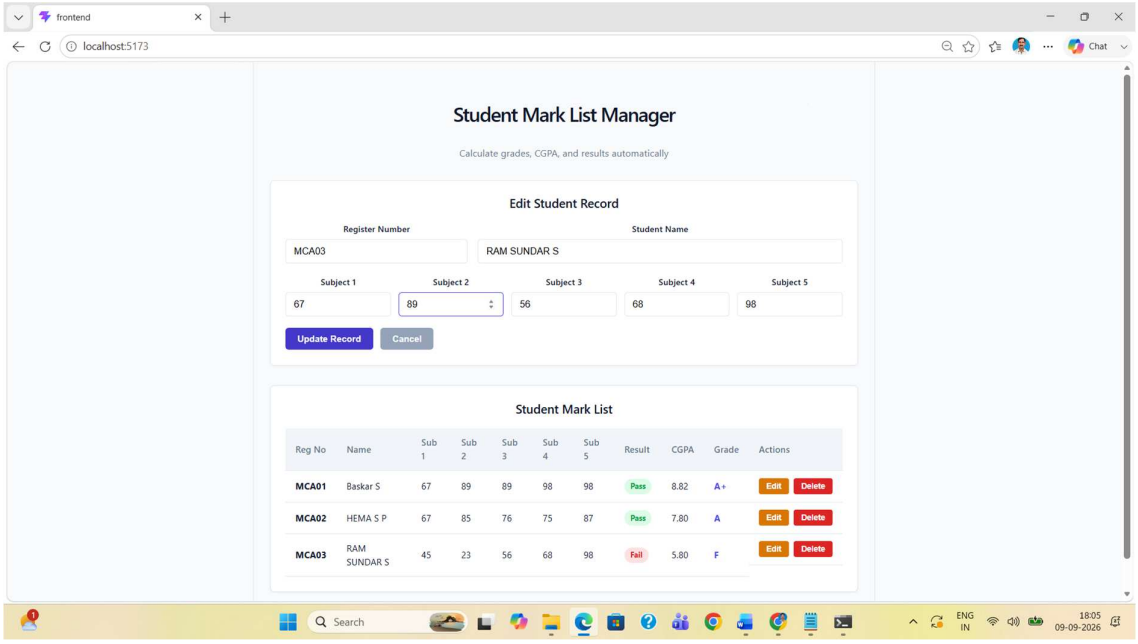
INPUT

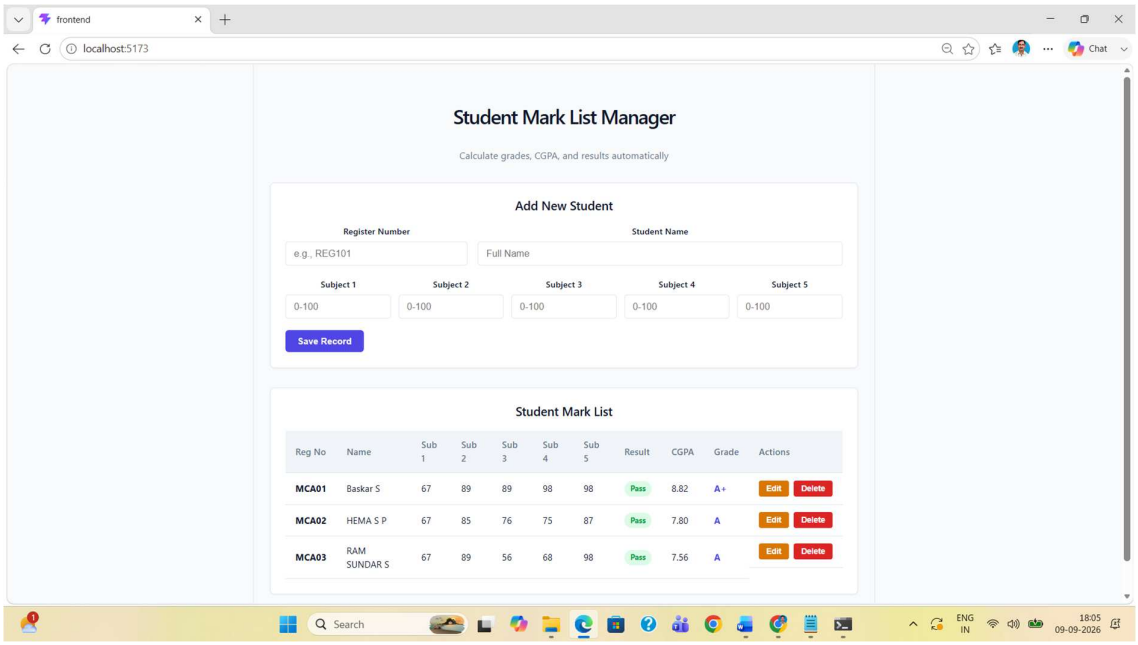


OUTPUT




frontend
localhost:5173
Chat
Student Mark List Manager
Calculate grades, CGPA, and results automatically
Add New Student
Register Number
Student Name
e.g., REG101
Full Name
Subject 1
Subject 2
Subject 3
Subject 4
Subject 5
0-100
Save Record
Student Mark List
Reg No
Name
Sub 1
Sub 2
Sub 3
Sub 4
Sub 5
Result
CGPA
Grade
Actions
MCA01
Baskar S
67 89 89 98 98
Pass
8.82
A+
Edit
Delete
MCA02
HEMA S P
67 85 76 75 87
Pass
7.80
A
Edit
Delete
MCA03
RAM SUNDAR S
45 23 56 68 98
Fail
5.80
F
Edit
Delete
Search
ENG IN
18:04
09-09-2026


frontend
localhost:5173
Chat
Student Mark List Manager
Calculate grades, CGPA, and results automatically
Edit Student Record
Register Number
Student Name
MCA03
RAM SUNDAR S
Subject 1
Subject 2
Subject 3
Subject 4
Subject 5
67
89
56
68
98
Update Record
Cancel
Student Mark List
Reg No
Name
Sub 1
Sub 2
Sub 3
Sub 4
Sub 5
Result
CGPA
Grade
Actions
MCA01
Baskar S
67 89 89 98 98
Pass
8.82
A+
Edit
Delete
MCA02
HEMA S P
67 85 76 75 87
Pass
7.80
A
Edit
Delete
MCA03
RAM SUNDAR S
45 23 56 68 98
Fail
5.80
F
Edit
Delete
Search
ENG IN
18:05
09-09-2026



12. Develop a simple CRUD application (Create, Read, Update, and Delete) using ReactJS for employee payroll generation with Employee ID, Name, Basic Pay, DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay. DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay are calculated whenever a new employee record is inserted or updated in the MySQL Database.

To create a ReactJS application, do the following:

Launch the WAMP/XAMPP Application Server and then execute the following script in MySQL to set up the payroll_db database and employees table:

SQL Commands

```
CREATE DATABASE IF NOT EXISTS payroll_db;

USE payroll_db;

CREATE TABLE employees (
    empId VARCHAR(50) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    basicPay DECIMAL(10,2) NOT NULL,
    da DECIMAL(10,2),
    hra DECIMAL(10,2),
    epf DECIMAL(10,2),
    loan DECIMAL(10,2),
    tax DECIMAL(10,2),
    grossPay DECIMAL(10,2),
    netPay DECIMAL(10,2)
);
```

Suppose any database access issue is shown in the frontend, we need to do the following:

```
USE payroll_db;

-- Option A: Drop and recreate the table with 'empId'
DROP TABLE IF EXISTS employees;

CREATE TABLE employees (
    empId VARCHAR(50) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    basicPay DECIMAL(10,2) NOT NULL,
    da DECIMAL(10,2),
    hra DECIMAL(10,2),
    epf DECIMAL(10,2),
    loan DECIMAL(10,2),
    tax DECIMAL(10,2),
    grossPay DECIMAL(10,2),
    netPay DECIMAL(10,2)
);
```

Open a command prompt and do the following:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**mkdir employee-payroll-MySQL**

C:\ReactJSProjects>**cd employee-payroll-MySQL**

C:\ReactJSProjects\employee-payroll-MySQL>**mkdir backend**

C:\ReactJSProjects\employee-payroll-MySQL>**cd backend**

C:\ReactJSProjects\employee-payroll-MySQL\backend>

Backend (server.js)
Initialize your Node.js project (npm init -y) and install the required dependencies (npm install express mysql2 cors).

server.js

```
const express = require('express');
const mysql = require('mysql2');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

// MySQL Connection
const db = mysql.createConnection({
```

```
  host: 'localhost',
  user: 'root',       // Your MySQL username
  password: '',       // Your MySQL password
  database: 'payroll_db'
});

db.connect((err) => {
  if (err) {
    console.error('MySQL connection failed:', err);
    return;
  }
  console.log('MySQL Connected...');
});

// Helper for payroll calculations
const calculatePayroll = (basic, loanVal) => {
  const basicPay = parseFloat(basic) || 0;
  const loan = parseFloat(loanVal) || 0;

  const da = basicPay * 0.10;
  const hra = basicPay * 0.20;
  const epf = basicPay * 0.12;
  const tax = basicPay > 20000 ? 200 : 0;

  const grossPay = basicPay + da + hra;
  const netPay = grossPay - (epf + loan + tax);

  return { basicPay, da, hra, epf, loan, tax, grossPay, netPay };
};

// READ: Get all employees
app.get('/api/employees', (req, res) => {
  db.query('SELECT * FROM employees', (err, results) => {
    if (err) {
      console.error('GET Error:', err);
      return res.status(500).send(err);
    }
    res.json(results);
  });
});

// CREATE: Add new employee
app.post('/api/employees', (req, res) => {
  const { empId, name, basicPay, loan } = req.body;
  const computed = calculatePayroll(basicPay, loan);

  const query = `INSERT INTO employees
    (empId, name, basicPay, da, hra, epf, loan, tax, grossPay, netPay)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`;
```

```
    const values = [empId, name, computed.basicPay, computed.da, computed.hra, computed.epf, computed.loan, computed.tax, computed.grossPay, computed.netPay];

  db.query(query, values, (err) => {
    if (err) {
      console.error('POST Error:', err.message); // Displays exact SQL issue in console
      return res.status(400).json({ error: err.message });
    }
    res.status(201).json({ message: 'Employee added successfully' });
  });
});

// UPDATE: Update existing employee
app.put('/api/employees/:empId', (req, res) => {
  const { name, basicPay, loan } = req.body;
  const computed = calculatePayroll(basicPay, loan);

  const query = `UPDATE employees SET
      name=?, basicPay=?, da=?, hra=?, epf=?, loan=?, tax=?, grossPay=?, netPay=?
    WHERE empId=?`;

   const values = [name, computed.basicPay, computed.da, computed.hra, computed.epf, computed.loan, computed.tax, computed.grossPay, computed.netPay, req.params.empId];

  db.query(query, values, (err) => {
    if (err) {
      console.error('PUT Error:', err);
      return res.status(500).send(err);
    }
    res.json({ message: 'Employee updated successfully' });
  });
});

// DELETE: Remove employee
app.delete('/api/employees/:empId', (req, res) => {
  db.query('DELETE FROM employees WHERE empId=?', [req.params.empId], (err) => {
    if (err) {
      console.error('DELETE Error:', err);
      return res.status(500).send(err);
    }
    res.json({ message: 'Employee deleted successfully' });
  });
});

app.listen(5000, () => console.log('Server running on port 5000'));
```

App.css

```
.payroll-container {
  padding: 20px;
  font-family: sans-serif;
  max-width: 1100px;
  margin: 0 auto;
}

.payroll-title {
  margin-bottom: 20px;
}

/* Form Styling */
.payroll-form {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
  flex-wrap: wrap;
}

.form-input {
  padding: 8px 12px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.form-input:disabled {
  background-color: #e9ecef;
  cursor: not-allowed;
}

.btn-submit {
  cursor: pointer;
  color: #fff;
  border: none;
  padding: 8px 16px;
  border-radius: 4px;
  font-weight: bold;
}

.btn-add {
  background-color: #16a34a;
}

.btn-update {
  background-color: #d97706;
}
```

```
.btn-cancel {
  cursor: pointer;
  padding: 8px 16px;
  border: 1px solid #ccc;
  background-color: #f3f4f6;
  border-radius: 4px;
}

/* Table Styling */
.payroll-table {
  width: 100%;
  text-align: left;
  border-collapse: collapse;
}

.payroll-table th,
.payroll-table td {
  border: 1px solid #ddd;
  padding: 8px;
}

.payroll-table thead tr {
  background-color: #f2f2f2;
}

.btn-edit {
  margin-right: 6px;
  cursor: pointer;
  padding: 4px 8px;
}

.btn-delete {
  color: red;
  cursor: pointer;
  padding: 4px 8px;
}

.no-records {
  text-align: center;
}
```

App.jsx

```
import React, { useState, useEffect } from 'react';
import './App.css';

export default function EmployeePayroll() {
  const [employees, setEmployees] = useState([]);
  const initialForm = { empId: '', name: '', basicPay: '', loan: '' };
  const [formData, setFormData] = useState(initialForm);
```

```
const [isEditing, setIsEditing] = useState(false);

// READ: Fetch employees from MySQL Backend
const fetchEmployees = async () => {
  try {
    const res = await fetch('http://localhost:5000/api/employees');
    const data = await res.json();
    setEmployees(data);
  } catch (err) {
    console.error('Error fetching data:', err);
  }
};

useEffect(() => {
  fetchEmployees();
}, []);

const handleChange = (e) => {
  setFormData({ ...formData, [e.target.name]: e.target.value });
};

// CREATE or UPDATE Record
const handleSubmit = async (e) => {
  e.preventDefault();
  if (!formData.empId || !formData.name || !formData.basicPay) return;

  try {
    if (isEditing) {
      // UPDATE
      await fetch(`http://localhost:5000/api/employees/${formData.empId}`,
       {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData),
      });
      setIsEditing(false);
    } else {
      // CREATE
      const res = await fetch('http://localhost:5000/api/employees', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(formData),
      });
      if (!res.ok) {
        alert('Employee ID already exists!');
        return;
      }
    }
    setFormData(initialForm);
    fetchEmployees(); // Refresh data table
  } catch (err) {
```

```
    console.error('Error submitting form:', err);
  }
};

// Populate form for Editing
const handleEdit = (emp) => {
  setFormData({
    empId: emp.empId,
    name: emp.name,
    basicPay: emp.basicPay,
    loan: emp.loan,
  });
  setIsEditing(true);
};

// DELETE Record
const handleDelete = async (empId) => {
  try {
    await fetch(`http://localhost:5000/api/employees/${empId}`, {
      method: 'DELETE',
    });
    fetchEmployees(); // Refresh data table
  } catch (err) {
    console.error('Error deleting employee:', err);
  }
};

return (
  <div className="payroll-container">
      <h2 className="payroll-title">Employee Payroll Management System
      (MySQL Integrated)</h2>

    {/* Input Form */}
    <form onSubmit={handleSubmit} className="payroll-form">
      <input
        type="text"
        name="empId"
        placeholder="Employee ID"
        className="form-input"
        value={formData.empId}
        onChange={handleChange}
        disabled={isEditing}
        required
      />
      <input
        type="text"
        name="name"
        placeholder="Employee Name"
        className="form-input"
        value={formData.name}
        onChange={handleChange}
```

```
    required
  />
  <input
    type="number"
    name="basicPay"
    placeholder="Basic Pay"
    className="form-input"
    value={formData.basicPay}
    onChange={handleChange}
    required
  />
  <input
    type="number"
    name="loan"
    placeholder="Loan Deduction"
    className="form-input"
    value={formData.loan}
    onChange={handleChange}
  />
  <button
    type="submit"
    className={`btn-submit ${isEditing ? 'btn-update' : 'btn-add'}`}
  >
    {isEditing ? 'Update Employee' : 'Add Employee'}
  </button>
  {isEditing && (
    <button
      type="button"
      className="btn-cancel"
      onClick={() => {
        setIsEditing(false);
        setFormData(initialForm);
      }}
    >
      Cancel
    </button>
  )}
</form>

{/* Payroll Table */}
<table className="payroll-table">
  <thead>
    <tr>
      <th>Emp ID</th>
      <th>Name</th>
      <th>Basic Pay</th>
      <th>DA</th>
      <th>HRA</th>
      <th>EPF</th>
      <th>Loan</th>
      <th>Prof. Tax</th>
```

```
            <th>Gross Pay</th>
            <th>Net Pay</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {employees.map((emp) => (
            <tr key={emp.empId}>
              <td>{emp.empId}</td>
              <td>{emp.name}</td>
              <td>{Number(emp.basicPay).toFixed(2)}</td>
              <td>{Number(emp.da).toFixed(2)}</td>
              <td>{Number(emp.hra).toFixed(2)}</td>
              <td>{Number(emp.epf).toFixed(2)}</td>
              <td>{Number(emp.loan).toFixed(2)}</td>
              <td>{Number(emp.tax).toFixed(2)}</td>
              <td><strong>{Number(emp.grossPay).toFixed(2)}</strong></td>
              <td><strong>{Number(emp.netPay).toFixed(2)}</strong></td>
              <td>
                <button onClick={() => handleEdit(emp)} className="btn-edit">
                  Edit
                </button>
                <button onClick={() => handleDelete(emp.empId)} className="btn-
          delete">
                  Delete
                </button>
              </td>
            </tr>
          ))}
          {employees.length === 0 && (
            <tr>
              <td colSpan="11" className="no-records">
                No employee records found.
              </td>
            </tr>
          )}
        </tbody>
      </table>
    </div>
  );
}
```

Steps to create and run the application

To run NodeJS, do the following steps in that command prompt:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**cd employee-payroll-MySQL**

C:\ReactJSProjects\employee-payroll-MySQL>**cd backend**

```
C:\ReactJSProjects\employee-payroll-MySQL\backend>dir
 Volume in drive C is Windows
 Volume Serial Number is EA37-06AE

 Directory of C:\ReactJSProjects\employee-payroll-MySQL\backend

09-09-2026  18:46    <DIR>          .
09-09-2026  18:43    <DIR>          ..
09-09-2026  18:46             3,653 server.js
               1 File(s)          3,653 bytes
               2 Dir(s)  55,889,281,024 bytes free

C:\ReactJSProjects\employee-payroll-MySQL\backend>node server.js
node:internal/modules/cjs/loader:1520
  throw err;
  ^

Error: Cannot find module 'express'
Require stack:
- C:\ReactJSProjects\employee-payroll-MySQL\backend\server.js
    at Module._resolveFilename (node:internal/modules/cjs/loader:1517:15)
    at wrapResolveFilename (node:internal/modules/cjs/loader:1071:27)
    at                                          defaultResolveImplForCJSLoading
(node:internal/modules/cjs/loader:1095:10)
    at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1122:12)
    at Module._load (node:internal/modules/cjs/loader:1294:5)
    at wrapModuleLoad (node:internal/modules/cjs/loader:255:19)
    at Module.require (node:internal/modules/cjs/loader:1617:12)
    at require (node:internal/modules/helpers:153:16)
    at         Object.<anonymous>         (C:\ReactJSProjects\employee-payroll-
MySQL\backend\server.js:1:17)
    at Module._compile (node:internal/modules/cjs/loader:1871:14) {
  code: 'MODULE_NOT_FOUND',
  requireStack:              [              'C:\\ReactJSProjects\\employee-payroll-
MySQL\\backend\\server.js' ]
}

Node.js v24.18.1

C:\ReactJSProjects\employee-payroll-MySQL\backend>npm install express
mysql2 cors

added 80 packages in 5s

30 packages are looking for funding
  run `npm fund` for details

C:\ReactJSProjects\employee-payroll-MySQL\backend>node server.js
Server running on http://localhost:5000
Connected to MySQL Database.
```

Open another command prompt and do the following:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**cd employee-payroll-MySQL**

C:\ReactJSProjects\employee-payroll-MySQL>**npm create vite@latest frontend -- --template react**

```
npm notice run npx
npm notice run create-vite frontend --template react
|
o  Which linter to use?
|  Oxlint
|
o  Install with npm and start now?
|  Yes
|
o  Scaffolding project in C:\ReactJSProjects\employee-payroll-MySQL\frontend...
|
o  Installing dependencies with npm...

added 24 packages, and audited 25 packages in 9s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
o  Starting dev server...
npm notice run frontend@0.0.0 dev
npm notice run vite

  VITE v8.2.2  ready in 1277 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
h

  Shortcuts
  press r + enter to restart the server
  press u + enter to show server url
  press o + enter to open in browser
  press c + enter to clear console
  press q + enter to quit
```

Suppose this application has already been created, then do the following steps:

C:\Users\periy>**cd C:\ReactJSProjects**

C:\ReactJSProjects>**cd employee-payroll-MySQL**

C:\ReactJSProjects\employee-payroll-MySQL>**cd frontend**

If the axios module is already installed for this project, but is no longer used for this project, we can uninstall it using the following command:

C:\ReactJSProjects\employee-payroll-MySQL\frontend>**npm uninstall axios**

removed 27 packages, and audited 25 packages in 2s

9 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

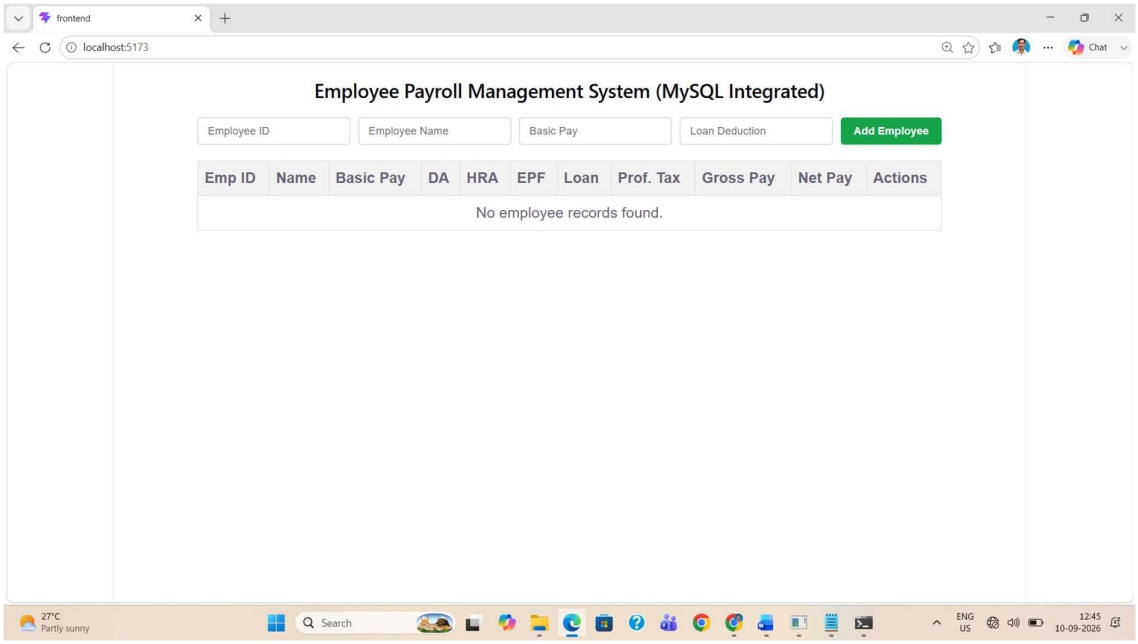
C:\ReactJSProjects\employee-payroll-MySQL\frontend>**npm run dev**
npm notice run frontend@0.0.0 dev
npm notice run vite

  VITE v8.2.2 ready in 378 ms

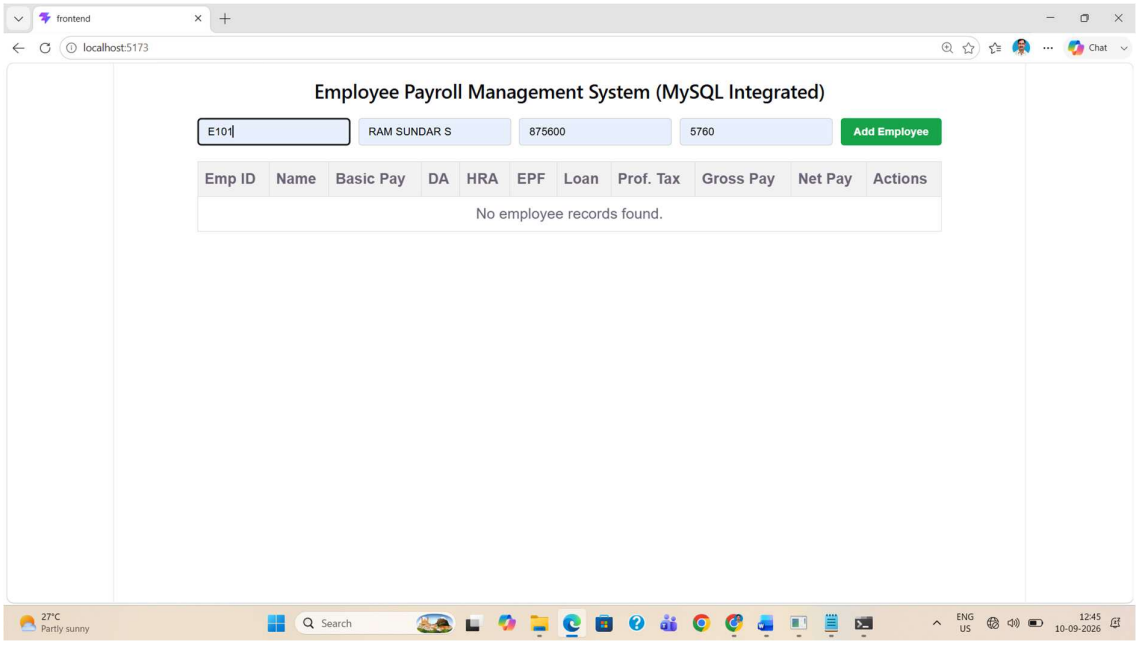
  ➜ Local: http://localhost:5173/
  ➜ Network: use --host to expose
  ➜ press h + enter to show help
h

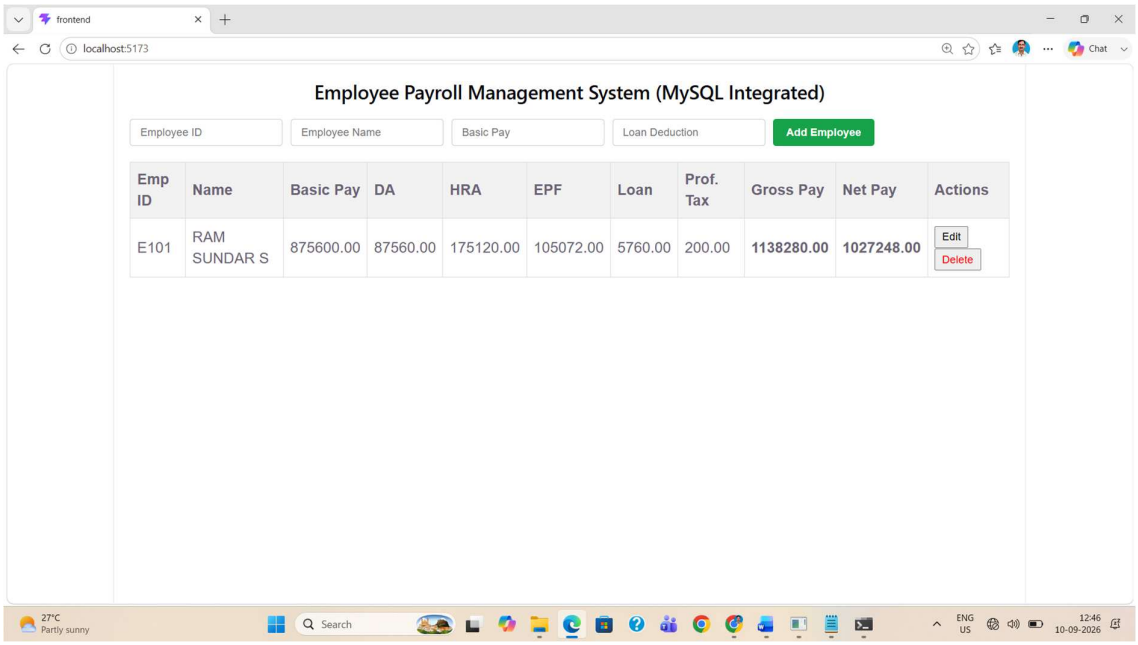
  Shortcuts
  press r + enter to restart the server
  press u + enter to show server url
  press o + enter to open in browser
  press c + enter to clear console
  press q + enter to quit
o

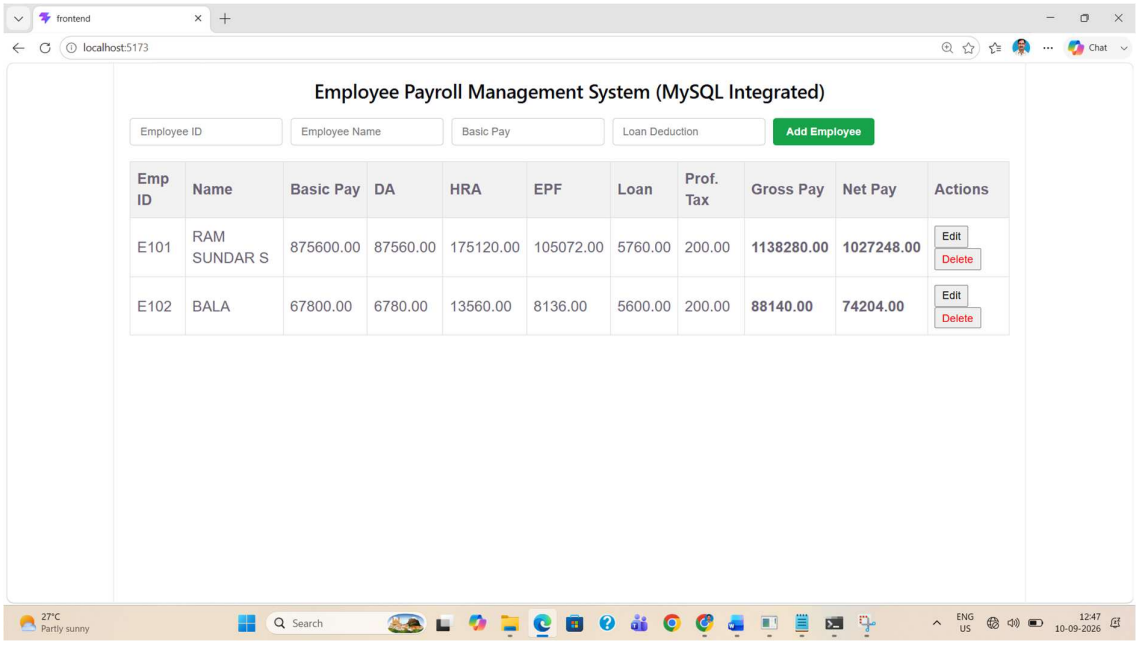
INPUT

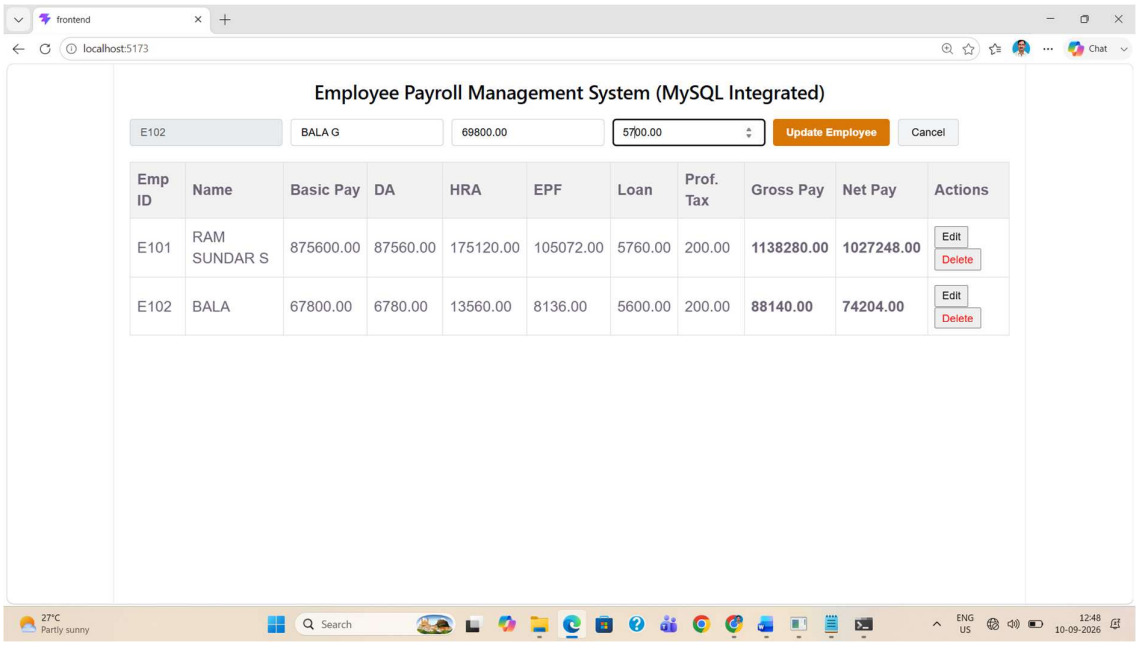


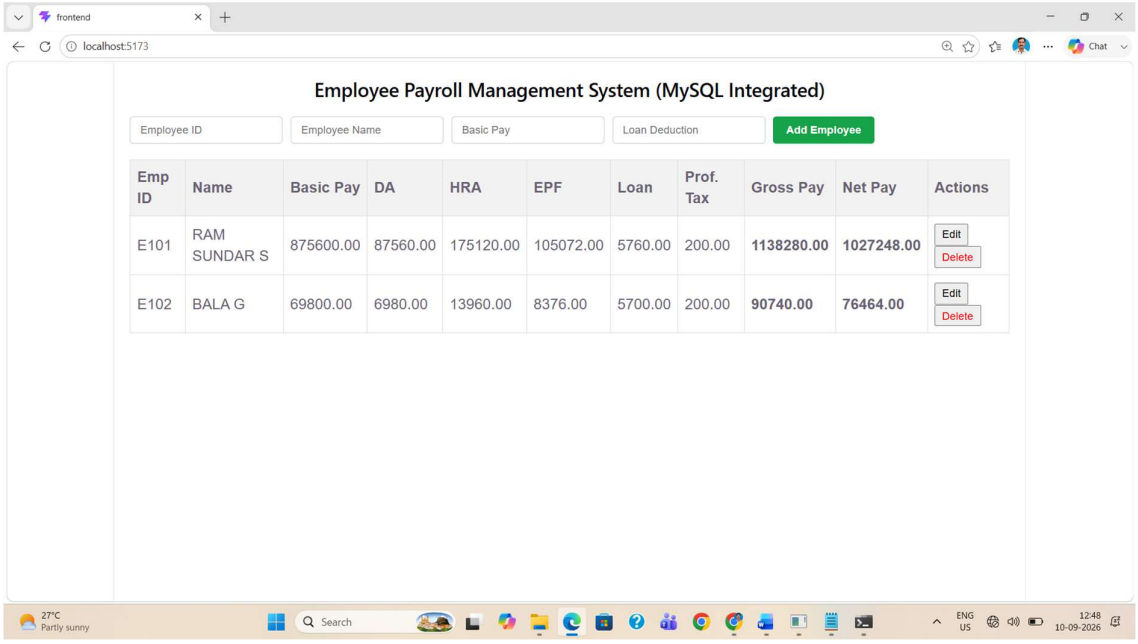
OUTPUT




frontend
localhost:5173
Chat
Employee Payroll Management System (MySQL Integrated)
Employee ID
Employee Name
Basic Pay
Loan Deduction
Add Employee
Emp ID
Name
Basic Pay
DA
HRA
EPF
Loan
Prof. Tax
Gross Pay
Net Pay
Actions
E101
RAM SUNDAR S
875600.00
87560.00
175120.00
105072.00
5760.00
200.00
1138280.00
1027248.00
Edit
Delete
27°C
Partly sunny
Search
ENG
US
12:46
10-09-2026


frontend
localhost:5173
Chat
Employee Payroll Management System (MySQL Integrated)
Employee ID
Employee Name
Basic Pay
Loan Deduction
Add Employee
Emp ID
Name
Basic Pay
DA
HRA
EPF
Loan
Prof. Tax
Gross Pay
Net Pay
Actions
E101
RAM SUNDAR S
875600.00
87560.00
175120.00
105072.00
5760.00
200.00
1138280.00
1027248.00
Edit
Delete
E102
BALA
67800.00
6780.00
13560.00
8136.00
5600.00
200.00
88140.00
74204.00
Edit
Delete
27°C
Partly sunny
Search
ENG
US
12:47
10-09-2026





NODE JS

13. Design and develop a Node.js web application utilizing HTML and CSS to create an interactive interface. Define a custom Node.js module to execute basic arithmetic operations (add, subtract, multiply, divide). The user enters two numerical values via form controls and clicks a specific operation button; the result must be dynamically rendered in a paragraph (<p>) element on the same page without refreshing.

[OR]

Develop a NodeJS web application with an HTML/CSS UI that performs arithmetic calculations using a custom module. The application should capture user input, process the selected operation

(Addition, Subtraction, Multiplication, Division) upon button submission, and display the result in a <p> tag on the same screen.

Project Structure

```
project-root/
│
├── calculator.js   # Custom Node.js module
├── server.js       # Main server file
├── public/
│   ├── index.html  # HTML frontend form
│   └── style.css   # Stylesheet
└── package.json
```

1. Custom Module (calculator.js)
Create this file to export the core arithmetic function that processes the mathematical operations.

calculator.js

```
exports.calculate = function (num1, num2, operation) {
  const a = parseFloat(num1);
  const b = parseFloat(num2);

  if (isNaN(a) || isNaN(b)) {
    return 'Please enter valid numbers.';
  }

  switch (operation) {
    case 'add':
      return `${a} + ${b} = ${a + b}`;
    case 'subtract':
      return `${a} - ${b} = ${a - b}`;
    case 'multiply':
      return `${a} × ${b} = ${a * b}`;
    case 'divide':
      return b !== 0 ? `${a} ÷ ${b} = ${a / b}` : 'Cannot divide by zero.';
    default:
      return 'Invalid Operation';
  }
};
```

2. Frontend HTML (public/index.html)
Create a simple HTML input form with separate submit buttons for each arithmetic operation.

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Node.js Arithmetic Calculator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="calculator-card">
    <h2>Arithmetic Calculator</h2>

    <form id="calcForm" onsubmit="return false;">
      <div class="input-group">
        <label for="num1">First Number:</label>
        <input type="number" step="any" id="num1" name="num1" required>
      </div>

      <div class="input-group">
        <label for="num2">Second Number:</label>
        <input type="number" step="any" id="num2" name="num2" required>
      </div>

      <div class="button-group">
            <button type="button" onclick="calculate('add')" class="btn btn-
add">Add (+)</button>
          <button type="button" onclick="calculate('subtract')" class="btn btn-
sub">Subtract (-)</button>
          <button type="button" onclick="calculate('multiply')" class="btn btn-
mul">Multiply (×)</button>
           <button type="button" onclick="calculate('divide')" class="btn btn-
div">Divide (÷)</button>
      </div>
    </form>

    <!-- Result displayed on same page in paragraph element -->
    <p id="result" class="result-text"></p>
  </div>

  <script>
    function calculate(operation) {
      const num1 = document.getElementById('num1').value;
      const num2 = document.getElementById('num2').value;
      const resultParagraph = document.getElementById('result');

      if (num1 === '' || num2 === '') {
        resultParagraph.textContent = "Please fill in both numbers.";
        resultParagraph.style.color = "#dc2626";
```

```
      return;
    }

    fetch('/calculate', {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify({ num1, num2, operation })
    })
    .then(response => response.json())
    .then(data => {
      resultParagraph.textContent = data.result;
      resultParagraph.style.color = "#2563eb";
    })
    .catch(error => {
      console.error('Error:', error);
      resultParagraph.textContent = "Server Error. Restart node server.";
      resultParagraph.style.color = "#dc2626";
    });
  }
</script>

</body>
</html>
```

3. Styling (public/style.css)
Add custom CSS to style the form layout, inputs, and styled submit action buttons.

public/style.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f7f6;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}

.calculator-card {
  background: #ffffff;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
  width: 320px;
}

h2 {
  text-align: center;
```

```
  margin-bottom: 20px;
  color: #333;
}

.input-group {
  margin-bottom: 15px;
}

.input-group label {
  display: block;
  margin-bottom: 5px;
  color: #666;
}

.input-group input {
  width: 100%;
  padding: 8px 10px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.button-group {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
  margin-top: 20px;
}

.btn {
  padding: 10px;
  border: none;
  border-radius: 4px;
  color: white;
  font-weight: bold;
  cursor: pointer;
  transition: opacity 0.2s;
}

.btn:hover { opacity: 0.9; }
.btn-add { background-color: #16a34a; }
.btn-sub { background-color: #d97706; }
.btn-mul { background-color: #2563eb; }
.btn-div { background-color: #dc2626; }

/* Paragraph Output Styling */
.result-text {
  margin-top: 20px;
  padding: 10px;
  background-color: #f8fafc;
  border: 1px solid #e2e8f0;
```

```
  border-radius: 4px;
  font-size: 1.1em;
  font-weight: bold;
  text-align: center;
  min-height: 24px;
}
```

4. Main Application Server (server.js)
Use Express to serve static frontend files, handle POST requests from the form, import the custom calculator.js module, and display the result.

server.js

```
const express = require('express');
const path = require('path');
const calculator = require('./calculator');

const app = express();

// Required to read JSON sent from frontend JS
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.post('/calculate', (req, res) => {
  const { num1, num2, operation } = req.body;

  // Perform calculation via custom module
  const result = calculator.calculate(num1, num2, operation);

  // Return result
  res.json({ result: result });
});

const PORT = 3000;
app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
```

How to Run the Application

C:\Users\periy>**cd C:\NodeJSExamples**

C:\NodeJSExamples>**mkdir Simple-Arithmetic**

C:\NodeJSExamples>**cd Simple-Arithmetic**

C:\NodeJSExamples\Simple-Arithmetic>**dir**
 Volume in drive C is Windows
 Volume Serial Number is EA37-06AE

 Directory of C:\NodeJSExamples\Simple-Arithmetic

```
10-09-2026  13:11    <DIR>          .
10-09-2026  13:09    <DIR>          ..
10-09-2026  13:10               506 calculator.js
10-09-2026  13:11             1,619 server.js
               2 File(s)          2,125 bytes
               2 Dir(s)  62,802,857,984 bytes free

C:\NodeJSExamples\Simple-Arithmetic>mkdir public

C:\NodeJSExamples\Simple-Arithmetic>cd public

C:\NodeJSExamples\Simple-Arithmetic\public>dir
 Volume in drive C is Windows
 Volume Serial Number is EA37-06AE

 Directory of C:\NodeJSExamples\Simple-Arithmetic\public

10-09-2026  13:13    <DIR>          .
10-09-2026  13:12    <DIR>          ..
10-09-2026  13:13             1,223 index.html
10-09-2026  13:13             1,291 style.css
               2 File(s)          2,514 bytes
               2 Dir(s)  62,801,829,888 bytes free

C:\NodeJSExamples\Simple-Arithmetic\public>cd ..

C:\NodeJSExamples\Simple-Arithmetic>dir
 Volume in drive C is Windows
 Volume Serial Number is EA37-06AE

 Directory of C:\NodeJSExamples\Simple-Arithmetic

10-09-2026  13:12    <DIR>          .
10-09-2026  13:09    <DIR>          ..
10-09-2026  13:10               506 calculator.js
10-09-2026  13:13    <DIR>          public
10-09-2026  13:11             1,619 server.js
               2 File(s)          2,125 bytes
               3 Dir(s)  62,801,784,832 bytes free
```

Initialize Project & Install Express

```
C:\NodeJSExamples\Simple-Arithmetic>npm init -y
Wrote to C:\NodeJSExamples\Simple-Arithmetic\package.json:

{
  "name": "simple-arithmetic",
  "version": "1.0.0",
  "description": "",
  "main": "calculator.js",
  "scripts": {
```

```
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs"
}
```

C:\NodeJSExamples\Simple-Arithmetic>**npm install express**

added 68 packages, and audited 69 packages in 3s

27 packages are looking for funding
  run `npm fund` for details

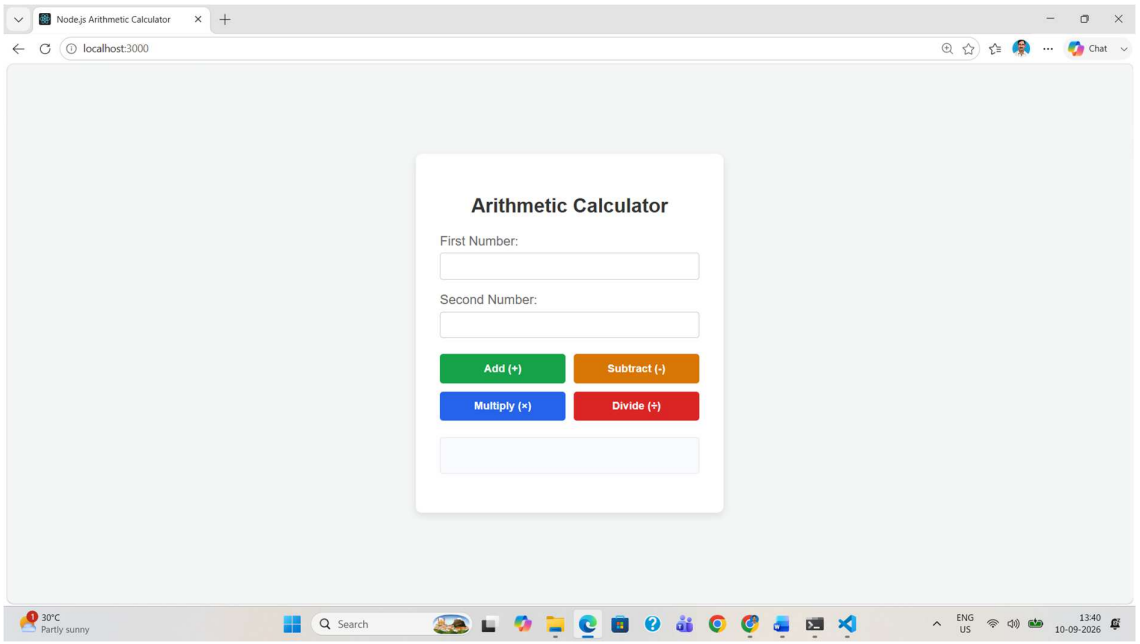
found 0 vulnerabilities

Start the Server

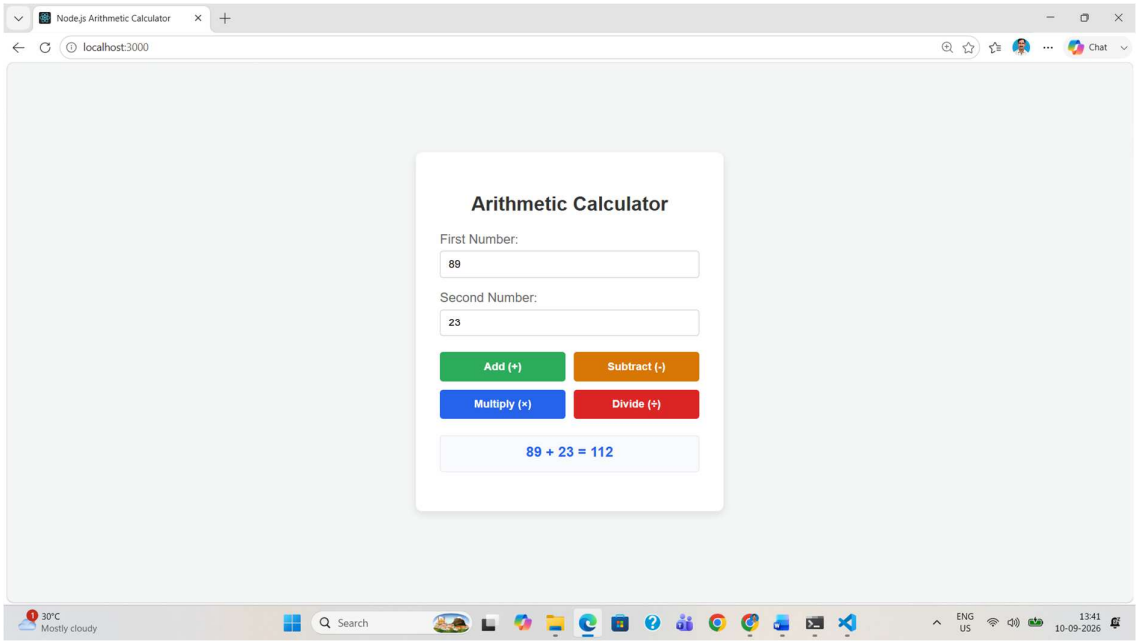
C:\NodeJSExamples\Simple-Arithmetic>**node server.js**
Server running on http://localhost:3000

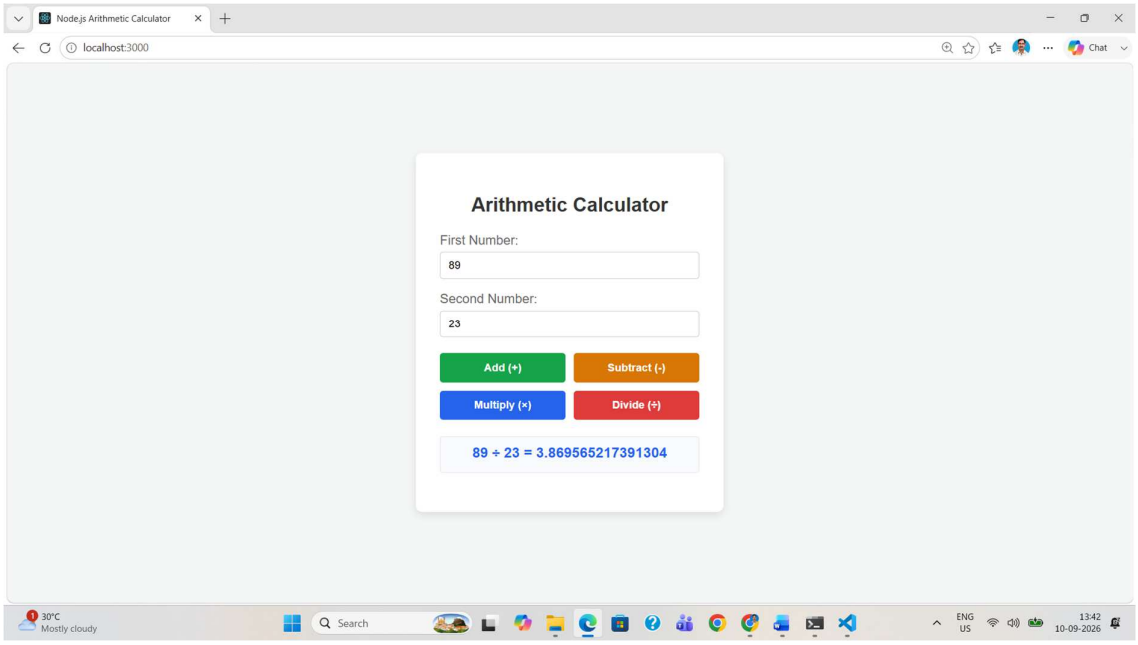
Open the browser to navigate the URL: http://localhost:3000

INPUT



OUTPUT





14. Develop a NodeJS Application using HTML and CSS to display all or specific records in a web page from a MySQL Database table based on user inputs through HTML input and submit button controls.

1. MySQL Database Setup

Run this script in MySQL Workbench or a terminal to create the database and sample records:

SQL Commands

```
CREATE DATABASE IF NOT EXISTS company_db;
USE company_db;
```

```
CREATE TABLE IF NOT EXISTS employees (
    empId VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    department VARCHAR(50) NOT NULL,
    salary DECIMAL(10,2) NOT NULL
);

INSERT INTO employees (empId, name, department, salary) VALUES
('E101', 'John Doe', 'Engineering', 75000.00),
('E102', 'Jane Smith', 'Marketing', 62000.00),
('E103', 'Alice Johnson', 'HR', 58000.00),
('E104', 'Bob Brown', 'Finance', 80000.00);
```

2. File Structure

```
project-root/
│
├── server.js         # Express server & MySQL query logic
├── package.json
└── public/
    ├── index.html    # Frontend layout with input form & table
    └── style.css     # UI styling
```

3. Frontend HTML (public/index.html)

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Employee Record Search</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <h2>Employee Database Lookup</h2>

    <!-- Input Form -->
    <form id="searchForm" onsubmit="return false;">
      <div class="input-group">
        <label for="empId">Employee ID (Leave blank to fetch all):</label>
        <input type="text" id="empId" placeholder="e.g. E101">
      </div>

      <div class="button-group">
```

```
      <button type="button" onclick="fetchRecords(false)" class="btn btn-search">Search Specific ID</button>
      <button type="button" onclick="fetchRecords(true)" class="btn btn-all">Display All Records</button>
    </div>
  </form>

  <p id="statusMsg" class="status-msg"></p>

  <!-- Display Table -->
  <table id="recordsTable" style="display: none;">
    <thead>
      <tr>
        <th>Emp ID</th>
        <th>Name</th>
        <th>Department</th>
        <th>Salary ($)</th>
      </tr>
    </thead>
    <tbody id="tableBody"></tbody>
  </table>
 </div>

 <script>
  async function fetchRecords(fetchAll) {
    const empId = document.getElementById('empId').value.trim();
    const statusMsg = document.getElementById('statusMsg');
    const table = document.getElementById('recordsTable');
    const tableBody = document.getElementById('tableBody');

    // Reset state
    tableBody.innerHTML = '';
    statusMsg.textContent = '';

    if (!fetchAll && !empId) {
      statusMsg.textContent = "Please enter an Employee ID or click 'Display All Records'.";
      statusMsg.style.color = "#dc2626";
      table.style.display = "none";
      return;
    }

    // Construct API endpoint URL
    const url = fetchAll ? '/api/employees' : `/api/employees/${encodeURIComponent(empId)}`;

    try {
      const response = await fetch(url);
      const data = await response.json();

      if (!response.ok || data.length === 0) {
```

```
        statusMsg.textContent = data.message || "No records found.";
        statusMsg.style.color = "#dc2626";
        table.style.display = "none";
        return;
      }

      // Render records into HTML table rows
      data.forEach(emp => {
        const row = document.createElement('tr');
        row.innerHTML = `
          <td>${emp.empId}</td>
          <td>${emp.name}</td>
          <td>${emp.department}</td>
          <td>${parseFloat(emp.salary).toFixed(2)}</td>
        `;
        tableBody.appendChild(row);
      });

      table.style.display = "table";
      statusMsg.textContent = `Displaying ${data.length} record(s).`;
      statusMsg.style.color = "#16a34a";
    } catch (err) {
      statusMsg.textContent = "Error retrieving data from backend server.";
      statusMsg.style.color = "#dc2626";
      table.style.display = "none";
    }
  }
 </script>

</body>
</html>
```

4. CSS Stylesheet (public/style.css)

style.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f7f6;
  display: flex;
  justify-content: center;
  padding: 40px 20px;
  margin: 0;
}

.container {
  background: #ffffff;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
  width: 100%;
```

```
  max-width: 650px;
}

h2 {
  text-align: center;
  margin-bottom: 25px;
  color: #333;
}

.input-group {
  margin-bottom: 15px;
}

.input-group label {
  display: block;
  margin-bottom: 6px;
  color: #555;
  font-size: 0.9em;
}

.input-group input {
  width: 100%;
  padding: 9px 12px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.button-group {
  display: flex;
  gap: 10px;
  margin-top: 15px;
}

.btn {
  flex: 1;
  padding: 10px;
  border: none;
  border-radius: 4px;
  color: white;
  font-weight: bold;
  cursor: pointer;
  transition: opacity 0.2s;
}

.btn:hover { opacity: 0.9; }
.btn-search { background-color: #2563eb; }
.btn-all { background-color: #16a34a; }

.status-msg {
  margin: 15px 0;
```

```
  font-weight: bold;
  text-align: center;
}

table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 15px;
}

th, td {
  border: 1px solid #e2e8f0;
  padding: 10px;
  text-align: left;
}

th {
  background-color: #f8fafc;
  color: #334155;
}

tr:nth-child(even) {
  background-color: #f1f5f9;
}
```

5. Backend Server (server.js)

server.js

```
const express = require('express');
const mysql = require('mysql2');
const path = require('path');

const app = express();

// Serve frontend static files from the 'public' folder
app.use(express.static(path.join(__dirname, 'public')));
app.use(express.json());

// MySQL Database Connection Configuration
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',       // Your MySQL username
  password: '',       // Your MySQL password
  database: 'company_db'
});

db.connect((err) => {
  if (err) {
    console.error('Database connection failed:', err.message);
```

```
    return;
  }
  console.log('Connected to MySQL Database...');
});

// ROUTE 1: Fetch ALL employee records
app.get('/api/employees', (req, res) => {
  const sql = 'SELECT * FROM employees';
  db.query(sql, (err, results) => {
    if (err) return res.status(500).json({ message: 'Database error occurred.' });
    res.json(results);
  });
});

// ROUTE 2: Fetch a SPECIFIC employee record by ID
app.get('/api/employees/:empId', (req, res) => {
  const sql = 'SELECT * FROM employees WHERE empId = ?';
  db.query(sql, [req.params.empId], (err, results) => {
    if (err) return res.status(500).json({ message: 'Database error occurred.' });

    if (results.length === 0) {
      return res.status(404).json({ message: 'Employee ID not found.' });
    }

    res.json(results);
  });
});

const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

Installation and Execution

```
C:\NodeJSExamples>mkdir employees-MySQL

C:\NodeJSExamples>cd employees-MySQL

C:\NodeJSExamples\employees-MySQL>mkdir public
```

Initialize Project & Install Dependencies

```
C:\NodeJSExamples\employees-MySQL>npm init -y
Wrote to C:\NodeJSExamples\employees-MySQL\package.json:

{
  "name": "employees-mysql",
  "version": "1.0.0",
  "description": "",
```

```
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs"
}
```

```
C:\NodeJSExamples\employees-MySQL>npm install express mysql2

added 78 packages, and audited 79 packages in 5s

29 packages are looking for funding
  run `npm fund` for details

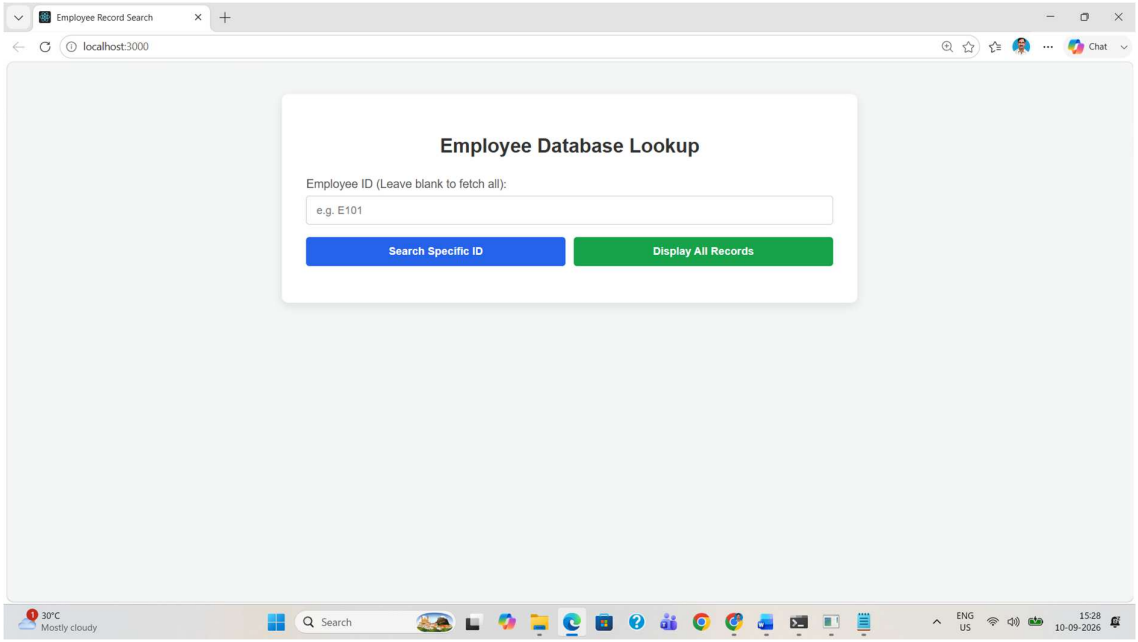
found 0 vulnerabilities
```

Start the Application

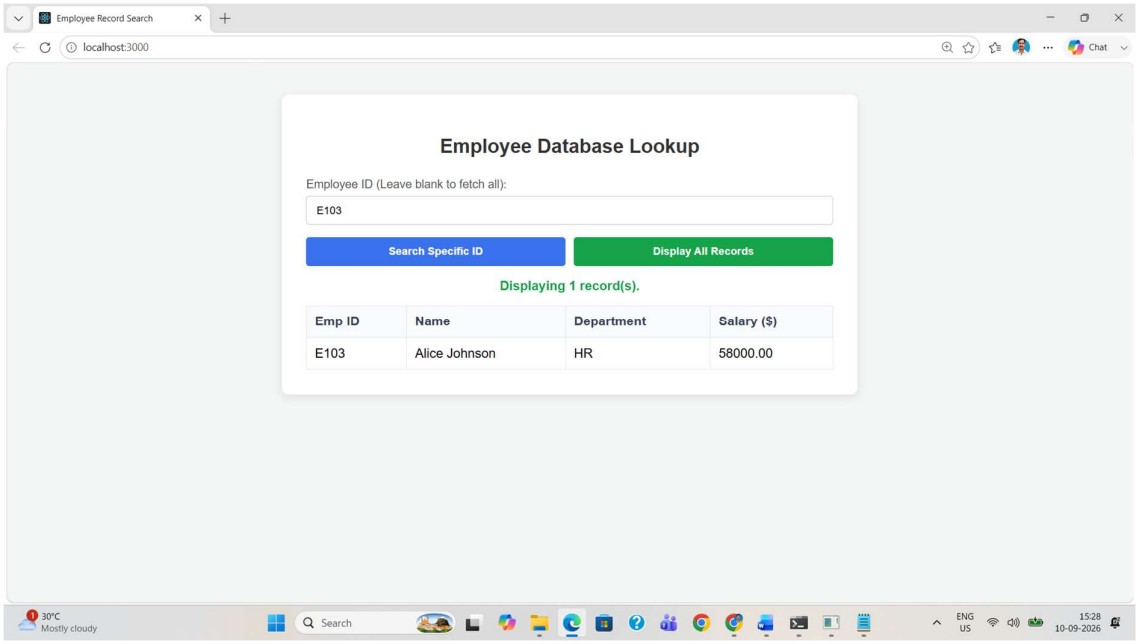
```
C:\NodeJSExamples\employees-MySQL>node server.js
Server running at http://localhost:3000
Connected to MySQL Database...
```

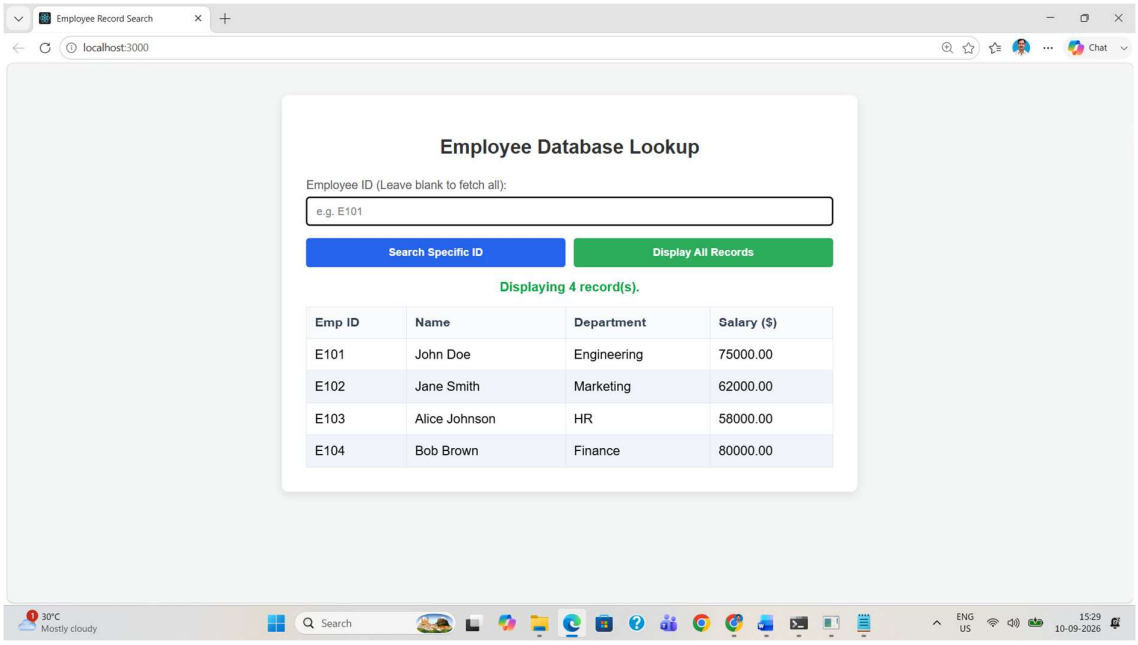
Open the browser to navigate the URL: http://localhost:3000

INPUT



OUTPUT


Employee Record Search
localhost:3000
Chat
Employee Database Lookup
Employee ID (Leave blank to fetch all):
E103
Search Specific ID
Display All Records
Displaying 1 record(s).
Emp ID
Name
Department
Salary ($)
E103
Alice Johnson
HR
58000.00
30°C
Mostly cloudy
Search
ENG
US
15:28
10-09-2026


Employee Record Search
localhost:3000
Chat
Employee Database Lookup
Employee ID (Leave blank to fetch all):
e.g. E101
Search Specific ID
Display All Records
Displaying 4 record(s).
Emp ID
Name
Department
Salary ($)
E101
John Doe
Engineering
75000.00
E102
Jane Smith
Marketing
62000.00
E103
Alice Johnson
HR
58000.00
E104
Bob Brown
Finance
80000.00
30°C
Mostly cloudy
Search
ENG
US
15:29
10-09-2026

15. Develop a simple CRUD application (Create, Read, Update, and Delete) using only NodeJS with HTML5 and CSS3 for employee payroll generation with Employee ID, Name, Basic Pay, DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay. DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay are calculated whenever a new employee record is inserted or updated in the MySQL Database.

Step 1: Project Setup

1. Create a new directory for your project and navigate into it:

   C:\Users\periy>**cd C:\NodeJSExamples**

   C:\NodeJSExamples>**mkdir employees-MySQL-CRUD**

   C:\NodeJSExamples>**cd employees-MySQL-CRUD**

2. Initialize a Node.js project:

   C:\NodeJSExamples\employees-MySQL-CRUD>**npm init -y**
   Wrote to C:\NodeJSExamples\employees-MySQL-CRUD\package.json:

```
{
  "name": "employees-mysql-crud",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs"
}
```

3. Install the required dependencies (express, mysql2, and body-parser):

   C:\NodeJSExamples\employees-MySQL-CRUD>**npm install express mysql2 body-parser**

   added 76 packages, and audited 77 packages in 4s

   29 packages are looking for funding
     run `npm fund` for details

   found 0 vulnerabilities

   C:\NodeJSExamples\employees-MySQL-CRUD>

4. Create the required file and folder structure:

C:\NodeJSExamples\employees-MySQL-CRUD>**mkdir public**



Step 2: Database Setup
Run the SQL script provided in your prompt in your MySQL database to create the database and table:

SQL Commands

```
CREATE DATABASE IF NOT EXISTS payroll_db;
USE payroll_db;

DROP TABLE IF EXISTS employees;

CREATE TABLE employees (
  empId VARCHAR(50) PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  basicPay DECIMAL(10, 2) NOT NULL DEFAULT 0.00,
  da DECIMAL(10, 2) DEFAULT 0.00,
  hra DECIMAL(10, 2) DEFAULT 0.00,
  epf DECIMAL(10, 2) DEFAULT 0.00,
  loan DECIMAL(10, 2) DEFAULT 0.00,
  tax DECIMAL(10, 2) DEFAULT 0.00,
  grossPay DECIMAL(10, 2) DEFAULT 0.00,
  netPay DECIMAL(10, 2) DEFAULT 0.00
);
```

Step 3: Backend (server.js)
Create server.js in the root directory. Make sure to update your MySQL credentials (host, user, password, database) as per your local setup.

server.js

```
const express = require('express');
const mysql = require('mysql2');
const bodyParser = require('body-parser');
const path = require('path');

const app = express();
const PORT = 3000;

// Middleware
```

```
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
app.use(express.static(path.join(__dirname, 'public')));

// MySQL Connection Pool
const db = mysql.createPool({
    host: 'localhost',
    user: 'root',
    password: '', // Replace with your MySQL password
    database: 'payroll_db'
});

// Helper function to calculate payroll components
function calculatePayroll(basicPay, loanInput) {
    const basic = parseFloat(basicPay) || 0;
    const loan = parseFloat(loanInput) || 0;

    // Standard business logic rules for calculation:
    const da = basic * 0.10;       // 10% of Basic Pay
    const hra = basic * 0.20;      // 20% of Basic Pay
    const epf = basic * 0.12;      // 12% of Basic Pay
    const tax = 200.00;            // Fixed Professional Tax

    // Gross Pay = Basic Pay + DA + HRA
    const grossPay = basic + da + hra;

    // Net Pay = Gross Pay - (EPF + Professional Tax + Loan)
    const netPay = grossPay - (epf + tax + loan);

    return {
        da: da.toFixed(2),
        hra: hra.toFixed(2),
        epf: epf.toFixed(2),
        tax: tax.toFixed(2),
        grossPay: grossPay.toFixed(2),
        netPay: netPay.toFixed(2)
    };
}

// 1. CREATE & UPDATE (Save Employee Route)
app.post('/api/employees', (req, res) => {
    const { empId, name, basicPay, loan, isUpdate } = req.body;

    const calculations = calculatePayroll(basicPay, loan);

    if (isUpdate === 'true') {
        // Update Query
          const query = `UPDATE employees SET name=?, basicPay=?, da=?,
hra=?, epf=?, loan=?, tax=?, grossPay=?, netPay=? WHERE empId=?`;
```

```
        const values = [name, basicPay, calculations.da, calculations.hra, calculations.epf, loan, calculations.tax, calculations.grossPay, calculations.netPay, empId];

        db.query(query, values, (err, result) => {
            if (err) return res.status(500).json({ error: err.message });
            res.json({ message: 'Employee updated successfully!' });
        });
    } else {
        // Insert Query
        const query = `INSERT INTO employees (empId, name, basicPay, da, hra, epf, loan, tax, grossPay, netPay) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)`;
        const values = [empId, name, basicPay, calculations.da, calculations.hra, calculations.epf, loan, calculations.tax, calculations.grossPay, calculations.netPay];

        db.query(query, values, (err, result) => {
            if (err) return res.status(500).json({ error: err.message });
            res.json({ message: 'Employee added successfully!' });
        });
    }
});

// 2. READ (Get all employees)
app.get('/api/employees', (req, res) => {
    db.query('SELECT * FROM employees', (err, results) => {
        if (err) return res.status(500).json({ error: err.message });
        res.json(results);
    });
});

// 3. DELETE
app.delete('/api/employees/:empId', (req, res) => {
    const empId = req.params.empId;
    db.query('DELETE FROM employees WHERE empId = ?', [empId], (err, result) => {
        if (err) return res.status(500).json({ error: err.message });
        res.json({ message: 'Employee deleted successfully!' });
    });
});

app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

Step 4: Frontend (public/index.html)

Create a public folder and inside it, create index.html. This includes the layout, forms, table, and JavaScript to handle client-side interaction.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Employee Payroll System</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
   <h2>Employee Payroll Management System</h2>

   <!-- Form Card -->
   <div class="card">
      <h3 id="formTitle">Add New Employee</h3>
      <form id="payrollForm">
         <input type="hidden" id="isUpdate" value="false">

         <div class="form-group">
            <label for="empId">Employee ID</label>
            <input type="text" id="empId" required>
         </div>

         <div class="form-group">
            <label for="name">Employee Name</label>
            <input type="text" id="name" required>
         </div>

         <div class="form-group">
            <label for="basicPay">Basic Pay</label>
            <input type="number" step="0.01" id="basicPay" required>
         </div>

         <div class="form-group">
            <label for="loan">Loan Deduction</label>
            <input type="number" step="0.01" id="loan" value="0.00">
         </div>

         <div class="btn-container">
               <button type="submit" class="btn-primary" id="saveBtn">Save
Employee</button>
                  <button type="button" class="btn-secondary" id="resetBtn"
onclick="resetForm()">Reset</button>
         </div>
      </form>
   </div>

   <!-- Data Table Card -->
```

```
    <div class="card">
        <h3>Employee Payroll Records</h3>
        <div class="table-responsive">
            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Basic Pay</th>
                        <th>DA</th>
                        <th>HRA</th>
                        <th>EPF</th>
                        <th>Loan</th>
                        <th>Tax</th>
                        <th>Gross Pay</th>
                        <th>Net Pay</th>
                        <th>Actions</th>
                    </tr>
                </thead>
                <tbody id="employeeTableBody">
                    <!-- Data loaded dynamically via JS -->
                </tbody>
            </table>
        </div>
    </div>
</div>

<script>
    const form = document.getElementById('payrollForm');
    const tableBody = document.getElementById('employeeTableBody');
    const formTitle = document.getElementById('formTitle');
    const isUpdateInput = document.getElementById('isUpdate');
    const empIdInput = document.getElementById('empId');

    async function fetchEmployees() {
        try {
            const response = await fetch('/api/employees');
            const employees = await response.json();

            tableBody.innerHTML = '';
            if (employees.length === 0) {
                tableBody.innerHTML = `<tr><td colspan="11" style="text-align:
center;">No records found.</td></tr>`;
                return;
            }

            employees.forEach(emp => {
                const row = document.createElement('tr');
                row.innerHTML = `
                    <td>${emp.empId}</td>
                    <td>${emp.name}</td>
```

```
                <td>₹${Number(emp.basicPay).toFixed(2)}</td>
                <td>₹${Number(emp.da).toFixed(2)}</td>
                <td>₹${Number(emp.hra).toFixed(2)}</td>
                <td>₹${Number(emp.epf).toFixed(2)}</td>
                <td>₹${Number(emp.loan).toFixed(2)}</td>
                <td>₹${Number(emp.tax).toFixed(2)}</td>
                 <td><strong>₹${Number(emp.grossPay).toFixed(2)}</strong></t
d>
                <td><strong>₹${Number(emp.netPay).toFixed(2)}</strong></td>
                <td class="actions">
                 <button class="btn-edit" onclick="editEmployee('${emp.empId}',
'${emp.name}', '${emp.basicPay}', '${emp.loan}')">Edit</button>
                                                        <button     class="btn-delete"
onclick="deleteEmployee('${emp.empId}')">Delete</button>
                </td>
            `;
            tableBody.appendChild(row);
        });
    } catch (error) {
        console.error('Error fetching employees:', error);
    }
}

form.addEventListener('submit', async (e) => {
    e.preventDefault();

    const data = {
        empId: empIdInput.value,
        name: document.getElementById('name').value,
        basicPay: document.getElementById('basicPay').value,
        loan: document.getElementById('loan').value,
        isUpdate: isUpdateInput.value
    };

    try {
        const response = await fetch('/api/employees', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(data)
        });

        const result = await response.json();
        alert(result.message);
        resetForm();
        fetchEmployees();
    } catch (error) {
        console.error('Error saving employee:', error);
    }
});

function editEmployee(empId, name, basicPay, loan) {
```

```
        empIdInput.value = empId;
        empIdInput.setAttribute('readonly', true);
        document.getElementById('name').value = name;
        document.getElementById('basicPay').value = basicPay;
        document.getElementById('loan').value = loan;

        isUpdateInput.value = 'true';
        formTitle.innerText = 'Update Employee Payroll';
        document.getElementById('saveBtn').innerText = 'Update Record';
        window.scrollTo({ top: 0, behavior: 'smooth' });
    }

    async function deleteEmployee(empId) {
        if (confirm(`Are you sure you want to delete employee ID: ${empId}?`)) {
            try {
                const response = await fetch(`/api/employees/${empId}`, {
                    method: 'DELETE'
                });
                const result = await response.json();
                alert(result.message);
                fetchEmployees();
            } catch (error) {
                console.error('Error deleting employee:', error);
            }
        }
    }

    function resetForm() {
        form.reset();
        empIdInput.removeAttribute('readonly');
        isUpdateInput.value = 'false';
        formTitle.innerText = 'Add New Employee';
        document.getElementById('saveBtn').innerText = 'Save Employee';
        document.getElementById('loan').value = '0.00';
    }

    fetchEmployees();
</script>

</body>
</html>
```

Frontend (public/style.css)

Create a file named style.css inside the public/ folder and paste the following styles into it:

style.css

```
:root {
    --primary-color: #4f46e5;
```

```
    --primary-hover: #4338ca;
    --bg-color: #f3f4f6;
    --card-bg: #ffffff;
    --text-color: #1f2937;
    --border-color: #e5e7eb;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: var(--bg-color);
    color: var(--text-color);
    margin: 0;
    padding: 20px;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
}

h2, h3 {
    text-align: center;
    color: var(--primary-color);
}

.card {
    background: var(--card-bg);
    padding: 25px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    margin-bottom: 30px;
}

form {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 15px;
}

.form-group {
    display: flex;
    flex-direction: column;
}

label {
    font-size: 14px;
    font-weight: 600;
    margin-bottom: 5px;
}

input {
```

```
    padding: 10px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    font-size: 14px;
}

input:focus {
    outline: none;
    border-color: var(--primary-color);
}

.btn-container {
    grid-column: 1 / -1;
    display: flex;
    gap: 10px;
    margin-top: 10px;
}

button {
    padding: 10px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-size: 14px;
    font-weight: 600;
}

.btn-primary {
    background-color: var(--primary-color);
    color: white;
}

.btn-primary:hover {
    background-color: var(--primary-hover);
}

.btn-secondary {
    background-color: #6b7280;
    color: white;
}

.btn-secondary:hover {
    background-color: #4b5563;
}

.table-responsive {
    overflow-x: auto;
}

table {
    width: 100%;
```

```
    border-collapse: collapse;
    background: var(--card-bg);
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

th, td {
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid var(--border-color);
    font-size: 13px;
}

th {
    background-color: var(--primary-color);
    color: white;
    text-transform: uppercase;
    font-size: 12px;
}

tr:hover {
    background-color: #f9fafb;
}

.actions button {
    padding: 6px 10px;
    font-size: 12px;
    margin-right: 5px;
}

.btn-edit {
    background-color: #eab308;
    color: white;
}

.btn-delete {
    background-color: #ef4444;
    color: white;
}
```

Step 5: Run the Application

1. Start your Node.js server:

   C:\NodeJSExamples\employees-MySQL-CRUD>**node server.js**
   Server is running on http://localhost:3000

2. Open your web browser and navigate to: http://localhost:3000

Overall Steps to create and run this application:

```
C:\Users\periy>cd C:\NodeJSExamples

C:\NodeJSExamples>mkdir employees-MySQL-CRUD

C:\NodeJSExamples>cd employees-MySQL-CRUD

C:\NodeJSExamples\employees-MySQL-CRUD>npm init -y
Wrote to C:\NodeJSExamples\employees-MySQL-CRUD\package.json:

{
  "name": "employees-mysql-crud",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs"
}

C:\NodeJSExamples\employees-MySQL-CRUD>npm install express mysql2
body-parser

added 76 packages, and audited 77 packages in 4s

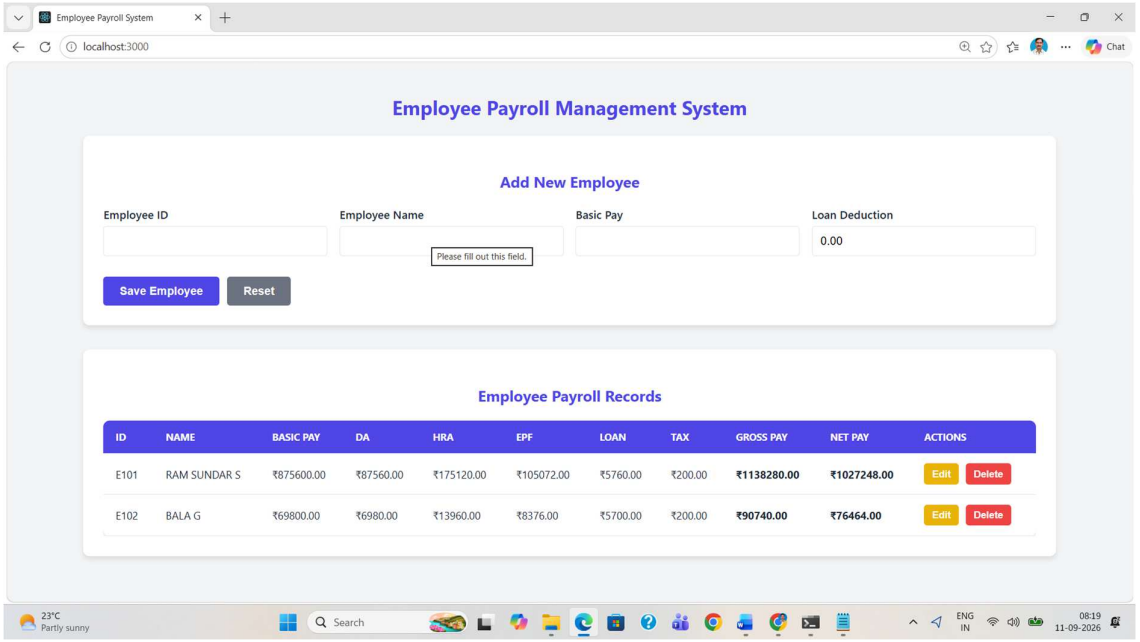
29 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

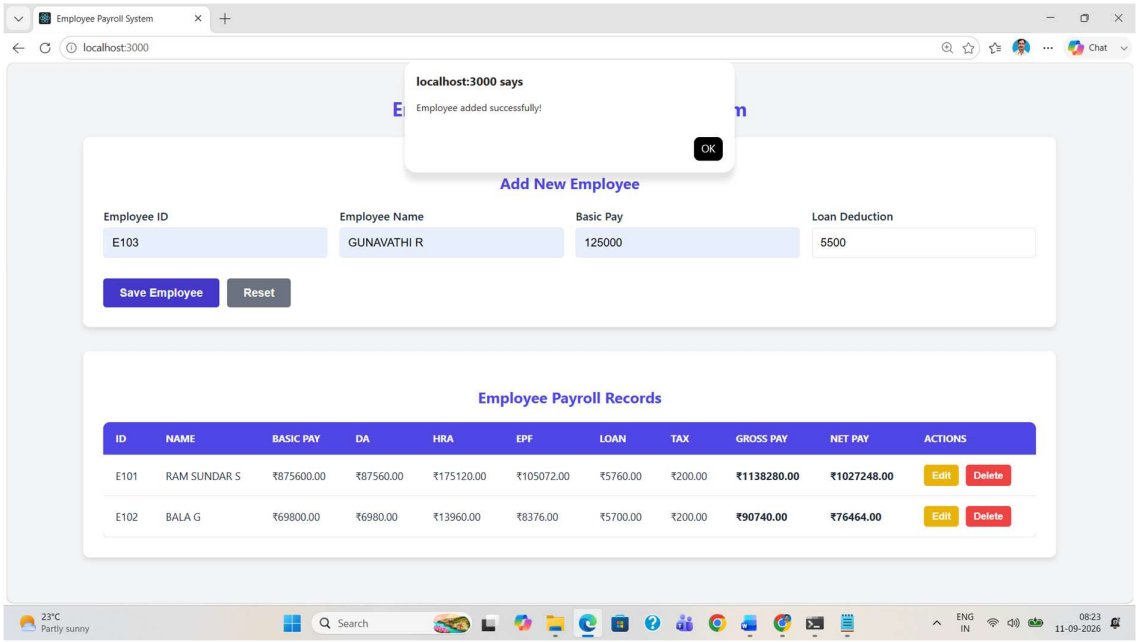
C:\NodeJSExamples\employees-MySQL-CRUD>mkdir public

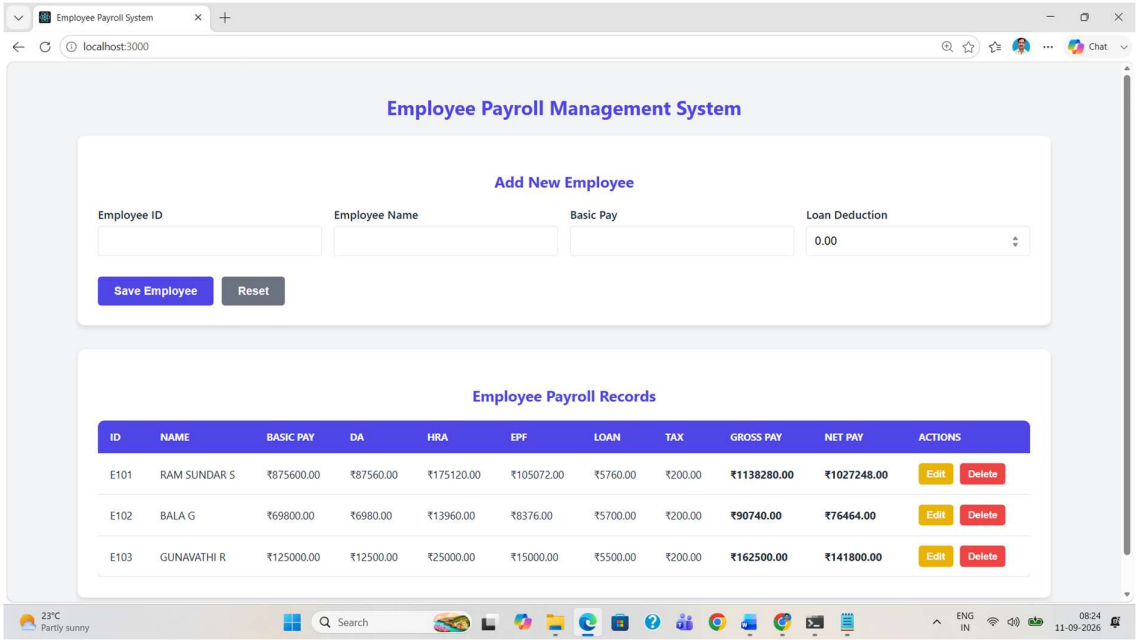
C:\NodeJSExamples\employees-MySQL-CRUD>node server.js
Server is running on http://localhost:3000
```

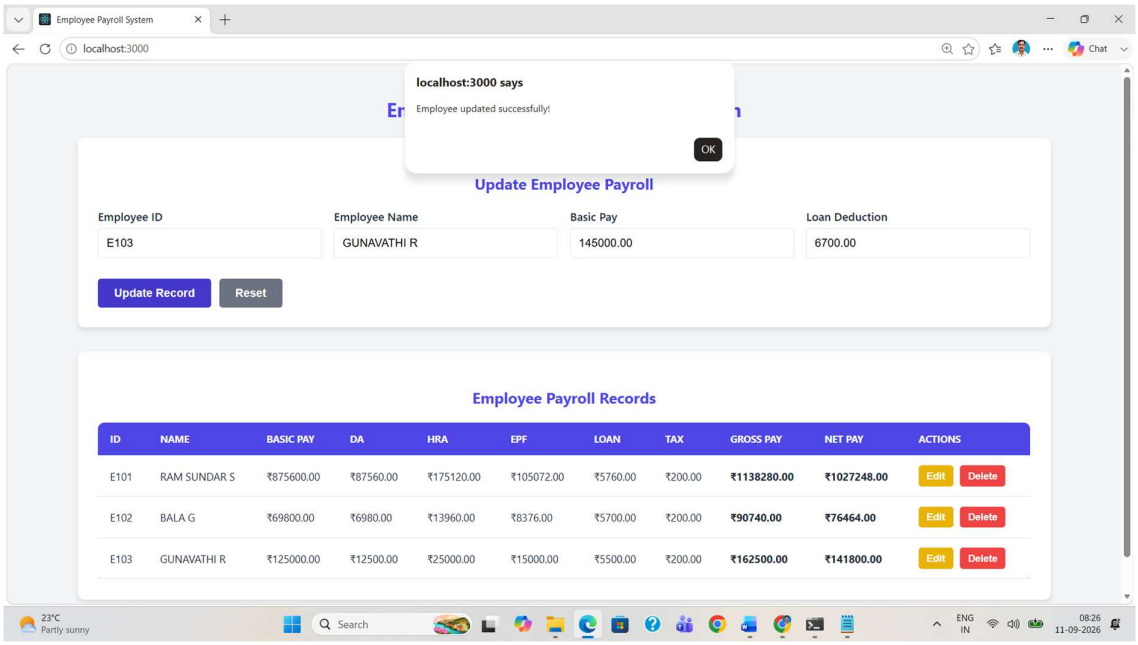
INPUT

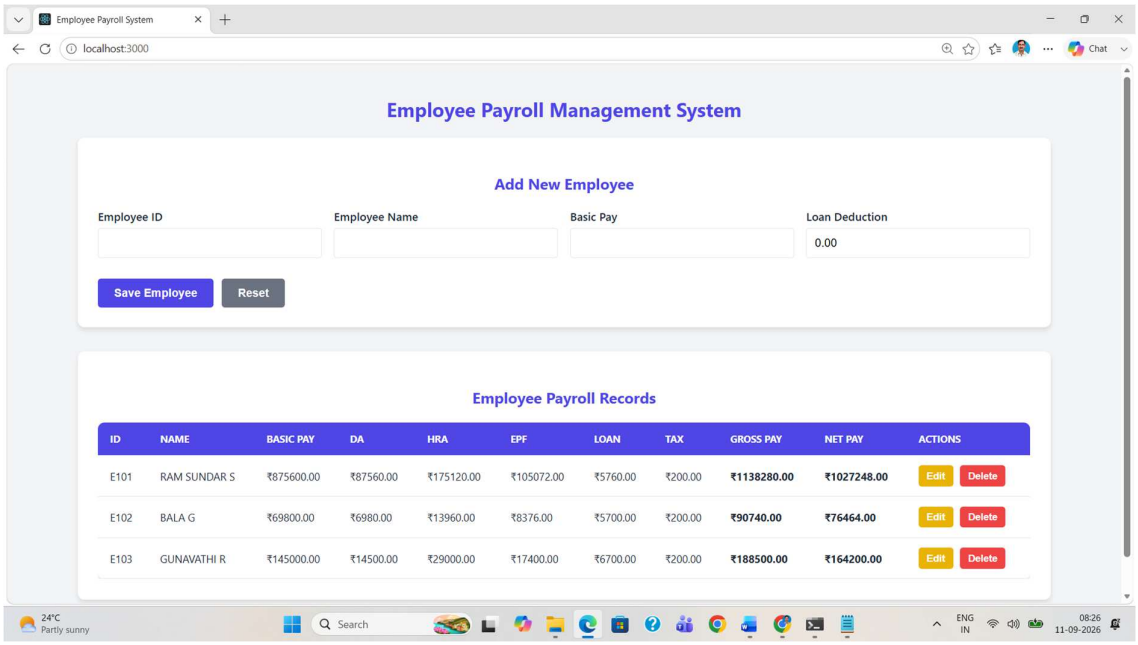


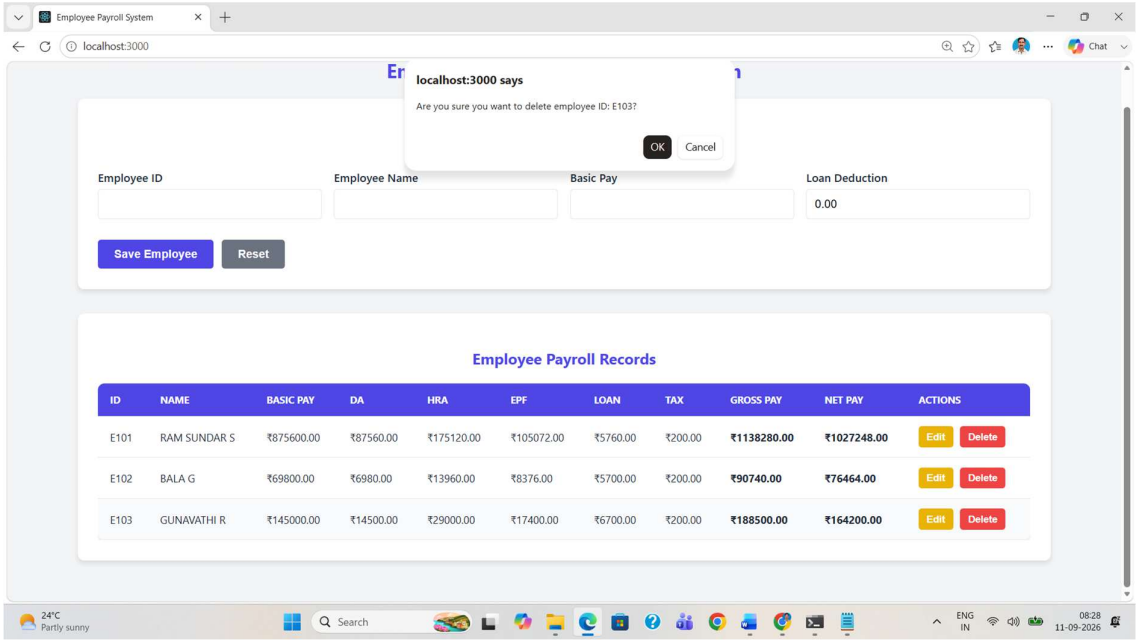
OUTPUT

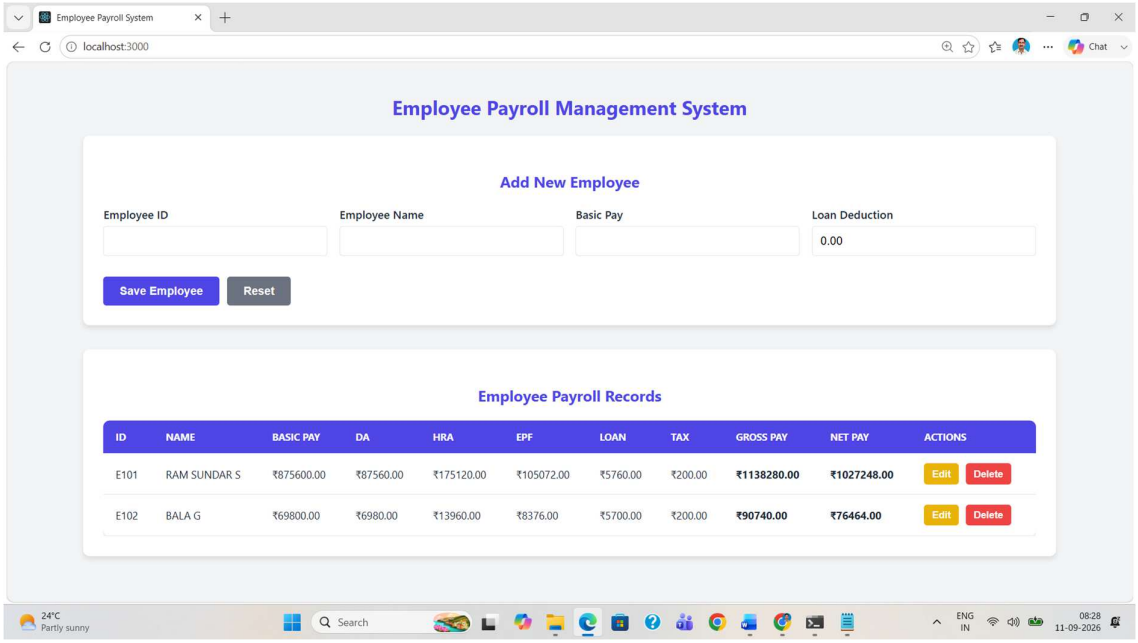



Employee Payroll System
localhost:3000
Chat
Employee Payroll Management System
Add New Employee
Employee ID
Employee Name
Basic Pay
Loan Deduction
0.00
Save Employee
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 RAM SUNDAR S ₹875600.00 ₹87560.00 ₹175120.00 ₹105072.00 ₹5760.00 ₹200.00 ₹1138280.00 ₹1027248.00 Edit Delete
E102 BALA G ₹69800.00 ₹6980.00 ₹13960.00 ₹8376.00 ₹5700.00 ₹200.00 ₹90740.00 ₹76464.00 Edit Delete
E103 GUNAVATHI R ₹125000.00 ₹12500.00 ₹25000.00 ₹15000.00 ₹5500.00 ₹200.00 ₹162500.00 ₹141800.00 Edit Delete
23°C
Partly sunny
Search
ENG
IN
08:24
11-09-2026


Employee Payroll System
localhost:3000
Chat
localhost:3000 says
Employee updated successfully!
OK
Update Employee Payroll
Employee ID
E103
Employee Name
GUNAVATHI R
Basic Pay
145000.00
Loan Deduction
6700.00
Update Record
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 RAM SUNDAR S ₹875600.00 ₹87560.00 ₹175120.00 ₹105072.00 ₹5760.00 ₹200.00 ₹1138280.00 ₹1027248.00 Edit Delete
E102 BALA G ₹69800.00 ₹6980.00 ₹13960.00 ₹8376.00 ₹5700.00 ₹200.00 ₹90740.00 ₹76464.00 Edit Delete
E103 GUNAVATHI R ₹125000.00 ₹12500.00 ₹25000.00 ₹15000.00 ₹5500.00 ₹200.00 ₹162500.00 ₹141800.00 Edit Delete
23°C
Partly sunny
Search
ENG
IN
08:26
11-09-2026

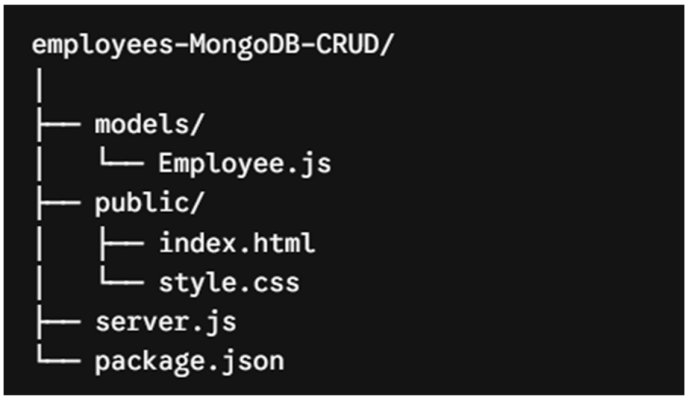

Employee Payroll System
localhost:3000
Chat
Employee Payroll Management System
Add New Employee
Employee ID
Employee Name
Basic Pay
Loan Deduction
0.00
Save Employee
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 RAM SUNDAR S ₹875600.00 ₹87560.00 ₹175120.00 ₹105072.00 ₹5760.00 ₹200.00 ₹1138280.00 ₹1027248.00 Edit Delete
E102 BALA G ₹69800.00 ₹6980.00 ₹13960.00 ₹8376.00 ₹5700.00 ₹200.00 ₹90740.00 ₹76464.00 Edit Delete
E103 GUNAVATHI R ₹145000.00 ₹14500.00 ₹29000.00 ₹17400.00 ₹6700.00 ₹200.00 ₹188500.00 ₹164200.00 Edit Delete
24°C
Partly sunny
Search
ENG
IN
08:26
11-09-2026


Employee Payroll System
localhost:3000
Chat
localhost:3000 says
Are you sure you want to delete employee ID: E103?
OK
Cancel
Employee ID
Employee Name
Basic Pay
Loan Deduction
0.00
Save Employee
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 RAM SUNDAR S ₹875600.00 ₹87560.00 ₹175120.00 ₹105072.00 ₹5760.00 ₹200.00 ₹1138280.00 ₹1027248.00 Edit Delete
E102 BALA G ₹69800.00 ₹6980.00 ₹13960.00 ₹8376.00 ₹5700.00 ₹200.00 ₹90740.00 ₹76464.00 Edit Delete
E103 GUNAVATHI R ₹145000.00 ₹14500.00 ₹29000.00 ₹17400.00 ₹6700.00 ₹200.00 ₹188500.00 ₹164200.00 Edit Delete
24°C
Partly sunny
Search
ENG
IN
08:28
11-09-2026


Employee Payroll System
localhost:3000
Chat
Employee Payroll Management System
Add New Employee
Employee ID
Employee Name
Basic Pay
Loan Deduction
0.00
Save Employee
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 RAM SUNDAR S ₹875600.00 ₹87560.00 ₹175120.00 ₹105072.00 ₹5760.00 ₹200.00 ₹1138280.00 ₹1027248.00 Edit Delete
E102 BALA G ₹69800.00 ₹6980.00 ₹13960.00 ₹8376.00 ₹5700.00 ₹200.00 ₹90740.00 ₹76464.00 Edit Delete
24°C
Partly sunny
Search
ENG
IN
08:28
11-09-2026

16. **Develop a simple CRUD application (Create, Read, Update, and Delete) using only NodeJS with HTML5 and CSS3 for employee payroll generation with Employee ID, Name, Basic Pay, DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay. DA, HRA, EPF, Loan, Professional Tax, Gross Pay, and Net Pay are calculated whenever a new employee record is inserted or updated in the MongoDB.**

Project Directory Structure



Step 1: Initialization & Dependencies

1. Create and enter the project folder:

   C:\Users\periy>**cd C:\NodeJSExamples**

   C:\NodeJSExamples>**mkdir employees-MongoDB-CRUD**

   C:\NodeJSExamples>**cd employees-MongoDB-CRUD**

2. Initialize the project:

   C:\NodeJSExamples\employees-MongoDB-CRUD>**npm init -y**
   Wrote to C:\NodeJSExamples\employees-MongoDB-CRUD\package.json:

```
{
  "name": "employees-mongodb-crud",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs"
}
```

   C:\NodeJSExamples\employees-MongoDB-CRUD>

3. Install dependencies (express, mongoose, and body-parser):

   C:\NodeJSExamples\employees-MongoDB-CRUD>**npm install express mongoose body-parser**

   added 84 packages, and audited 85 packages in 10s

   28 packages are looking for funding
     run `npm fund` for details

   found 0 vulnerabilities

   C:\NodeJSExamples\employees-MongoDB-CRUD>**mkdir models**

   C:\NodeJSExamples\employees-MongoDB-CRUD>**mkdir public**

Step 2: Mongoose Schema (models/Employee.js)

Employee.js

```
const mongoose = require('mongoose');

const employeeSchema = new mongoose.Schema({
    empId: { type: String, required: true, unique: true },
    name: { type: String, required: true },
    basicPay: { type: Number, required: true, default: 0.00 },
    da: { type: Number, default: 0.00 },
    hra: { type: Number, default: 0.00 },
    epf: { type: Number, default: 0.00 },
    loan: { type: Number, default: 0.00 },
    tax: { type: Number, default: 0.00 },
    grossPay: { type: Number, default: 0.00 },
    netPay: { type: Number, default: 0.00 }
});

module.exports = mongoose.model('Employee', employeeSchema);
```

Step 3: Backend Server (server.js)

server.js

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const path = require('path');
const Employee = require('./models/Employee');

const app = express();
const PORT = 3000;

// Middleware
```

```
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());
app.use(express.static(path.join(__dirname, 'public')));

// MongoDB Connection
mongoose.connect('mongodb://localhost:27017/payroll_db')
.then(() => {
    console.log('Connected to MongoDB');
}).catch(err => {
    console.error('MongoDB connection error:', err);
});

// Helper function for payroll calculations
function calculatePayroll(basicPay, loanInput) {
    const basic = parseFloat(basicPay) || 0;
    const loan = parseFloat(loanInput) || 0;

    const da = basic * 0.10;
    const hra = basic * 0.20;
    const epf = basic * 0.12;
    const tax = 200.00;
    const grossPay = basic + da + hra;
    const netPay = grossPay - (epf + tax + loan);

    return {
        da: Number(da.toFixed(2)),
        hra: Number(hra.toFixed(2)),
        epf: Number(epf.toFixed(2)),
        tax: Number(tax.toFixed(2)),
        grossPay: Number(grossPay.toFixed(2)),
        netPay: Number(netPay.toFixed(2))
    };
}

// 1. CREATE & UPDATE (Upsert using empId)
app.post('/api/employees', async (req, res) => {
    try {
        const { empId, name, basicPay, loan } = req.body;
        const calculations = calculatePayroll(basicPay, loan);

        const updateData = {
            name,
            basicPay: parseFloat(basicPay),
            loan: parseFloat(loan) || 0,
            ...calculations
        };

        const result = await Employee.findOneAndUpdate(
            { empId },
            updateData,
            { returnDocument: 'after', upsert: true, setDefaultsOnInsert: true }
```

```
        );

        res.json({ message: 'Employee payroll record saved successfully!', result
});
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// 2. READ (Get all employees)
app.get('/api/employees', async (req, res) => {
    try {
        const employees = await Employee.find({});
        res.json(employees);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

// 3. DELETE
app.delete('/api/employees/:empId', async (req, res) => {
    try {
        const empId = req.params.empId;
        await Employee.findOneAndDelete({ empId });
        res.json({ message: 'Employee record deleted successfully!' });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
});

app.listen(PORT, () => {
    console.log(`Server running on http://localhost:${PORT}`);
});
```

Step 4: Frontend Files

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Employee Payroll System - MongoDB</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
    <h2>Employee Payroll Management System (MongoDB)</h2>
```

```
    <div class="card">
        <h3 id="formTitle">Add New Employee</h3>
        <form id="payrollForm">
            <input type="hidden" id="isUpdate" value="false">

            <div class="form-group">
                <label for="empId">Employee ID</label>
                <input type="text" id="empId" required>
            </div>

            <div class="form-group">
                <label for="name">Employee Name</label>
                <input type="text" id="name" required>
            </div>

            <div class="form-group">
                <label for="basicPay">Basic Pay</label>
                <input type="number" step="0.01" id="basicPay" required>
            </div>

            <div class="form-group">
                <label for="loan">Loan Deduction</label>
                <input type="number" step="0.01" id="loan" value="0.00">
            </div>

            <div class="btn-container">
                    <button type="submit" class="btn-primary" id="saveBtn">Save
Employee</button>
                        <button type="button" class="btn-secondary" id="resetBtn"
onclick="resetForm()">Reset</button>
            </div>
        </form>
    </div>

    <div class="card">
        <h3>Employee Payroll Records</h3>
        <div class="table-responsive">
            <table>
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Name</th>
                        <th>Basic Pay</th>
                        <th>DA</th>
                        <th>HRA</th>
                        <th>EPF</th>
                        <th>Loan</th>
                        <th>Tax</th>
                        <th>Gross Pay</th>
                        <th>Net Pay</th>
```

```
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody id="employeeTableBody">
                <!-- Dynamic data injection -->
            </tbody>
        </table>
    </div>
</div>
</div>

<script>
    const form = document.getElementById('payrollForm');
    const tableBody = document.getElementById('employeeTableBody');
    const formTitle = document.getElementById('formTitle');
    const isUpdateInput = document.getElementById('isUpdate');
    const empIdInput = document.getElementById('empId');

    async function fetchEmployees() {
        try {
            const response = await fetch('/api/employees');
            const employees = await response.json();

            tableBody.innerHTML = '';
            if (employees.length === 0) {
                tableBody.innerHTML = `<tr><td colspan="11" style="text-align:
center;">No records found.</td></tr>`;
                return;
            }

            employees.forEach(emp => {
                const row = document.createElement('tr');
                row.innerHTML = `
                    <td>${emp.empId}</td>
                    <td>${emp.name}</td>
                    <td>₹${Number(emp.basicPay).toFixed(2)}</td>
                    <td>₹${Number(emp.da).toFixed(2)}</td>
                    <td>₹${Number(emp.hra).toFixed(2)}</td>
                    <td>₹${Number(emp.epf).toFixed(2)}</td>
                    <td>₹${Number(emp.loan).toFixed(2)}</td>
                    <td>₹${Number(emp.tax).toFixed(2)}</td>
                    <td><strong>₹${Number(emp.grossPay).toFixed(2)}</strong></t
d>
                    <td><strong>₹${Number(emp.netPay).toFixed(2)}</strong></td>
                    <td class="actions">
                     <button class="btn-edit" onclick="editEmployee('${emp.empId}',
'${emp.name}', '${emp.basicPay}', '${emp.loan}')">Edit</button>
                                                        <button     class="btn-delete"
onclick="deleteEmployee('${emp.empId}')">Delete</button>
                    </td>
                `;
```

```
            tableBody.appendChild(row);
        });
    } catch (error) {
        console.error('Error fetching employees:', error);
    }
}

form.addEventListener('submit', async (e) => {
    e.preventDefault();

    const data = {
        empId: empIdInput.value,
        name: document.getElementById('name').value,
        basicPay: document.getElementById('basicPay').value,
        loan: document.getElementById('loan').value
    };

    try {
        const response = await fetch('/api/employees', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(data)
        });

        const result = await response.json();
        alert(result.message);
        resetForm();
        fetchEmployees();
    } catch (error) {
        console.error('Error saving employee:', error);
    }
});

function editEmployee(empId, name, basicPay, loan) {
    empIdInput.value = empId;
    empIdInput.setAttribute('readonly', true);
    document.getElementById('name').value = name;
    document.getElementById('basicPay').value = basicPay;
    document.getElementById('loan').value = loan;

    formTitle.innerText = 'Update Employee Payroll';
    document.getElementById('saveBtn').innerText = 'Update Record';
    window.scrollTo({ top: 0, behavior: 'smooth' });
}

async function deleteEmployee(empId) {
    if (confirm(`Are you sure you want to delete employee ID: ${empId}?`)) {
        try {
            const response = await fetch(`/api/employees/${empId}`, {
                method: 'DELETE'
            });
```

```
                const result = await response.json();
                alert(result.message);
                fetchEmployees();
            } catch (error) {
                console.error('Error deleting employee:', error);
            }
        }
    }

    function resetForm() {
        form.reset();
        empIdInput.removeAttribute('readonly');
        formTitle.innerText = 'Add New Employee';
        document.getElementById('saveBtn').innerText = 'Save Employee';
        document.getElementById('loan').value = '0.00';
    }

    fetchEmployees();
</script>

</body>
</html>
```

public/style.css

```
:root {
    --primary-color: #059669;
    --primary-hover: #047857;
    --bg-color: #f3f4f6;
    --card-bg: #ffffff;
    --text-color: #1f2937;
    --border-color: #e5e7eb;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: var(--bg-color);
    color: var(--text-color);
    margin: 0;
    padding: 20px;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
}

h2, h3 {
    text-align: center;
    color: var(--primary-color);
```

```
}

.card {
    background: var(--card-bg);
    padding: 25px;
    border-radius: 8px;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
    margin-bottom: 30px;
}

form {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 15px;
}

.form-group {
    display: flex;
    flex-direction: column;
}

label {
    font-size: 14px;
    font-weight: 600;
    margin-bottom: 5px;
}

input {
    padding: 10px;
    border: 1px solid var(--border-color);
    border-radius: 4px;
    font-size: 14px;
}

input:focus {
    outline: none;
    border-color: var(--primary-color);
}

.btn-container {
    grid-column: 1 / -1;
    display: flex;
    gap: 10px;
    margin-top: 10px;
}

button {
    padding: 10px 20px;
    border: none;
    border-radius: 4px;
    cursor: pointer;
```

```
    font-size: 14px;
    font-weight: 600;
}

.btn-primary {
    background-color: var(--primary-color);
    color: white;
}

.btn-primary:hover {
    background-color: var(--primary-hover);
}

.btn-secondary {
    background-color: #6b7280;
    color: white;
}

.btn-secondary:hover {
    background-color: #4b5563;
}

.table-responsive {
    overflow-x: auto;
}

table {
    width: 100%;
    border-collapse: collapse;
    background: var(--card-bg);
    border-radius: 8px;
    overflow: hidden;
    box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}

th, td {
    padding: 12px 15px;
    text-align: left;
    border-bottom: 1px solid var(--border-color);
    font-size: 13px;
}

th {
    background-color: var(--primary-color);
    color: white;
    text-transform: uppercase;
    font-size: 12px;
}

tr:hover {
    background-color: #f9fafb;
```

```
}

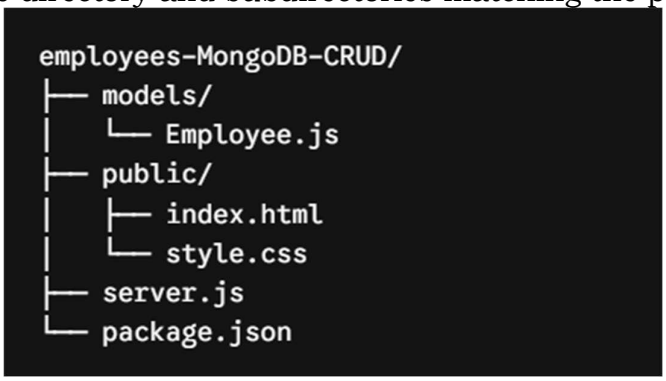
.actions button {
    padding: 6px 10px;
    font-size: 12px;
    margin-right: 5px;
}

.btn-edit {
    background-color: #eab308;
    color: white;
}

.btn-delete {
    background-color: #ef4444;
    color: white;
}
```

Steps to run this application

1. **Ensure MongoDB is Installed and Running**
   Make sure your local MongoDB service is active. You can start it via your system services or run mongod in a separate terminal window.
2. **Set Up the Project Folder Structure**
   Create the directory and subdirectories matching the project tree:



3. **Install Dependencies**
   Open your terminal, navigate into the employees-MongoDB-CRUD directory, and run the following command to install express, mongoose, and body-parser:

   C:\NodeJSExamples\employees-MongoDB-CRUD>**npm install express mongoose body-parser**

   added 84 packages, and audited 85 packages in 10s

   28 packages are looking for funding
     run `npm fund` for details

   found 0 vulnerabilities

C:\NodeJSExamples\employees-MongoDB-CRUD>

4. **Start the Node.js Server**
   Run the application from the root of your project folder:

   C:\NodeJSExamples\employees-MongoDB-CRUD>**node server.js**
   Server running on http://localhost:3000
   Connected to MongoDB

   You should see confirmation messages in the console indicating that the server is running and connected to MongoDB.

5. **Access the Application**
   Open your web browser and go to:
   http://localhost:3000

Overall Steps to create and run this application:

C:\Users\periy>**cd C:\NodeJSExamples**

C:\NodeJSExamples>**mkdir employees-MongoDB-CRUD**

C:\NodeJSExamples>**cd employees-MongoDB-CRUD**

C:\NodeJSExamples\employees-MongoDB-CRUD>**npm init -y**
Wrote to C:\NodeJSExamples\employees-MongoDB-CRUD\package.json:

```
{
  "name": "employees-mongodb-crud",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "type": "commonjs"
}
```

C:\NodeJSExamples\employees-MongoDB-CRUD>**npm install express mongoose body-parser**

added 84 packages, and audited 85 packages in 10s

28 packages are looking for funding
  run `npm fund` for details

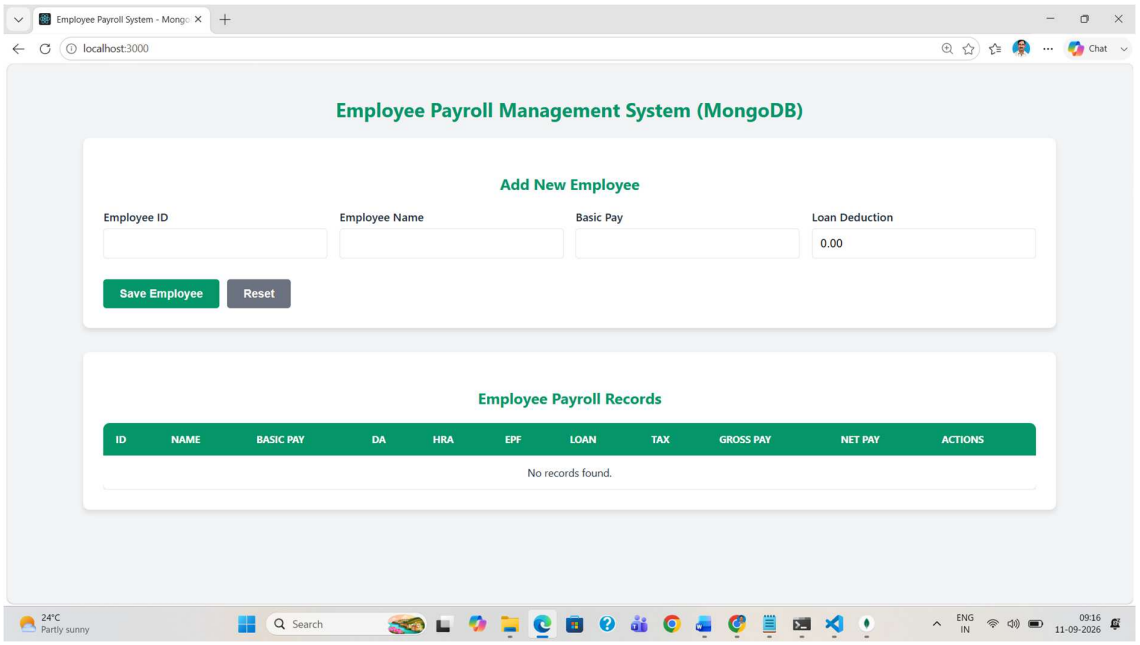
found 0 vulnerabilities

C:\NodeJSExamples\employees-MongoDB-CRUD>**mkdir models**

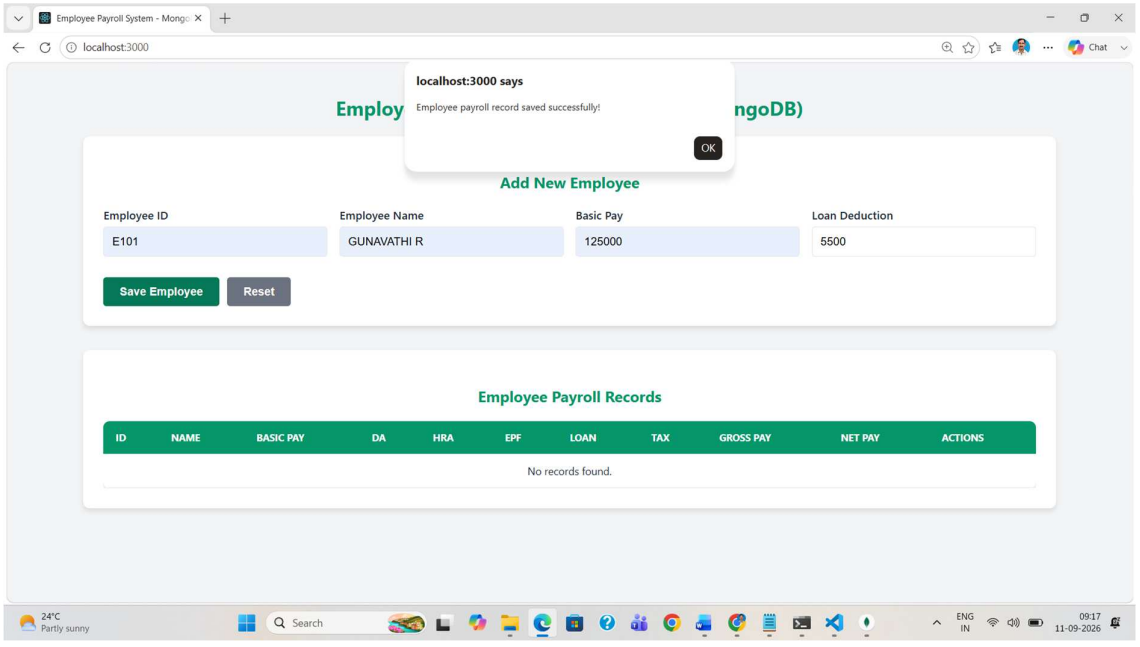
C:\NodeJSExamples\employees-MongoDB-CRUD>**mkdir public**

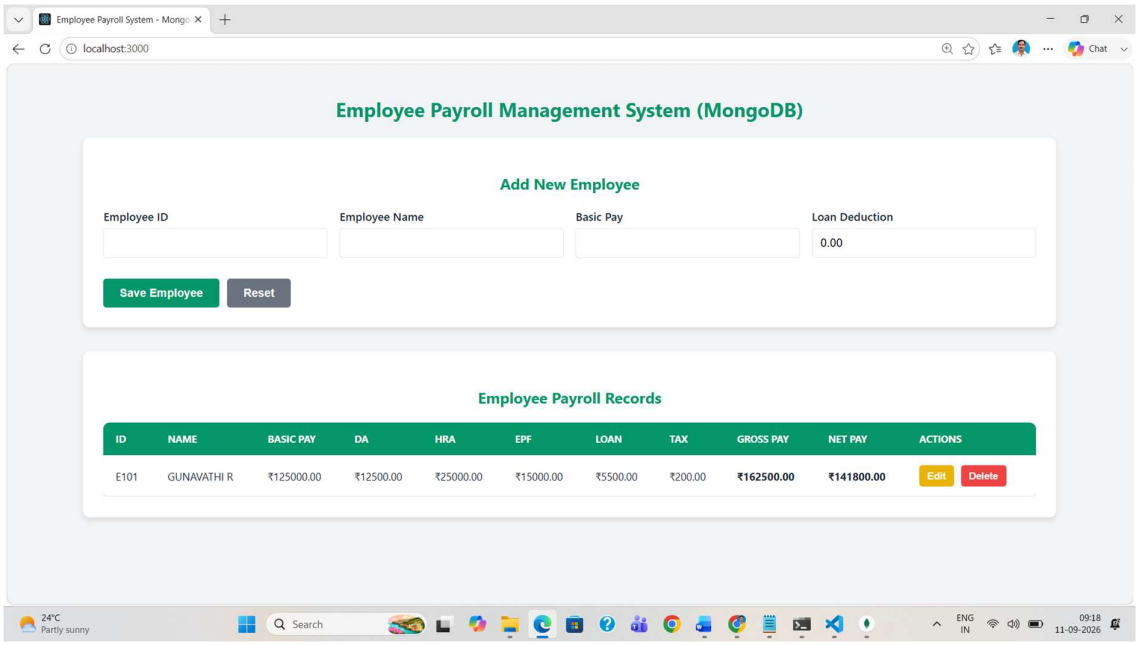
C:\NodeJSExamples\employees-MongoDB-CRUD>**node server.js**
Server running on http://localhost:3000
Connected to MongoDB

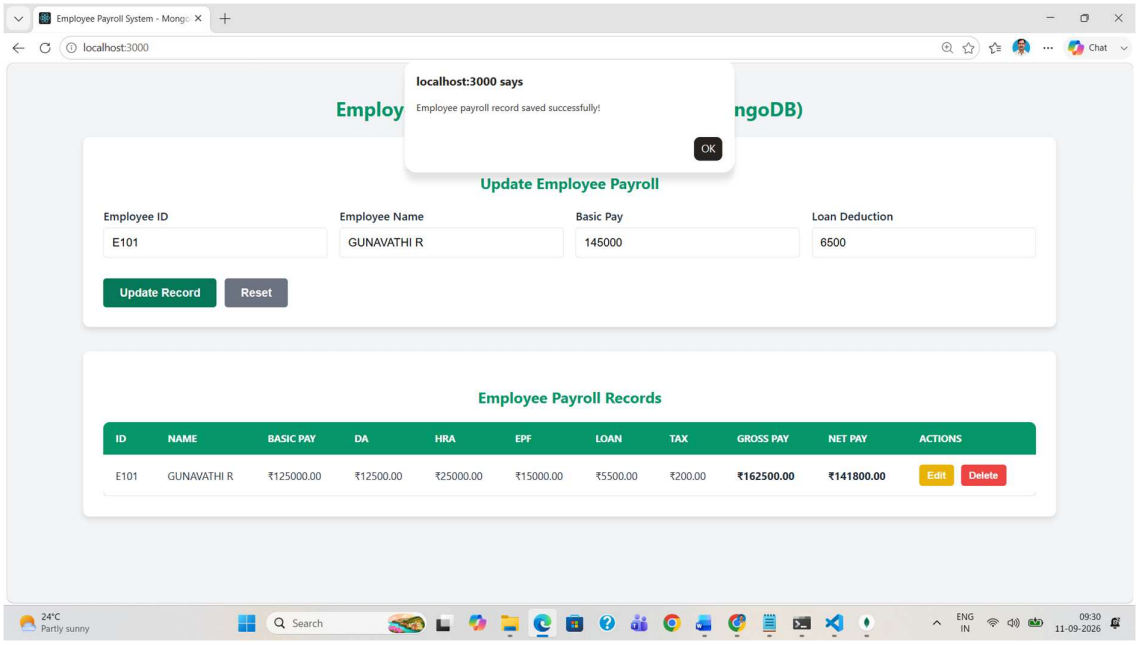
INPUT

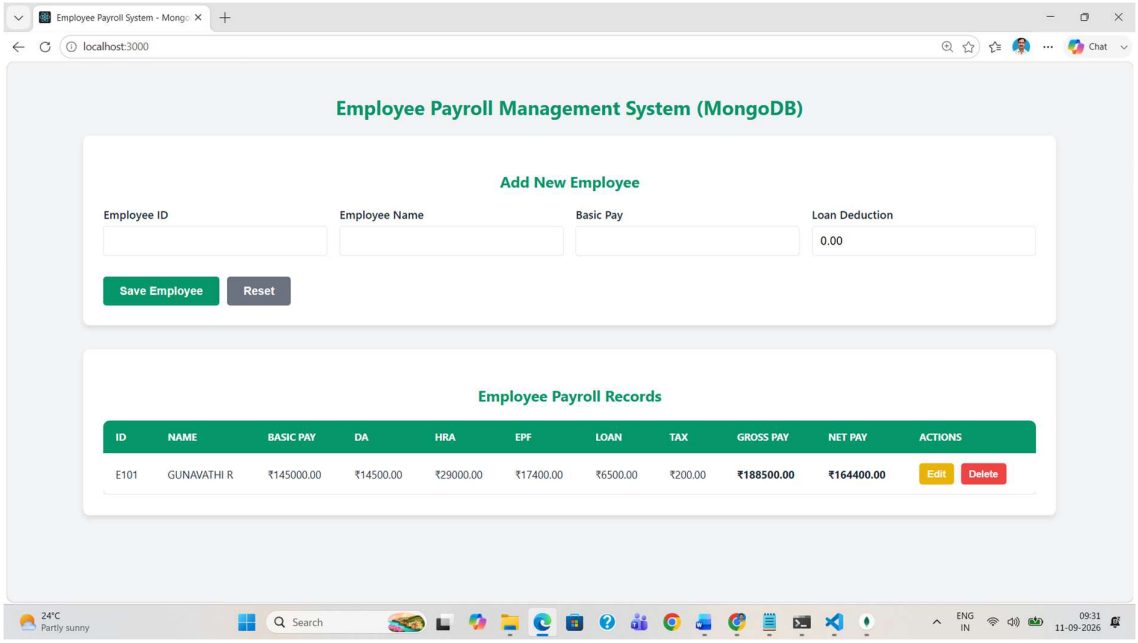


OUTPUT




Employee Payroll System - Mongo
localhost:3000
Chat
Employee Payroll Management System (MongoDB)
Add New Employee
Employee ID
Employee Name
Basic Pay
Loan Deduction
0.00
Save Employee
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 GUNAVATHI R ₹125000.00 ₹12500.00 ₹25000.00 ₹15000.00 ₹5500.00 ₹200.00 ₹162500.00 ₹141800.00 Edit Delete
24°C
Partly sunny
Search
ENG
IN
09:18
11-09-2026


Employee Payroll System - Mongo
localhost:3000
Chat
localhost:3000 says
Employee payroll record saved successfully!
OK
Employ
ngoDB)
Update Employee Payroll
Employee ID
E101
Employee Name
GUNAVATHI R
Basic Pay
145000
Loan Deduction
6500
Update Record
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 GUNAVATHI R ₹125000.00 ₹12500.00 ₹25000.00 ₹15000.00 ₹5500.00 ₹200.00 ₹162500.00 ₹141800.00 Edit Delete
24°C
Partly sunny
Search
ENG
IN
09:30
11-09-2026


Employee Payroll System - Mongo
localhost:3000
Chat
Employee Payroll Management System (MongoDB)
Add New Employee
Employee ID
Employee Name
Basic Pay
Loan Deduction
0.00
Save Employee
Reset
Employee Payroll Records
ID NAME BASIC PAY DA HRA EPF LOAN TAX GROSS PAY NET PAY ACTIONS
E101 GUNAVATHI R ₹145000.00 ₹14500.00 ₹29000.00 ₹17400.00 ₹6500.00 ₹200.00 ₹188500.00 ₹164400.00 Edit Delete
24°C
Partly sunny
Search
ENG
IN
09:31
11-09-2026